



INSTITUTO INFNET
CAMPUS RIO DE JANEIRO
ENGENHARIA DA COMPUTAÇÃO – SEXTO PERÍODO

AT ASSESSMENT

Professor: Daniel Vidal

Gabriel Carvalho

Rio de Janeiro, 20 de setembro de 2026.

GABRIEL CARVALHO

AT
ASSESSMENT

Trabalho de análise apresentado com o objetivo de aprovação na disciplina de Fundamentos de Verilog e FPGA do curso de Engenharia da Computação do Instituto INFNET.

Professor: Daniel Vidal

Rio de Janeiro

2026

Sumário

Sobre o Projeto	6
Visão Geral	6
Arquitetura do Sistema	7
Fluxograma	11
A Evolução do Desenvolvimento	12
TP1: Concepção e Prova de Conceito.....	12
TP2: Primeiros Módulos e Dados Simulados	12
TP3: Acesso a GPIO e Máquina de Estados.....	13
TP4: Sensores Reais e Revisão Arquitetural	14
TP5: Handshaking, Pré Processamento e Vetorização	15
Ajustes Finais para a AT.....	16
Conclusões Técnicas.....	17
Arquivos Técnicos – Raspberry Pi e Assembly	19
main.s.....	19
estabelece_conexao.s	23
gpio.s.....	28
atuadores.s	36
usa_lcd.s.....	38
fsm_principal.s.....	52
recebe_info.s	64
converte_temp.s	85
converte_umidade_ar.s	88
converte_umidade_solo.s.....	93
converte_luz.s	96
coeficientes_calibracao.s	97
tabelas_amstras.s	99

calcula_info.s	101
eh_dia.s	113
apresenta_info_em_tela.s.....	115
Makefile.....	121
Arquivos Técnicos - Tang Nano 9k e Verilog.....	126
top_fpga.v	126
fsm_ambiente.v.....	131
i2c_recebe_dados_temp_umiAr_pres.v.....	145
i2c_recebe_dados_luz.v	148
fsm_umiSolo.v.....	149
spi_recebe_dados_umiSolo.v	154
spi_transmite_dados.v	156
Testbenches.....	159
spi_transmite_dados_tb.v	159
Resultado	166
GTK Wave.....	168
Análise	169
fsm_umiSolo_tb.v.....	171
Resultado	174
GTK Wave.....	175
Análise	176
fsm_ambiente_tb.v.....	178
Resultado	190
GTK Wave.....	192
Análise	193
top_fpga_tb.v	196
Resultado	211

GTK Wave.....	213
Análise	214
Evidências.....	217
Código.....	217
Vídeo.....	217
Imagens do Hardware	217
Imagens das Waves.....	217

Sobre o Projeto

Visão Geral

O projeto consiste na construção de um sistema embarcado de monitoramento e controle ambiental voltado ao cultivo de um pé de manjerição, que foi adotado como objeto de teste real ao longo de todo o desenvolvimento, já que é uma planta que responde relativamente rápido a modificações ambientais. A arquitetura combina duas unidades de processamento com papéis bem distintos entre si, sendo a primeira delas uma FPGA Tang Nano 9K, programada inteiramente em Verilog, responsável apenas por adquirir e tratar eletricamente os dados vindos dos sensores, e a segunda um Raspberry Pi Zero 2W, programado inteiramente em Assembly ARM64, responsável por todo o processamento final, pela tomada de decisão propriamente dita e pelo acionamento dos atuadores físicos que cuidam da planta. Essa divisão de trabalho foi mantida como um princípio do projeto do início ao fim, de modo que a FPGA nunca chega a interpretar ou decidir nada sobre os dados que ela mesma coleta, apenas os adquire, trata e transmite adiante, enquanto toda a inteligência do sistema, ou seja, toda a parte que realmente decide se a planta precisa de água ou de luz, mora exclusivamente dentro do Raspberry Pi.

A escolha desse projeto foi motivada por dois critérios principais, sendo o primeiro deles o interesse em produzir um equipamento físico de verdade, que pudesse ser observado funcionando na prática, já que isso permite validar de forma concreta a integração entre hardware e software embarcado, algo que uma simulação sozinha não seria capaz de comprovar. O segundo critério foi o aproveitamento do paralelismo natural que uma FPGA oferece para ler vários sensores ao mesmo tempo, combinado com a flexibilidade que a programação em Assembly no processador ARM oferece para a parte de decisão e controle, o que justifica tecnicamente a divisão de responsabilidades adotada, uma vez que cada unidade de processamento acaba sendo usada exatamente para aquilo em que ela é mais forte. Funcionalidades como autonomia a bateria e notificações remotas por Wi-Fi foram conscientemente deixadas de fora do escopo desde o começo, já que o cronograma da disciplina, de vinte e duas semanas, não deixava espaço confortável para esse tipo de extensão sem colocar em risco a entrega das partes centrais do sistema.

Ao longo dos cinco Testes de Performance que compuseram o bloco, a arquitetura sofreu revisões importantes em relação ao planejamento inicial, sempre como resposta a limitações reais encontradas na prática e nunca por simples mudança de ideia. O conjunto

de atuadores, que originalmente contava com três dispositivos independentes, sendo eles uma válvula conta gotas, um módulo de luz ultravioleta e um LED de alerta, acabou sendo simplificado para apenas dois atuadores, uma bomba peristáltica responsável pela irrigação e uma fita de luz de crescimento nas cores vermelha e azul, já que essa configuração cobre de forma suficiente as necessidades reais de um pé de manjerição sem introduzir complexidade que não traria benefício prático. O display, que também havia sido planejado inicialmente como um módulo OLED, foi substituído por um display de cristal líquido de dezesseis colunas por duas linhas, conhecido como LCD 16x2, por ser mais simples de operar e por já existir suporte nativo dentro do próprio sistema operacional do Raspberry Pi para comunicação por I2C, o que evitou a necessidade de implementar esse driver manualmente do zero. Por fim, os perfis agrônômicos configuráveis por espécie de planta, que apareciam como uma possibilidade nas primeiras entregas, acabaram sendo descartados em favor de uma lógica de decisão mais direta, já que, tratando-se de uma única espécie de planta usada durante todo o projeto, a introdução de vários perfis diferentes adicionaria complexidade sem trazer nenhum benefício real ao funcionamento do sistema.

Arquitetura do Sistema

O sistema pode ser entendido como uma sequência de quatro etapas pelas quais um dado percorre, desde o momento em que sai do sensor físico até o momento em que aparece no display ou aciona um atuador. A primeira dessas etapas é a aquisição, que acontece inteiramente dentro da Tang Nano 9K e dos sensores conectados a ela, sendo responsável por ler os valores brutos de cada sensor em paralelo e já aplicar um tratamento eletrônico sobre eles antes de qualquer transmissão, sem que a FPGA em nenhum momento interprete o significado desses valores. A segunda etapa é o transporte, feito através de uma interface SPI implementada por manipulação direta de pinos, com o Raspberry Pi atuando como mestre da comunicação e a FPGA como escravo, sempre protegendo o quadro de dados enviado com um comando de leitura no início e com um byte de status e um byte de checksum ao final. A terceira etapa é o processamento, que acontece inteiramente dentro do Raspberry Pi em Assembly ARM64, e que cuida de validar o quadro recebido, converter os valores brutos para unidades reais, calcular médias sobre uma janela de tempo e, só então, decidir se algum atuador precisa ser

acionado. A quarta e última etapa é a interface com quem observa o sistema, feita através do display de cristal líquido conectado por I2C, que mostra em tempo real os valores ambientais e o estado dos atuadores.

No que diz respeito ao fluxo geral de dados, o sensor BME280, responsável pela temperatura, pela umidade do ar e pela pressão atmosférica, e o sensor BH1750, responsável pela luminosidade, compartilham o mesmo barramento I2C e são lidos por uma única máquina de estados dentro da FPGA, enquanto o sensor capacitivo de umidade do solo é lido separadamente através do conversor analógico digital MCP3008, que se comunica por SPI. Antes mesmo de qualquer transmissão para o Raspberry Pi, a FPGA já aplica um pré-processamento de suavização sobre essas leituras, sendo que um buffer circular, construído sobre blocos de memória BRAM disponíveis dentro do próprio chip, suaviza as leituras de temperatura, pressão e umidade do ar, enquanto um filtro de média móvel exponencial, construído sobre blocos de DSP também disponíveis na FPGA, trata separadamente a leitura de umidade do solo. Os valores resultantes desse pré processamento são então agregados em um quadro de doze bytes de dados, que corresponde exatamente aos cinco parâmetros ambientais medidos, e esse quadro é precedido por um comando de leitura de um único byte, com valor fixo em hexadecimal 0xA5, e seguido por um byte de status, que assume o valor hexadecimal 0x5A quando o comando é reconhecido como correto ou zero caso contrário, e por um byte de checksum, calculado como a soma simples dos doze bytes de dados. Ao todo, portanto, catorze bytes são transmitidos a cada ciclo de leitura, e ambos os lados da comunicação validam esse quadro antes de aceitar qualquer dado como válido, o que evita que uma falha de comunicação passe despercebida e seja tratada como se fosse uma leitura real do ambiente. Vale destacar também que dois LEDs físicos presentes na própria placa da FPGA tornam esse funcionamento observável a olho nu, sendo que um deles acende sempre que uma transação SPI está em andamento, enquanto o outro acende especificamente quando o byte de comando enviado pelo Raspberry Pi é reconhecido como válido, funcionando como uma espécie de indicador visual de que o aperto de mão entre as duas unidades, chamado de handshake, aconteceu com sucesso.

Assim que o Raspberry Pi recebe o quadro completo, a primeira coisa que ele faz é validar o byte de status e recalcular o checksum sobre os doze bytes de dados recebidos, descartando a leitura inteira caso qualquer uma dessas duas verificações falhe, já que aceitar um dado potencialmente corrompido seria pior do que simplesmente esperar pela próxima leitura. Superadas essas verificações, cada campo do quadro é separado e

convertido para sua unidade real através de uma rotina de conversão específica para cada sensor, e o valor já convertido é armazenado em uma tabela de amostras na memória, formando um histórico que será usado mais tarde no cálculo das médias. A cada cento e cinquenta leituras acumuladas, o que, como cada leitura acontece a cada dois segundos, corresponde a uma janela de cinco minutos inteiros, a média de cada um dos quatro parâmetros é calculada através de rotinas vetoriais NEON, uma tecnologia de instruções vetoriais disponível no processador ARM, e é apenas com base nessas médias que a lógica de decisão do sistema é finalmente executada, escrevendo o resultado tanto nos pinos GPIO dos atuadores quanto no display de cristal líquido.

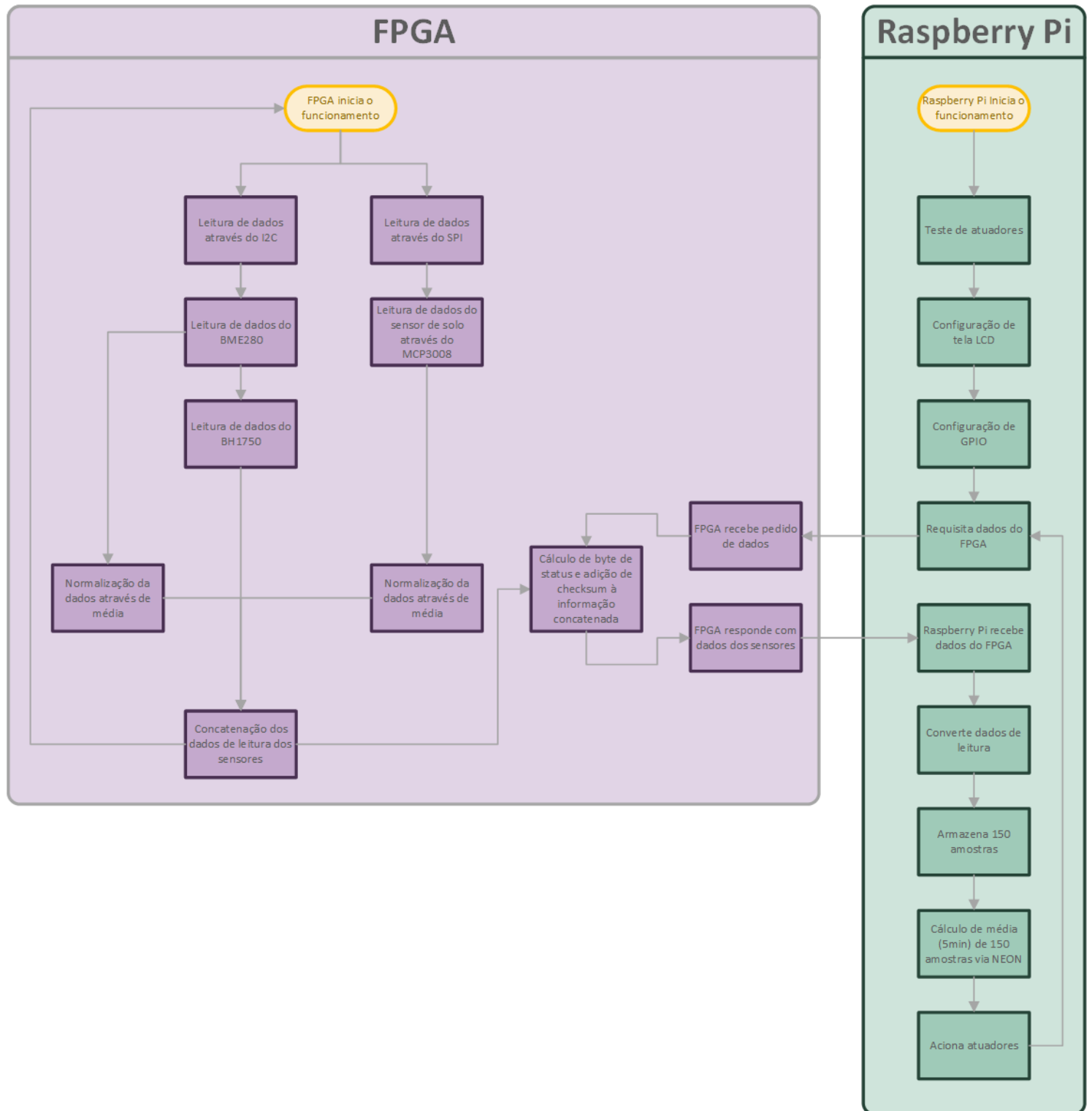
No que diz respeito ao hardware físico empregado, o Raspberry Pi Zero 2W funciona como a unidade de controle e decisão de todo o sistema, sendo programado inteiramente em Assembly ARM64, enquanto a Tang Nano 9K funciona como a unidade de aquisição dedicada, sendo programada em Verilog. O sensor BME280 é responsável por medir a umidade do ar, a temperatura e a pressão atmosférica, comunicando-se por I2C, enquanto o sensor BH1750 é um sensor digital dedicado exclusivamente à medição de luminosidade, também comunicando-se por I2C no mesmo barramento compartilhado. A umidade do solo, por sua vez, é lida através de um sensor capacitivo cujo sinal analógico é convertido para digital pelo conversor MCP3008, que se comunica por SPI diretamente com a FPGA. A interface com quem observa o sistema é feita por um display de cristal líquido de dezesseis por duas colunas, equipado com um módulo expansor PCF8574T que permite a ele se comunicar por I2C em vez de exigir vários pinos digitais separados como normalmente aconteceria com esse tipo de display. Por fim, os dois atuadores físicos do sistema são a bomba peristáltica modelo G328, usada para a irrigação, e uma fita de luz de crescimento nas cores vermelha e azul, usada para compensar a falta de luminosidade natural durante o dia.

No que se refere à lógica de decisão propriamente dita, a decisão de irrigar ou de acionar a iluminação artificial é tomada por uma comparação direta entre a média de cada parâmetro, já calculada sobre a janela de cinco minutos, e limiares fixos definidos a partir de referências agronômicas específicas para a cultura do manjeriço. O motor de decisão também mantém um estado entre um ciclo e outro, através de uma variável que guarda se a luz de crescimento está ligada ou desligada naquele momento, evitando assim que o atuador seja acionado ou desligado de forma redundante quando a condição que motivou a decisão anterior ainda continua verdadeira. Existe também uma regra de prioridade entre os dois atuadores, de modo que a necessidade de luz artificial é avaliada e, se for o caso,

acionada antes de qualquer decisão sobre irrigação, que só é resolvida depois, com base na média de umidade do solo. A irrigação, por sua vez, é aplicada como um pulso único de duração fixa, de quinze segundos.

O uso de instruções vetoriais NEON no cálculo das médias, que já era cogitado desde as primeiras entregas do projeto, acabou sendo implementado com duas variantes, uma trabalhando com números inteiros e outra com ponto flutuante, e ambas foram comparadas por um benchmark próprio, construído com a chamada de sistema `clock_gettime`, contra uma versão puramente escalar equivalente. O resultado confirmou a expectativa teórica, já que a versão vetorial inteira executa cerca de quatro vezes mais rápido do que a versão escalar, apesar de esse ganho não ser perceptível no funcionamento do sistema como um todo, uma vez que o processamento das amostras já acontecia em um tempo muito pequeno se comparado à janela de dois segundos disponível entre uma leitura e outra. A justificativa para essa implementação, portanto, não é de desempenho real, mas sim de demonstração da técnica de vetorização exigida nessa etapa da disciplina. Por fim, o código Assembly do Raspberry Pi encontra-se organizado em vários arquivos separados por tema, agrupados em quatro bibliotecas estáticas, sendo elas `libaritmética`, `libio`, `libparsing` e `libutilidades`, e todo o processo de compilação é conduzido por um único `Makefile`.

Fluxograma



A Evolução do Desenvolvimento

TP1: Concepção e Prova de Conceito

A primeira etapa do projeto, entregue em maio de 2026, foi dedicada a desenhar a arquitetura no papel e a confirmar que as duas cadeias de ferramentas escolhidas estavam corretamente instaladas e funcionando, já que nenhuma linha de código real do sistema final havia sido escrita ainda. Nesse momento, o sistema ainda previa comunicação por UART entre a FPGA e o Raspberry Pi, um conjunto de três atuadores, sendo eles a válvula conta gotas, a luz ultravioleta e o LED de alerta, e um display OLED para leitura em tempo real dos dados ambientais, arquitetura essa que viria a ser revista de forma significativa mais adiante. Como prova de conceito do lado da FPGA, foi implementado um módulo Verilog puramente combinacional, representando a porta lógica q igual a a ou não b e c , validado em um testbench através da dupla de ferramentas Icarus Verilog e GTKWave, confirmando que a simulação e a visualização de formas de onda estavam operacionais. Do lado do Raspberry Pi, foi escrito um programa de hello world em Assembly ARM64, usando a chamada de sistema de escrita para imprimir uma mensagem no terminal e a chamada de saída para encerrar o programa corretamente, o que serviu para confirmar que a toolchain aarch64 linux gnu estava funcionando sobre o sistema operacional ARM64 instalado na placa. Nenhum problema técnico relevante surgiu ao longo dessa etapa, já que seu propósito era estritamente o de preparar o terreno para o desenvolvimento incremental que viria a acontecer nos testes de performance seguintes.

TP2: Primeiros Módulos e Dados Simulados

Já em junho de 2026, essa segunda etapa trouxe os primeiros módulos com lógica de processamento real, apesar de os sensores físicos do projeto ainda não terem chegado. Do lado da FPGA, foi implementado um módulo chamado `media_amstras`, responsável por calcular a média entre duas amostras consecutivas de um sensor através de lógica sequencial disparada por borda de subida do clock, representando assim o primeiro estágio de pré processamento que seria expandido bastante nas etapas seguintes, e também um módulo multiplexador quatro para um, chamado `mux_sensores`, responsável por selecionar qual dentre quatro canais de sensor seria o canal transmitido, sendo esse

segundo módulo sintetizado fisicamente na Tang Nano 9K e mapeado nos botões físicos já presentes na própria placa. Do lado do Raspberry Pi, foi construído o primeiro motor de decisão de irrigação em Assembly, que percorria em um laço um vetor de cinco leituras de umidade do solo guardadas na seção de dados do programa, comparando cada uma delas contra dois limiares agronômicos fixos, de trinta e cinco por cento e de sessenta por cento, e imprimindo no terminal uma entre três decisões possíveis, sendo elas irrigar, manter o estado ou tratar o solo como criticamente seco. Esse comportamento foi verificado tanto pela saída impressa no terminal quanto por uma sessão de depuração com o GDB, na qual pontos de parada, chamados de breakpoints, foram colocados em cada rótulo relevante da rotina para observar o valor dos registradores a cada iteração do laço. O risco identificado desde o TP1, relativo ao atraso na importação internacional dos sensores, acabou se materializando exatamente nessa etapa, e por isso tanto o testbench Verilog quanto o programa Assembly precisaram operar sobre dados simulados, ou mockados, em vez de leituras reais, estratégia essa que já havia sido prevista como mitigação na tabela de riscos entregue no TP1.

TP3: Acesso a GPIO e Máquina de Estados

Com os sensores ainda em trânsito no momento dessa terceira entrega, feita em agosto de 2026, o esforço dessa etapa foi direcionado para a construção da infraestrutura de acesso direto a hardware que seria reaproveitada por todo o restante do projeto. Foi implementado o arquivo `gpio.s`, que mapeia o arquivo especial de dispositivo `dev/gpiomem` através da chamada de sistema `mmap`, expondo em seguida rotinas simples de configuração de pino como entrada ou saída e de leitura e escrita de nível lógico, sendo esse comportamento demonstrado com o uso de um LED comum e de um botão de teste, e não com um sensor real, já que os sensores efetivamente usados no projeto seriam lidos mais adiante por SPI e por I2C, e não como entradas digitais simples de GPIO. O motor de decisão evoluiu bastante nessa etapa, passando a combinar quatro estruturas de controle distintas dentro de um único programa, sendo elas um laço percorrendo uma lista de pacotes simulados, uma tabela de saltos usada para despachar o processamento de cada pacote conforme o tipo de sensor de origem, e um bloco de decisão com múltiplas condições, capaz de resultar em quatro ações possíveis, sendo elas irrigação urgente, irrigação padrão, ausência de irrigação ou manutenção do estado atual, todas elas

aplicadas aos atuadores através das rotinas de GPIO já validadas na primeira parte dessa mesma entrega. Do lado da FPGA, foi implementada a primeira máquina de estados finitos de verdade do projeto, seguindo o estilo conhecido como Moore, no qual a saída depende exclusivamente do estado atual e nunca diretamente das entradas, alternando entre os três estados ocioso, amostra e transmite, e controlando assim o ciclo completo de aquisição para os dois sensores que já estavam ativos naquele momento, sendo esse comportamento validado em um testbench que reproduziu dois ciclos completos de leitura, confirmando tanto a transição correta entre todos os estados quanto a alternância adequada entre os dois canais de sensor disponíveis.

TP4: Sensores Reais e Revisão Arquitetural

Essa quarta etapa, entregue em setembro de 2026, foi a de maior revisão arquitetural de todo o projeto. Ao tentar migrar a comunicação de UART para SPI, conforme havia sido sugerido pelo próprio professor em aula por considerar essa opção mais simples e mais interessante do ponto de vista didático, foi identificado um dano físico no pino GPIO onze do Raspberry Pi utilizado, que é justamente o pino dedicado ao sinal de clock do periférico SPI de hardware da placa. A solução encontrada para contornar esse problema foi reimplementar o protocolo inteiramente por software, através de manipulação direta dos registradores GPFSEL, GPSET e GPCLR, usando os pinos GPIO cinco, seis, doze e treze no lugar do periférico de hardware danificado, com o Raspberry Pi continuando a atuar como mestre da comunicação. Foi justamente nessa mesma etapa que os sensores reais finalmente chegaram e puderam ser integrados ao sistema, sendo o BME280 e o BH1750 conectados a um único barramento I2C compartilhado dentro da FPGA e lidos por uma única máquina de estados, enquanto o sensor capacitivo de umidade do solo passou a ser lido através do conversor MCP3008 por SPI, canal esse em que um erro de alinhamento de bits na janela de captura foi identificado e corrigido ainda dentro dessa mesma entrega. O conjunto de atuadores foi reduzido de três para dois dispositivos, restando apenas a bomba peristáltica e a fita de luz de crescimento, já que essa configuração já cobria de forma suficiente as necessidades reais de um único pé de manjerição, e o display OLED planejado originalmente foi substituído por um LCD de dezesseis por duas colunas, operado através do driver de kernel nativo do próprio Linux para comunicação I2C, o que eliminou a necessidade de

implementar manualmente o protocolo HD44780 usado por esse tipo de display. A lógica de decisão do Raspberry Pi também passou a ser controlada por uma máquina de estados de seis estados, com uma janela de médias de cinco minutos, e a vetorização usando instruções SIMD, que havia sido prevista desde o planejamento inicial do TP1, foi conscientemente adiada nesse momento, já que se considerou que o volume de dados processado a cada ciclo era pequeno demais para justificar a complexidade adicional que essa técnica exigiria, decisão essa que viria a ser revista já na etapa seguinte. Além do que consta formalmente no relatório entregue, duas outras dificuldades de hardware precisaram ser resolvidas ao longo dessa fase, sendo a primeira delas uma inversão de fiação entre os pinos GPIO dezessete e GPIO vinte e sete, que inicialmente foi confundida com um defeito de software antes de ser identificada como um problema puramente físico, e a segunda a constatação de que o display LCD exige alimentação de cinco volts mesmo operando suas linhas de comunicação I2C em lógica de três vírgula três volts, sendo que uma falha intermitente de confirmação, chamada de ACK, na escrita I2C do display também precisou ser contornada através de uma rotina de repetição que insiste até obter sucesso.

TP5: Handshaking, Pré Processamento e Vetorização

Nessa quinta e última entrega antes do fechamento final do bloco, feita em setembro de 2026, o protocolo SPI foi consolidado em sua forma definitiva, passando a ser composto por um quadro de catorze bytes, formado por um comando de leitura, doze bytes de dados, um byte de status e um byte de checksum, sendo esse quadro validado nas duas pontas da comunicação antes de qualquer processamento posterior. A FPGA passou a incorporar dois blocos de pré-processamento que já haviam sido previstos desde o planejamento original do TP1, mas que só foram de fato implementados nessa etapa, sendo o primeiro deles um buffer circular construído sobre blocos de memória BRAM, suavizando as leituras de temperatura, pressão e umidade do ar, e o segundo um filtro de média móvel exponencial construído sobre blocos de DSP, tratando especificamente a leitura de umidade do solo. Do lado do Raspberry Pi, a vetorização que havia sido adiada no TP4 foi finalmente implementada, com duas variantes de instruções NEON para a rotina de cálculo de médias, uma trabalhando com números inteiros e outra com ponto flutuante, ambas comparadas por um benchmark próprio à versão puramente escalar

equivalente, através da chamada de sistema `clock_gettime`. O resultado confirmou a expectativa teórica levantada já no TP4, mostrando que a versão vetorial inteira executa cerca de quatro vezes mais rápido do que a escalar, apesar de esse ganho não ser perceptível no funcionamento geral do sistema, já que o processamento das amostras já acontecia em um tempo desprezível se comparado à janela de dois segundos disponível entre uma leitura e outra, de modo que a justificativa real para essa implementação passou a ser a demonstração da técnica de vetorização exigida nessa etapa da disciplina, e não uma necessidade prática de desempenho. O código Assembly também foi reorganizado nessa entrega, deixando de ser um conjunto plano de arquivos soltos para se tornar quatro bibliotecas estáticas organizadas por tema, ligadas por um único Makefile, e o módulo de topo da FPGA passou a expor dois LEDs físicos de status, tornando o handshaking entre as duas unidades observável diretamente na própria placa.

Ajustes Finais para a AT

Depois da entrega do TP5, um conjunto de ajustes foi realizado para o fechamento definitivo do projeto, sem que a arquitetura descrita anteriormente precisasse ser alterada em nada. O sensor capacitivo de umidade do solo originalmente utilizado foi confirmado como defeituoso através de um teste controlado, no qual o mesmo sensor foi submerso em água e depois exposto ao ar livre, revelando leituras praticamente idênticas nas duas condições, o que evidenciava que o componente simplesmente não respondia à variação real de umidade, e por isso ele foi fisicamente substituído por uma segunda unidade. As constantes de calibração associadas a esse sensor, tanto o limiar bruto lido diretamente do conversor analógico digital quanto o limiar percentual usado na máquina de estados principal, foram recalibradas para refletir a faixa de resposta real do novo componente instalado, passando o limiar bruto de seiscentos e quarenta e quatro para seiscentos, e o limiar percentual de trinta e cinco por cento, valor esse originalmente documentado no TP5, para setenta por cento na versão final. A prioridade entre os dois atuadores também foi invertida nessa fase final, de modo que a decisão sobre a luz de crescimento passou a ser avaliada e aplicada antes da decisão sobre irrigação, e não depois, como estava na implementação original documentada no TP5.

Um ponto que merece destaque especial nesse fechamento é a revisão das testbenches de simulação da FPGA, já que a comparação direta entre o que estava

documentado no TP5 e o que de fato está na entrega final revelou uma diferença bem maior do que apenas um ajuste pontual. O módulo `spi_transmite_dados`, responsável pelo lado escravo da comunicação SPI, já implementava o protocolo completo de handshaking desde o TP5, incluindo a validação do comando, o byte de status e o checksum, porém as duas testbenches responsáveis por validar esse módulo, sendo elas `spi_transmite_dados_tb` e `top_fpga_tb`, ainda estavam testando uma versão antiga do protocolo, com um quadro de apenas doze bytes, sem enviar comando algum e sem verificar status, checksum ou o LED indicador de comando reconhecido, o que significa que aquelas testbenches, tal como documentadas no TP5, simplesmente não validavam a parte mais importante do protocolo já implementado no próprio módulo. Diante disso, as duas testbenches precisaram ser reescritas de forma bem mais ampla do que uma simples correção pontual, passando a enviar o byte de comando através da linha mosi em cada transação simulada, a verificar o status e o checksum recebidos através de novas tarefas de verificação, a observar o comportamento do LED de comando válido antes, durante e depois de uma transação, e a incluir dois casos de teste inteiramente novos, sendo o primeiro um comando quase correto, que difere do valor esperado por um único bit, e o segundo a corrupção deliberada de um bit de dado já recebido, usada para confirmar que o checksum recalculado no lado do Raspberry Pi de fato acusa essa divergência. Além dessas duas testbenches, um terceiro ajuste mais simples também foi aplicado à testbench `fsm_umiSolo_tb`, na qual o valor de teste que representava um solo tipicamente seco foi atualizado de seiscentos e quarenta e quatro para quinhentos e noventa e cinco, acompanhando a mesma recalibração feita depois da substituição física do sensor de solo. Por fim, o histórico do repositório Git foi completado com tags retroativas apontando para os commits de entrega do TP1 e do TP3, que ainda não possuíam essa marcação, e um arquivo README foi adicionado à raiz do projeto, documentando de forma resumida a estrutura geral do repositório.

Conclusões Técnicas

Em termos de desempenho, o sistema opera com uma folga bastante confortável dentro dos seus próprios limites de tempo, já que o ciclo de leitura de dois segundos entre uma amostra e outra absorve sem qualquer dificuldade tanto o processamento escalar quanto o vetorial das médias, e o ganho de aproximadamente quatro vezes obtido com as

rotinas NEON, apesar de real e mensurável através do benchmark próprio, não chega a se traduzir em uma mudança perceptível no comportamento do sistema como um todo, cumprindo antes um papel de demonstração da técnica do que de necessidade prática de desempenho. O protocolo SPI final, com seu quadro de catorze bytes validado por status e checksum em ambas as pontas da comunicação, se mostrou robusto o suficiente para detectar tanto um comando quase correto quanto a corrupção de um único bit em trânsito, sem impor nenhuma sobrecarga perceptível ao ciclo normal de leitura.

Entre as limitações que foram conscientemente aceitas ao longo do projeto, destaca-se a ausência de perfis agronômicos configuráveis por espécie de planta, já que o sistema foi desenvolvido e calibrado especificamente para o cultivo de manjerição, o tratamento de fuso horário fixo, sem qualquer ajuste automático para o horário de verão europeu, e o acionamento da irrigação por um pulso único de duração fixa, sem histerese entre dois limiares diferentes, o que privilegia a simplicidade de implementação em detrimento de uma resposta mais adaptativa a pequenas oscilações do sensor perto do limiar de decisão. A ausência de autonomia a bateria e de notificações remotas por Wi-Fi permanece também como uma limitação de escopo, excluída conscientemente já desde o TP1 para manter o projeto viável dentro do cronograma de vinte e duas semanas da disciplina.

Como sugestões de evolução futura para esse sistema, destaca-se a possibilidade de reintroduzir perfis agronômicos configuráveis, já que o núcleo central de aquisição, transporte e decisão está agora estável e bem testado, o que tornaria essa extensão significativamente menos arriscada do que seria se tentada nas primeiras etapas do projeto. Também seria interessante substituir o pulso único de irrigação por uma lógica de histerese entre dois limiares distintos, reduzindo o risco de acionamentos repetidos em condições de leitura instável, além de implementar o tratamento automático de horário de verão na rotina que calcula se é dia ou noite. Por fim, o uso de instruções vetoriais NEON poderia ser estendido a outras rotinas de processamento do sistema, caso o volume de dados processado a cada ciclo venha a crescer o suficiente no futuro para que o ganho de desempenho observado no benchmark passe a ter relevância prática real.

Arquivos Técnicos – Raspberry Pi e Assembly

main.s

```
.global _start
.extern estabelece_conexao
.extern fsm_principal
.extern lcd_inicializa
.extern inicializa_atuadores
.extern aciona_bomba
.extern aciona_luz
.extern gpio_unmap
.extern calcula_media_benchmark

.equ SYS_WRITE, 64
.equ SYS_EXIT, 93
.equ SYS_NANOSLEEP, 101

.section .data
msg_inicio: .asciz "\n=== Leitura do BME280 via I2C (Tang Nano) -> SPI -> Raspberry Pi ===\n"
msg_inicio_fim = . - msg_inicio - 1

msg_erro_conexao: .asciz "ERRO: nao foi possivel mapear o GPIO (/dev/gpiomem).\n"
msg_erro_conexao_fim = . - msg_erro_conexao - 1

msg_teste_leds: .asciz "[TESTE] Ligando os dois atuadores (GPIO17 e GPIO27) por 5 segundos...\n"
msg_teste_leds_fim = . - msg_teste_leds - 1

.align 8
tempo_teste_leds:
```

.quad 5

.quad 0

.section .text

_start:

mov x0, #1

ldr x1, =msg_inicio

mov x2, #msg_inicio_fim

mov x8, #SYS_WRITE

svc #0

bl calcula_media_benchmark

bl estabelece_conexao

cmp x1, #0

b.ne erro_geral

mov x19, x0

bl lcd_inicializa

mov x0, x19

bl inicializa_atuadores

mov x0, #1

ldr x1, =msg_teste_leds

mov x2, #msg_teste_leds_fim

mov x8, #SYS_WRITE

svc #0

mov x0, x19

mov x1, #1

```
bl aciona_bomba
```

```
mov x0, x19
```

```
mov x1, #1
```

```
bl aciona_luz
```

```
ldr x0, =tempo_teste_leds
```

```
mov x1, #0
```

```
mov x8, #SYS_NANOSLEEP
```

```
svc #0
```

```
mov x0, x19
```

```
mov x1, #0
```

```
bl aciona_bomba
```

```
mov x0, x19
```

```
mov x1, #0
```

```
bl aciona_luz
```

```
mov x0, x19
```

```
bl fsm_principal
```

```
erro_geral:
```

```
mov x0, #1
```

```
ldr x1, =msg_erro_conexao
```

```
mov x2, #msg_erro_conexao_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, #1
```

```
mov x8, #SYS_EXIT
```

```
svc #0
```

Este arquivo contém o ponto de entrada do programa inteiro, identificado pelo rótulo `_start`, que é o único ponto que o sistema operacional Linux realmente chama quando o executável é iniciado, e a partir dele o programa nunca mais retorna a nenhum lugar de fora. Logo na primeira instrução, uma mensagem inicial é impressa no terminal através da chamada de sistema de escrita, informando que a leitura do BME280 pela FPGA está prestes a começar, e essa mensagem é seguida imediatamente por uma chamada à rotina `calcula_media_benchmark`, que roda sozinha, sem depender de nenhum outro módulo, e mede o tempo de execução dos três métodos de cálculo de média discutidos ao longo do projeto antes de qualquer sensor ser sequer consultado.

Depois do benchmark, a rotina `estabelece_conexao` é chamada para mapear o GPIO e preparar os pinos usados pela interface SPI, e o resultado dessa chamada é verificado através do registrador `x1`, que funciona como um código de erro, sendo zero em caso de sucesso. Caso esse valor seja diferente de zero, o fluxo desvia imediatamente para o rótulo `erro_geral`, sem tentar nada mais, o que evita que o programa continue tentando ligar atuadores ou escrever no display sobre um barramento que nunca foi corretamente mapeado. Em caso de sucesso, o ponteiro de GPIO retornado em `x0` é guardado em `x19`, um registrador salvo entre chamadas, já que ele precisa sobreviver a todas as chamadas seguintes até o fim da função.

Na sequência, o display de cristal líquido é inicializado através de `lcd_inicializa`, e os dois atuadores são preparados através de `inicializa_atuadores`, que configura os pinos correspondentes à bomba e à luz de crescimento como saída. Depois disso, o programa realiza um autoteste de cinco segundos, imprimindo um aviso no terminal e ligando os dois atuadores ao mesmo tempo através de `aciona_bomba` e `aciona_luz`, aguardando o tempo definido na estrutura `tempo_teste_leds` através da chamada de sistema de `nanosleep`, e então desligando os dois novamente antes de prosseguir. Esse autoteste serve como uma confirmação visual rápida, sem depender de nenhuma leitura de sensor, de que a fiação dos dois atuadores está correta antes de o sistema começar a operar de fato.

Por fim, o controle é entregue à rotina `fsm_principal`, passando o ponteiro de GPIO em `x0`, e é essa chamada que efetivamente inicia o funcionamento contínuo do sistema, já que `fsm_principal` nunca retorna. O rótulo `erro_geral`, alcançado apenas se o mapeamento do GPIO falhar, imprime uma mensagem de erro específica e encerra o programa através da chamada de sistema de saída com código um, sinalizando falha para quem estiver observando o valor de retorno do processo.

estabelece_conexao.s

```
.global estabelece_conexao
.extern gpio_map
.extern gpio_set_output
.extern gpio_set_input
.extern gpio_write
.extern gpio_read

.equ SYS_NANOSLEEP, 101

.equ SCLK_GPIO, 5
.equ CS_GPIO, 6
.equ MOSI_GPIO, 12
.equ MISO_GPIO, 13

.equ TAMANHO_FRAME_DESCARTE, 44

.section .data
.align 8
tempo_espera_descarte:
    .quad 0
    .quad 10000

.section .text

estabelece_conexao:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    bl gpio_map
    cmp x1, #0
    b.ne conexao_erro
```

```
mov x19, x0
```

```
mov x0, x19
```

```
mov w1, #SCLK_GPIO
```

```
bl gpio_set_output
```

```
mov x0, x19
```

```
mov w1, #SCLK_GPIO
```

```
mov w2, #0
```

```
bl gpio_write
```

```
mov x0, x19
```

```
mov w1, #CS_GPIO
```

```
bl gpio_set_output
```

```
mov x0, x19
```

```
mov w1, #CS_GPIO
```

```
mov w2, #1
```

```
bl gpio_write
```

```
mov x0, x19
```

```
mov w1, #MOSI_GPIO
```

```
bl gpio_set_output
```

```
mov x0, x19
```

```
mov w1, #MOSI_GPIO
```

```
mov w2, #0
```

```
bl gpio_write
```

```
mov x0, x19
```

```
mov w1, #MISO_GPIO
```

```
bl gpio_set_input
```

```
mov x0, x19
```

```
mov w1, #CS_GPIO
```

```
mov w2, #0
bl gpio_write

bl atraso_curto_descarte

mov x20, #0

loop_bytes_descarte:
    cmp x20, #TAMANHO_FRAME_DESCARTE
    b.ge fim_bytes_descarte

    mov x21, #0

loop_bits_descarte:
    cmp x21, #8
    b.ge fim_bits_byte_descarte

    mov x0, x19
    mov w1, #SCLK_GPIO
    mov w2, #1
    bl gpio_write

    bl atraso_curto_descarte

    mov x0, x19
    mov w1, #MISO_GPIO
    bl gpio_read

    mov x0, x19
    mov w1, #SCLK_GPIO
    mov w2, #0
    bl gpio_write
```

```

    bl atraso_curto_descarte

    add x21, x21, #1
    b loop_bits_descarte

fim_bits_byte_descarte:
    add x20, x20, #1
    b loop_bytes_descarte

fim_bytes_descarte:
    mov x0, x19
    mov w1, #CS_GPIO
    mov w2, #1
    bl gpio_write

    mov x0, x19
    mov x1, #0
    ldp x29, x30, [sp], 16
    ret

conexao_erro:
    mov x0, #0
    mov x1, #1
    ldp x29, x30, [sp], 16
    ret

atraso_curto_descarte:
    stp x29, x30, [sp, -16]!
    mov x29, sp
    ldr x0, =tempo_espera_descarte
    mov x1, #0
    mov x8, #SYS_NANOSLEEP
    svc #0

```

```
ldp x29, x30, [sp], 16
```

```
ret
```

Esta rotina é responsável por preparar os quatro pinos usados na comunicação SPI implementada inteiramente por software, já que o periférico SPI de hardware da placa não pode ser usado por causa do dano físico no pino de clock original. Logo no início, `gpio_map` é chamada para mapear a região de memória dos registradores de GPIO, e o resultado é verificado da mesma forma que em `main.s`, através do registrador `x1`, desviando para o rótulo `conexao_erro` em caso de falha. Em caso de sucesso, o ponteiro retornado é guardado em `x19` para ser reutilizado ao longo de toda a rotina.

Em seguida, cada um dos quatro pinos é configurado individualmente. O pino de clock, GPIO cinco, e o pino de saída de dados, GPIO doze, são configurados como saída através de `gpio_set_output` e colocados imediatamente em nível baixo através de `gpio_write`, já que o protocolo SPI espera essas linhas paradas em repouso fora de uma transação. O pino de seleção de chip, GPIO seis, também é configurado como saída, mas é colocado inicialmente em nível alto, já que a convenção do protocolo é que a seleção de chip fica em nível alto quando nenhuma transação está em andamento. O pino de entrada de dados, GPIO treze, é configurado como entrada através de `gpio_set_input`, já que é por ele que a FPGA envia informação de volta para o Raspberry Pi.

Depois de configurar os quatro pinos, a rotina abaixa a seleção de chip e realiza uma leitura de descarte de quarenta e quatro bytes, controlada pela constante `TAMANHO_FRAME_DESCARTE`, gerando pulsos de clock sem se importar com o conteúdo lido através dos dois laços aninhados `loop_bytes_descarte` e `loop_bits_descarte`. Essa leitura de descarte serve para sincronizar o estado interno da máquina de estados da FPGA com o do Raspberry Pi logo depois do reset de ambos os lados, evitando que a primeira leitura de dados de verdade comece em um ponto intermediário e desalinhado de um quadro. Ao final dessa leitura de descarte, a seleção de chip é levantada novamente, encerrando essa transação de sincronização, e a rotina retorna com `x1` igual a zero, sinalizando sucesso.

A rotina auxiliar `atraso_curto_descarte` apenas aguarda um pequeno intervalo de tempo, definido em `tempo_espera_descarte`, entre cada mudança de nível dos sinais de clock, através da chamada de sistema de `nanosleep`, garantindo que a FPGA tenha tempo suficiente para amostrar cada pulso corretamente antes que o próximo comece.

gpio.s

```
.global gpio_map
.global gpio_unmap
.global gpio_set_output
.global gpio_set_input
.global gpio_write
.global gpio_read

.text

.equ SYS_openat, 56
.equ SYS_close, 57
.equ SYS_mmap, 222
.equ SYS_munmap, 215

.equ AT_FDCWD, -100
.equ O_RDWR, 2
.equ PROT_READ, 1
.equ PROT_WRITE, 2
.equ MAP_SHARED, 1

.equ GPIO_MAP_LEN, 4096

.equ GPFSEL0, 0x00
.equ GPSET0, 0x1C
.equ GPCLR0, 0x28
.equ GPLEV0, 0x34

.data
gpiomem_path:
    .asciz "/dev/gpiomem"

last_fd:
```

.quad -1

.text

gpio_map:

stp x29, x30, [sp, -16]!

mov x29, sp

mov x0, #AT_FDCWD

ldr x1, =gpiomem_path

mov x2, #O_RDWR

mov x3, #0

mov x8, #SYS_openat

svc #0

cmp x0, #0

b.lt map_fail

ldr x9, =last_fd

str x0, [x9]

mov x9, x0

mov x0, #0

mov x1, #GPIO_MAP_LEN

mov x2, #(PROT_READ | PROT_WRITE)

mov x3, #MAP_SHARED

mov x4, x9

mov x5, #0

mov x8, #SYS_mmap

svc #0

cmp x0, #0

b.lt mmap_fail

```
mov x1, #0
ldp x29, x30, [sp], 16
ret
```

mmap_fail:

```
ldr x9, =last_fd
ldr x0, [x9]
cmp x0, #0
b.lt map_fail
mov x8, #SYS_close
svc #0
```

map_fail:

```
mov x0, #0
mov x1, #1
ldp x29, x30, [sp], 16
ret
```

gpio_unmap:

```
stp x29, x30, [sp, -16]!
mov x29, sp

mov x19, x0

mov x0, x19
mov x1, #GPIO_MAP_LEN
mov x8, #SYS_munmap
svc #0

ldr x9, =last_fd
ldr x0, [x9]
cmp x0, #0
```

b.lt unmap_done

mov x8, #SYS_close

svc #0

unmap_done:

ldp x29, x30, [sp], 16

ret

gpio_set_output:

stp x29, x30, [sp, -16]!

mov x29, sp

mov w2, #10

udiv w3, w1, w2

mul w4, w3, w2

sub w5, w1, w4

mov w6, #3

mul w6, w5, w6

lsl w7, w3, #2

add x7, x0, x7

ldr w8, [x7]

mov w9, #7

lsl w9, w9, w6

bic w8, w8, w9

mov w10, #1

lsl w10, w10, w6

orr w8, w8, w10

```
str w8, [x7]
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
gpio_set_input:
```

```
stp x29, x30, [sp, -16]!
```

```
mov x29, sp
```

```
mov w2, #10
```

```
udiv w3, w1, w2
```

```
mul w4, w3, w2
```

```
sub w5, w1, w4
```

```
mov w6, #3
```

```
mul w6, w5, w6
```

```
lsl w7, w3, #2
```

```
add x7, x0, x7
```

```
ldr w8, [x7]
```

```
mov w9, #7
```

```
lsl w9, w9, w6
```

```
bic w8, w8, w9
```

```
str w8, [x7]
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
gpio_write:
```

```
stp x29, x30, [sp, -16]!
```

```
mov x29, sp
```

```
mov w3, #1
lsl w3, w3, w1
```

```
cmp w2, #0
b.eq write_low
```

```
add x4, x0, #GPSET0
str w3, [x4]
b write_done
```

```
write_low:
    add x4, x0, #GPCLR0
    str w3, [x4]
```

```
write_done:
    ldp x29, x30, [sp], 16
    ret
```

```
gpio_read:
    stp x29, x30, [sp, -16]!
    mov x29, sp
```

```
add x2, x0, #GPLEV0
ldr w3, [x2]
```

```
mov w4, #1
lsl w4, w4, w1
```

```
and w5, w3, w4
cmp w5, #0
cset w0, ne
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

Este arquivo concentra todo o acesso de baixo nível aos registradores de GPIO do Raspberry Pi, e é reutilizado por praticamente todas as demais rotinas do projeto que precisam ligar, desligar ou ler um pino físico. A rotina `gpio_map` abre o arquivo especial de dispositivo `dev gpiomem` através da chamada de sistema `openat`, usando as flags `AT_FDCWD` e `O_RDWR`, e guarda o descritor de arquivo retornado na variável `last_fd`, já que ele será necessário mais tarde para fechar o mapeamento corretamente. Em seguida, esse descritor é usado na chamada de sistema `mmap`, mapeando uma região de quatro mil e noventa e seis bytes, o tamanho de uma página de memória, com permissão de leitura e escrita e no modo compartilhado, o que dá ao programa acesso direto aos registradores físicos de controle dos pinos sem exigir privilégios de administrador completos. Caso qualquer uma das duas chamadas falhe, a rotina desvia para os rótulos `map_fail` ou `mmap_fail`, fechando o descritor de arquivo já aberto quando necessário e retornando um código de erro em `x1`.

A rotina `gpio_unmap` desfaz esse mapeamento, chamando a chamada de sistema `munmap` sobre o mesmo ponteiro e tamanho usados na criação, e depois fecha o descritor de arquivo guardado em `last_fd`, encerrando de forma limpa o acesso ao hardware quando o programa termina.

As rotinas `gpio_set_output` e `gpio_set_input` configuram um pino específico como saída ou como entrada, e a matemática no início das duas é idêntica, calculando primeiro em qual dos seis registradores `GPFSEL` o pino em questão se encontra, já que cada registrador desses controla dez pinos, e depois calculando o deslocamento de três bits dentro desse registrador que corresponde exatamente àquele pino, já que cada pino usa três bits para definir sua função. A diferença entre as duas rotinas está apenas no valor final escrito nesses três bits, sendo zero para entrada e um para saída, e ambas usam uma máscara de sete deslocada para a posição correta, através da instrução `bic`, para zerar esses três bits antes de escrever o novo valor por cima com `orr`, preservando os demais pinos controlados pelo mesmo registrador de trinta e dois bits.

A rotina `gpio_write` recebe o número do pino, o valor desejado e escreve um bit deslocado para a posição correspondente àquele pino, usando o registrador `GPSET0` quando o valor é diferente de zero e o registrador `GPCLR0` quando o valor é zero, já que esses são registradores de escrita apenas, nos quais escrever um bit em uma posição ativa

ou desativa exclusivamente aquele pino, sem afetar os demais. Por fim, a rotina `gpio_read` lê o registrador `GPLEV0`, que reflete o nível lógico atual de todos os pinos de entrada, isola o bit correspondente ao pino solicitado através de uma máscara e de uma operação `and`, e usa a instrução `cset` para transformar o resultado dessa comparação diretamente em um valor de zero ou um em `w0`, sem precisar de nenhum desvio condicional explícito.

atuadores.s

```
.global aciona_bomba
.global aciona_luz
.global inicializa_atuadores
.extern gpio_write
.extern gpio_set_output

.equ GPIO_BOMBA, 17
.equ GPIO_LUZ, 27

inicializa_atuadores:
    stp x29, x30, [sp, -32]!
    mov x29, sp
    str x19, [sp, 16]
    mov x19, x0

    mov w1, #GPIO_BOMBA
    bl gpio_set_output

    mov x0, x19
    mov w1, #GPIO_LUZ
    bl gpio_set_output

    ldr x19, [sp, 16]
    ldp x29, x30, [sp], 32
    ret

aciona_bomba:
    stp x29, x30, [sp, -16]!
    mov x29, sp
    mov w2, w1
    mov w1, #GPIO_BOMBA
    bl gpio_write
```

```
ldp x29, x30, [sp], 16
ret

aciona_luz:
stp x29, x30, [sp, -16]!
mov x29, sp
mov w2, w1
mov w1, #GPIO_LUZ
bl gpio_write
ldp x29, x30, [sp], 16
ret
```

Este arquivo é responsável por controlar os dois atuadores físicos do sistema, a bomba peristáltica e a luz de crescimento, servindo como uma camada intermediária entre a lógica de decisão de `fsm_principal.s` e as rotinas genéricas de GPIO já descritas em `gpio.s`. A rotina `inicializa_atuadores` recebe o ponteiro de GPIO e chama `gpio_set_output` duas vezes, uma para o pino dezessete, identificado pela constante `GPIO_BOMBA`, e outra para o pino vinte e sete, identificado pela constante `GPIO_LUZ`, configurando os dois como saída antes de qualquer acionamento ser tentado. O registrador `x19`, salvo na pilha logo no início da rotina, guarda o ponteiro de GPIO entre as duas chamadas, já que ele precisa ser passado novamente em `x0` na segunda chamada depois de ter sido usado na primeira.

As rotinas `aciona_bomba` e `aciona_luz` são praticamente idênticas entre si, diferindo apenas na constante de pino usada. Cada uma recebe o ponteiro de GPIO em `x0` e o valor desejado, ligado ou desligado, em `x1`, mas antes de chamar `gpio_write` esse valor precisa ser movido para `w2`, já que `gpio_write` espera o número do pino em `w1` e o valor em `w2`, o que exige reorganizar os parâmetros antes da chamada, movendo primeiro o valor recebido de `w1` para `w2` e só depois carregando a constante de pino em `w1`. Essa pequena reorganização de registradores é necessária justamente porque a assinatura de `aciona_bomba` e `aciona_luz`, pensada para ser chamada de forma simples a partir de `fsm_principal.s` com apenas ponteiro e valor, não corresponde diretamente à assinatura de `gpio_write`, que também precisa saber qual pino está sendo escrito.

usa_lcd.s

```
.global lcd_inicializa
.global lcd_posiciona_cursor
.global lcd_escreve_string
.global lcd_buffer_escreve
.global lcd_buffer_completa_espacos
.global formata_valor_fixo_buffer
.extern converte_num_para_string

.equ AT_FDCWD, -100
.equ O_RDWR, 2
.equ I2C_SLAVE, 0x0703
.equ ENDERECO_LCD, 0x27
.equ SYS_OPENAT, 56
.equ SYS_IOCTL, 29
.equ SYS_WRITE, 64
.equ SYS_NANOSLEEP, 101
.equ RS_COMANDO, 0
.equ RS_DADO, 1

.section .data
i2c_dev_path: .asciz "/dev/i2c-1"
.align 8
fd_lcd: .quad 0
buffer_envio_lcd: .skip 4

.section .text

lcd_abre_dispositivo:
    stp x29, x30, [sp, -16]!
    mov x29, sp
```

```
mov x0, #AT_FDCWD
ldr x1, =i2c_dev_path
mov x2, #O_RDWR
mov x3, #0
mov x8, #SYS_OPENAT
svc #0
```

```
ldr x1, =fd_lcd
str x0, [x1]
```

```
mov x1, #I2C_SLAVE
mov x2, #ENDERECO_LCD
mov x8, #SYS_IOCTL
svc #0
```

```
ldp x29, x30, [sp], 16
ret
```

i2c_escreve_retry:

```
stp x29, x30, [sp, -32]!
mov x29, sp
stp x19, x20, [sp, 16]
```

```
mov x19, x0
mov x20, x1
```

loop_retry_i2c:

```
ldr x1, =fd_lcd
ldr x9, [x1]
mov x0, x9
mov x1, x19
mov x2, x20
mov x8, #SYS_WRITE
```

```
svc #0
cmp x0, #0
blt loop_retry_i2c
```

```
ldp x19, x20, [sp, 16]
ldp x29, x30, [sp], 32
ret
```

lcd_espera_us:

```
stp x29, x30, [sp, -32]!
mov x29, sp
```

```
mov x1, #1000
mul x1, x0, x1
stp xzr, x1, [sp, 16]
add x0, sp, #16
mov x1, #0
mov x8, #SYS_NANOSLEEP
svc #0
```

```
ldp x29, x30, [sp], 32
ret
```

lcd_envia_nibble_apenas:

```
stp x29, x30, [sp, -16]!
mov x29, sp
```

```
and w1, w0, #0xF
lsl w1, w1, #4
orr w1, w1, #0x8
```

```
orr w2, w1, #0x4
ldr x3, =buffer_envio_lcd
```

```
strb w2, [x3]
strb w1, [x3, #1]

mov x0, x3
mov x1, #1
bl i2c_escreve_retry

mov x0, #60
bl lcd_espera_us

add x0, x3, #1
mov x1, #1
bl i2c_escreve_retry

mov x0, #60
bl lcd_espera_us

ldp x29, x30, [sp], 16
ret
```

```
lcd_envia_byte:
    stp x29, x30, [sp, -16]!
    mov x29, sp
```

```
lsr w2, w0, #4
and w2, w2, #0xF
lsl w2, w2, #4
orr w2, w2, #0x8
orr w2, w2, w1
```

```
and w3, w0, #0xF
lsl w3, w3, #4
orr w3, w3, #0x8
```

```
orr w3, w3, w1

ldr x4, =buffer_envio_lcd
orr w5, w2, #0x4
strb w5, [x4]
strb w2, [x4, #1]
orr w5, w3, #0x4
strb w5, [x4, #2]
strb w3, [x4, #3]

mov x0, x4
mov x1, #1
bl i2c_escreve_retry

mov x0, #60
bl lcd_espera_us

add x0, x4, #1
mov x1, #1
bl i2c_escreve_retry

mov x0, #60
bl lcd_espera_us

add x0, x4, #2
mov x1, #1
bl i2c_escreve_retry

mov x0, #60
bl lcd_espera_us

add x0, x4, #3
mov x1, #1
```

```
bl i2c_escreve_retry
```

```
mov x0, #60
```

```
bl lcd_espera_us
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
lcd_inicializa:
```

```
stp x29, x30, [sp, -16]!
```

```
mov x29, sp
```

```
bl lcd_abre_dispositivo
```

```
mov x0, #40000
```

```
bl lcd_espera_us
```

```
mov x0, #0x3
```

```
bl lcd_envia_nibble_apenas
```

```
mov x0, #5000
```

```
bl lcd_espera_us
```

```
mov x0, #0x3
```

```
bl lcd_envia_nibble_apenas
```

```
mov x0, #200
```

```
bl lcd_espera_us
```

```
mov x0, #0x3
```

```
bl lcd_envia_nibble_apenas
```

```
mov x0, #200
```

```
bl lcd_espera_us
```

```
mov x0, #0x2
```

```
bl lcd_envia_nibble_apenas
```

```
mov x0, #200
```

```
bl lcd_espera_us
```

```
mov x0, #0x28
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #100
```

```
bl lcd_espera_us
```

```
mov x0, #0x08
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #100
```

```
bl lcd_espera_us
```

```
mov x0, #0x01
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #2000
```

```
bl lcd_espera_us
```

```
mov x0, #0x06
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #100
```

```
bl lcd_espera_us
```

```
mov x0, #0x0C
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #100
```

```
bl lcd_espera_us
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
lcd_posiciona_cursor:
```

```
stp x29, x30, [sp, -16]!
```

```
mov x29, sp
```

```
cmp x0, #0
```

```
beq linha0_cursor
```

```
mov w2, #0x40
```

```
b base_linha_pronta
```

```
linha0_cursor:
```

```
mov w2, #0x00
```

```
base_linha_pronta:
```

```
add w2, w2, w1
```

```
orr w0, w2, #0x80
```

```
mov x1, #RS_COMANDO
```

```
bl lcd_envia_byte
```

```
mov x0, #100
```

```
bl lcd_espera_us
```

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
lcd_escreve_string:
```

```
stp x29, x30, [sp, -48]!
```

```
mov x29, sp
```

```
stp x19, x20, [sp, 16]
```

```
str x21, [sp, 32]
```

```
mov x19, x0
```

```
mov w20, w1
```

```

mov w21, #0

loop_escreve_str:
    cmp w21, w20
    bge fim_escreve_str

    ldrb w0, [x19, x21]
    mov x1, #RS_DADO
    bl lcd_envia_byte
    mov x0, #100
    bl lcd_espera_us

    add w21, w21, #1
    b loop_escreve_str

fim_escreve_str:
    ldr x21, [sp, 32]
    ldp x19, x20, [sp, 16]
    ldp x29, x30, [sp], 48
    ret

lcd_buffer_escreve:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    ldr w4, [x1]
    mov w5, #16
    subs w5, w5, w4
    ble fim_buffer_escreve

    cmp w3, w5
    ble copia_tudo_buffer
    mov w3, w5

```

copia_tudo_buffer:

mov w6, #0

loop_copia_buffer:

cmp w6, w3

bge fim_copia_buffer

ldrb w7, [x2, x6]

add x8, x0, x4

strb w7, [x8]

add w4, w4, #1

add w6, w6, #1

b loop_copia_buffer

fim_copia_buffer:

str w4, [x1]

fim_buffer_escreve:

ldp x29, x30, [sp], 16

ret

lcd_buffer_completa_espacos:

stp x29, x30, [sp, -16]!

mov x29, sp

ldr w2, [x1]

loop_completa_espacos:

cmp w2, #16

bge fim_completa_espacos

add x3, x0, x2

mov w4, #0x20

strb w4, [x3]

add w2, w2, #1

b loop_completa_espacos

fim_completa_espacos:

str w2, [x1]

```
ldp x29, x30, [sp], 16
```

```
ret
```

```
formata_valor_fixo_buffer:
```

```
stp x29, x30, [sp, -64]!
```

```
mov x29, sp
```

```
stp x19, x20, [sp, 16]
```

```
stp x21, x22, [sp, 32]
```

```
stp x23, x24, [sp, 48]
```

```
mov x19, x1
```

```
mov x20, x2
```

```
mov x23, x3
```

```
mov x24, x4
```

```
udiv x9, x0, x19
```

```
msub x21, x9, x19, x0
```

```
mov x0, x9
```

```
bl converte_num_para_string
```

```
mov x2, x0
```

```
mov x3, x1
```

```
mov x0, x23
```

```
mov x1, x24
```

```
bl lcd_buffer_escreve
```

```
cmp x20, #0
```

```
beq fim_formata_valor_buffer
```

```
mov x0, x23
```

```
mov x1, x24
```

```
ldr x2, =string_ponto_lcd
```

```

mov x3, #1
bl lcd_buffer_escreve

mov x0, x21
bl converte_num_para_string
mov x21, x0
mov x22, x1

subs x9, x20, x22
ble escreve_fracao_buffer

loop_zero_pad_buffer:
    mov x0, x23
    mov x1, x24
    ldr x2, =string_zero_lcd
    mov x3, #1
    bl lcd_buffer_escreve
    subs x9, x9, #1
    bgt loop_zero_pad_buffer

escreve_fracao_buffer:
    mov x0, x23
    mov x1, x24
    mov x2, x21
    mov x3, x22
    bl lcd_buffer_escreve

fim_formata_valor_buffer:
    ldp x23, x24, [sp, 48]
    ldp x21, x22, [sp, 32]
    ldp x19, x20, [sp, 16]
    ldp x29, x30, [sp], 64
    ret

```

```
.section .data
string_ponto_lcd: .ascii "."
string_zero_lcd: .ascii "0"
```

Este arquivo concentra toda a lógica de comunicação com o display de cristal líquido, que é acessado através de um módulo expensor PCF8574T ligado ao barramento I2C do Raspberry Pi, e por isso boa parte das rotinas aqui lida com detalhes bem específicos tanto do protocolo I2C do Linux quanto do protocolo HD44780 usado internamente pelo display. A rotina `lcd_abre_dispositivo` abre o arquivo `dev i2c-1` através de `openat` e em seguida usa a chamada de sistema `ioctl` com a constante `I2C_SLAVE` para informar ao driver do kernel qual endereço de barramento, `0x27`, deve ser usado nas escritas seguintes, guardando o descritor de arquivo resultante em `fd_lcd` para ser reaproveitado por todas as demais rotinas deste arquivo. A rotina `i2c_escreve_retry` encapsula a chamada de sistema de escrita em um laço que insiste enquanto o valor de retorno for negativo, já que esse display específico apresenta falhas intermitentes de confirmação na escrita I2C, e sem essa insistência a primeira tentativa de escrita poderia simplesmente falhar de forma silenciosa.

O expensor PCF8574T controla o display através de apenas oito linhas digitais, o que obriga o driver a operar o HD44780 no modo de quatro bits, enviando cada informação de quatro em quatro bits, e é justamente essa divisão que as rotinas `lcd_envia_nibble_apenas` e `lcd_envia_byte` implementam. A primeira envia isoladamente os quatro bits mais altos de um valor, usada apenas durante a sequência especial de inicialização do display, enquanto a segunda monta e envia um byte completo, primeiro os quatro bits altos e depois os quatro bits baixos, cada metade acompanhada de um pulso no bit de habilitação, que é o bit `0x04` somado momentaneamente ao valor antes de ser retirado na escrita seguinte, e também do bit de seleção de registrador, passado no parâmetro `RS_COMANDO` ou `RS_DADO`, que informa ao display se aquele byte deve ser interpretado como uma instrução de controle ou como um caractere a ser exibido na tela. Entre cada uma dessas escritas, a rotina `lcd_espera_us` converte um valor em microssegundos para a estrutura de tempo exigida pela chamada de sistema de `nanosleep`, respeitando os tempos mínimos de processamento interno do HD44780, que variam de sessenta microssegundos entre bytes até valores bem maiores durante a sequência de inicialização.

A rotina `lcd_inicializa` segue exatamente a sequência de inicialização recomendada pelo fabricante do HD44780 para o modo de quatro bits, enviando três vezes seguidas o nibble 0x3, cada vez com um tempo de espera diferente e decrescente, seguido do nibble 0x2, que finalmente comuta o display para o modo de quatro bits, e só depois disso os comandos de configuração completos passam a ser enviados como bytes inteiros através de `lcd_envia_byte`, sendo eles a definição de duas linhas e fonte pequena, o desligamento inicial do display, a limpeza completa da tela, o modo de incremento automático do cursor e por fim o religamento do display com o cursor invisível. A rotina `lcd_posiciona_cursor` calcula o endereço interno da memória de exibição do HD44780 a partir de um número de linha e de coluna, somando 0x40 quando a linha solicitada é a segunda, já que é assim que a memória do HD44780 está organizada internamente, e envia esse endereço com o bit mais significativo ligado, conforme exige o comando de posicionamento de cursor do próprio display.

As rotinas `lcd_buffer_escreve`, `lcd_buffer_completa_espacos` e `formata_valor_fixo_buffer` trabalham em conjunto para montar uma linha inteira de dezesseis caracteres na memória antes de qualquer escrita física acontecer, o que evita que o display pisque ou mostre estados intermediários enquanto uma linha está sendo composta. A primeira copia uma sequência de caracteres para dentro do buffer, respeitando o limite de dezesseis posições e atualizando a posição de escrita corrente guardada em um segundo parâmetro por referência. A segunda completa com espaços em branco qualquer posição do buffer que ainda não tenha sido preenchida até completar as dezesseis posições, garantindo que qualquer caractere residual de uma escrita anterior seja apagado. A terceira monta a representação textual de um valor em ponto fixo diretamente dentro do buffer, separando a parte inteira da parte fracionária através de uma divisão inteira, escrevendo um ponto entre as duas partes quando o número de casas decimais solicitado for maior que zero, e preenchendo com zeros à esquerda a parte fracionária sempre que a conversão de `converte_num_para_string` devolver menos dígitos do que o número de casas decimais esperado, o que evita, por exemplo, que um valor de três centésimos apareça como três em vez de zero três.

fsm_principal.s

```
.global fsm_principal
.extern recebe_info
.extern calcula_media
.extern eh_dia
.extern aciona_bomba
.extern aciona_luz
.extern imprime_valor_fixo
.extern converte_num_para_string
.extern tabela_temp
.extern indice_temp
.extern tabela_umidade_ar
.extern indice_umidade_ar
.extern tabela_luz
.extern indice_luz
.extern tabela_solo
.extern indice_solo
.extern lcd_inicializa

.equ SYS_WRITE, 64
.equ SYS_NANOSLEEP, 101
.extern apresenta_info_em_tela

.equ AMOSTRAS_POR_JANELA, 150
.equ LIMIAR_SOLO_SECO_CENTIPERCENT, 7000

.section .data

luz_ligada: .word 0

.align 8
numero_iteracao_global: .word 1
```

.align 8

tempo_entre_leituras_fsm:

.quad 2

.quad 0

.align 8

tempo_pulso_rega:

.quad 15

.quad 0

msg_media_temp: .asciz "\n[DIGERIR_DADOS] Media 5min - Temperatura: "

msg_media_temp_fim = . - msg_media_temp - 1

msg_media_umidade_ar: .asciz "\n[DIGERIR_DADOS] Media 5min - Umidade do ar: "
"

msg_media_umidade_ar_fim = . - msg_media_umidade_ar - 1

msg_media_luz: .asciz "\n[DIGERIR_DADOS] Media 5min - Luminosidade: "

msg_media_luz_fim = . - msg_media_luz - 1

msg_media_solo: .asciz "\n[DIGERIR_DADOS] Media 5min - Umidade do solo: "

msg_media_solo_fim = . - msg_media_solo - 1

msg_graus_c_fsm: .asciz " °C"

msg_graus_c_fsm_fim = . - msg_graus_c_fsm - 1

msg_percent_fsm: .asciz " %"

msg_percent_fsm_fim = . - msg_percent_fsm - 1

msg_lux_fsm: .asciz " lux\n"

msg_lux_fsm_fim = . - msg_lux_fsm - 1

msg_molhar: .asciz "[DIGERIR_DADOS] Solo seco -> MOLHAR_PLANTA\n"

```
msg_molhar_fim = . - msg_molhar - 1
```

```
msg_iluminar: .asciz "[DIGERIR_DADOS] Escuro de dia ->  
ILUMINAR_PLANTA\n"
```

```
msg_iluminar_fim = . - msg_iluminar - 1
```

```
msg_parar_luz: .asciz "[DIGERIR_DADOS] Luz suficiente ou noite ->  
PARA_DE_ILUMINAR_PLANTA\n"
```

```
msg_parar_luz_fim = . - msg_parar_luz - 1
```

```
msg_nada_a_fazer: .asciz "[DIGERIR_DADOS] Nenhuma transicao necessaria ->  
RECEBER_DADOS\n"
```

```
msg_nada_a_fazer_fim = . - msg_nada_a_fazer - 1
```

```
.section .text
```

```
fsm_principal:
```

```
    stp x29, x30, [sp, -48]!
```

```
    mov x29, sp
```

```
    stp x19, x20, [sp, 16]
```

```
    stp x21, x22, [sp, 32]
```

```
    mov x19, x0
```

```
estado_receber_dados:
```

```
    mov w20, #0
```

```
loop_receber_dados:
```

```
    mov x0, x19
```

```
    ldr x1, =numero_iteracao_global
```

```
    ldr w1, [x1]
```

```
    bl recebe_info
```

```
ldr x1, =numero_iteracao_global
```

```
ldr w2, [x1]
```

```
add w2, w2, #1
```

```
str w2, [x1]
```

```
ldr x0, =tempo_entre_leituras_fsm
```

```
mov x1, #0
```

```
mov x8, #SYS_NANOSLEEP
```

```
svc #0
```

```
add w20, w20, #1
```

```
cmp w20, #AMOSTRAS_POR_JANELA
```

```
blt loop_receber_dados
```

```
estado_digerir_dados:
```

```
ldr x0, =tabela_temp
```

```
ldr x1, =indice_temp
```

```
bl calcula_media
```

```
mov w21, w0
```

```
mov x0, #1
```

```
ldr x1, =msg_media_temp
```

```
mov x2, #msg_media_temp_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, x21
```

```
mov x1, #100
```

```
mov x2, #2
```

```
bl imprime_valor_fixo
```

```
mov x0, #1
```

```
ldr x1, =msg_graus_c_fsm
```

```
mov x2, #msg_graus_c_fsm_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
ldr x0, =tabela_umidade_ar
```

```
ldr x1, =indice_umidade_ar
```

```
bl calcula_media
```

```
mov w22, w0
```

```
mov x0, #1
```

```
ldr x1, =msg_media_umidade_ar
```

```
mov x2, #msg_media_umidade_ar_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, x22
```

```
mov x1, #100
```

```
mov x2, #2
```

```
bl imprime_valor_fixo
```

```
mov x0, #1
```

```
ldr x1, =msg_percent_fsm
```

```
mov x2, #msg_percent_fsm_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
ldr x0, =tabela_luz
```

```
ldr x1, =indice_luz
```

```
bl calcula_media
```

```
mov w23, w0
```

```
mov x0, #1
```

```
ldr x1, =msg_media_luz
```

```
mov x2, #msg_media_luz_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, x23
```

```
mov x1, #10
mov x2, #1
bl imprime_valor_fixo
mov x0, #1
ldr x1, =msg_lux_fsm
mov x2, #msg_lux_fsm_fim
mov x8, #SYS_WRITE
svc #0

ldr x0, =tabela_solo
ldr x1, =indice_solo
bl calcula_media
mov w24, w0

mov x0, #1
ldr x1, =msg_media_solo
mov x2, #msg_media_solo_fim
mov x8, #SYS_WRITE
svc #0
mov x0, x24
mov x1, #100
mov x2, #2
bl imprime_valor_fixo
mov x0, #1
ldr x1, =msg_percent_fsm
mov x2, #msg_percent_fsm_fim
mov x8, #SYS_WRITE
svc #0

bl lcd_inicializa
mov x0, x21
mov x1, x22
mov x2, x23
```

```
mov x3, x24
bl apresenta_info_em_tela

bl eh_dia
cmp x0, #0
beq decide_noite

ldr w9, =5000
cmp w23, w9
blt decide_liga_luz
b decide_desliga_luz_se_ligada
```

```
decide_noite:
    b decide_desliga_luz_se_ligada
```

```
decide_liga_luz:
    ldr x1, =luz_ligada
    ldr w2, [x1]
    cmp w2, #0
    bne checa_solo
    b estado_iluminar_planta
```

```
decide_desliga_luz_se_ligada:
    ldr x1, =luz_ligada
    ldr w2, [x1]
    cmp w2, #0
    beq checa_solo
    b estado_para_de_iluminar_planta
```

```
checa_solo:
    ldr w9, =LIMIAR_SOLO_SECO_CENTIPERCENT
    cmp w24, w9
    blt estado_molhar_planta
```

b estado_receber_dados_nada

estado_receber_dados_nada:

mov x0, #1

ldr x1, =msg_nada_a_fazer

mov x2, #msg_nada_a_fazer_fim

mov x8, #SYS_WRITE

svc #0

b estado_receber_dados

estado_molhar_planta:

mov x0, #1

ldr x1, =msg_molhar

mov x2, #msg_molhar_fim

mov x8, #SYS_WRITE

svc #0

mov x0, x19

mov x1, #1

bl aciona_bomba

ldr x0, =tempo_pulso_rega

mov x1, #0

mov x8, #SYS_NANOSLEEP

svc #0

mov x0, x19

mov x1, #0

bl aciona_bomba

bl lcd_inicializa

mov x0, x21

mov x1, x22

```
mov x2, x23
mov x3, x24
bl apresenta_info_em_tela

b estado_receber_dados

estado_iluminar_planta:
    mov x0, #1
    ldr x1, =msg_iluminar
    mov x2, #msg_iluminar_fim
    mov x8, #SYS_WRITE
    svc #0

    mov x0, x19
    mov x1, #1
    bl aciona_luz

    ldr x1, =luz_ligada
    mov w2, #1
    str w2, [x1]

    bl lcd_inicializa
    mov x0, x21
    mov x1, x22
    mov x2, x23
    mov x3, x24
    bl apresenta_info_em_tela

    b estado_receber_dados

estado_para_de_iluminar_planta:
    mov x0, #1
    ldr x1, =msg_parar_luz
```

```
mov x2, #msg_parar_luz_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x19
mov x1, #0
bl aciona_luz
```

```
ldr x1, =luz_ligada
mov w2, #0
str w2, [x1]
```

```
bl lcd_inicializa
mov x0, x21
mov x1, x22
mov x2, x23
mov x3, x24
bl apresenta_info_em_tela

b estado_receber_dados
```

Este arquivo reúne a máquina de estados que comanda todo o comportamento do sistema depois que o ambiente já foi configurado, sendo chamada uma única vez a partir de main.s e nunca mais devolvendo o controle para quem a chamou, já que o próprio funcionamento do sistema é pensado como um ciclo infinito de leitura e decisão. Logo no início da rotina, os registradores x19 a x22 são preservados na pilha junto com o par x29 e x30, seguindo a convenção de chamada do ARM64 em que esses registradores precisam ser devolvidos intactos ao final, uma vez que serão usados ao longo de toda a função para guardar valores que sobrevivem a diversas chamadas de sub-rotina. O ponteiro recebido em x0, que aponta para a área de memória mapeada do GPIO, é então copiado para x19, que a partir desse ponto passa a representar esse mesmo endereço em todas as chamadas seguintes a recebe_info, aciona_bomba e aciona_luz.

O primeiro estado, chamado estado_receber_dados, zera o contador w20 e entra em um laço, o loop_receber_dados, que chama recebe_info passando o ponteiro de GPIO

e o número da iteração atual, lido a partir da variável `numero_iteracao_global`. Depois de cada chamada, essa variável é incrementada e uma pausa de dois segundos é aplicada através da chamada de sistema de `nanosleep`, usando a estrutura `tempo_entre_leituras_fsm`, que guarda dois campos de sessenta e quatro bits representando segundos e nanossegundos, de acordo com o formato exigido por essa chamada de sistema no Linux. Esse laço se repete até que o contador `w20` alcance o valor de `AMOSTRAS_POR_JANELA`, fixado em cento e cinquenta, o que significa que, a cada dois segundos por leitura, o sistema acumula amostras durante cinco minutos inteiros antes de tomar qualquer decisão, formando assim a janela de médias descrita ao longo de todo o projeto.

Encerrada essa janela, o fluxo entra no estado `_digerir_dados`, no qual a rotina `calcula_media` é chamada quatro vezes seguidas, uma para cada parâmetro ambiental, sempre recebendo o endereço da tabela de amostras correspondente e o endereço do índice associado a ela. O resultado de cada chamada é guardado respectivamente em `w21` para a temperatura, `w22` para a umidade do ar, `w23` para a luminosidade e `w24` para a umidade do solo, e esses quatro registradores permanecem com esses mesmos valores até o final da função, já que nenhuma outra rotina chamada depois os sobrescreve. Entre uma chamada e outra, o valor calculado também é impresso no terminal através de `imprime_valor_fixo`, que recebe o valor bruto e a quantidade de casas decimais a considerar, sendo cem para temperatura, umidade do ar e umidade do solo, e dez para a luminosidade, já que essa última é tratada com apenas uma casa decimal de precisão.

Assim que as quatro médias estão calculadas, a rotina `lcd_inicializa` é chamada antes de os quatro valores serem carregados nos registradores `x0` a `x3` para a chamada de `apresenta_info_em_tela`, que é responsável por escrever esses valores no display de cristal líquido. Essa ordem importa bastante, pois `lcd_inicializa` usa internamente registradores que não são preservados entre chamadas, e se ela fosse chamada depois de `x0` a `x3` já estarem carregados com os parâmetros da tela, esses valores seriam destruídos antes mesmo de chegarem à função que realmente os utiliza, corrompendo a primeira atualização do display a cada ciclo.

Depois de atualizar o display, a rotina `eh_dia` é chamada para saber se o sistema está no período diurno, e, caso o retorno seja diferente de zero, o valor de luminosidade guardado em `w23` é comparado contra um limiar fixo de cinco mil, decidindo entre acender a luz, através do rótulo `decide_liga_luz`, ou verificar se ela já está acesa e precisa ser apagada, através de `decide_desliga_luz_se_ligada`. Os dois caminhos consultam a

variável `luz_ligada`, que guarda um simples zero ou um indicando o estado atual do atuador, evitando que a luz seja acionada ou desligada repetidamente de forma redundante, e ambos, depois de resolvida a decisão sobre a luz, desviam para um bloco chamado `checa_solo`, que só então compara a média de umidade do solo, guardada em `w24`, contra o limiar `LIMIAR_SOLO_SECO_CENTIPERCENT`, fixado em sete mil centésimos de por cento. Como esse valor ultrapassa o intervalo de doze bits aceito pela instrução de comparação em modo imediato do ARM64, ele precisa primeiro ser carregado em um registrador auxiliar, `w9`, antes de ser comparado com `w24`, o que explica a presença da instrução `ldr` logo antes do `cmp` nesse trecho.

Caso a comparação em `checa_solo` indique solo seco, o fluxo segue para `estado_molhar_planta`, que aciona a bomba peristáltica ligando o atuador, aguarda quinze segundos através da estrutura `tempo_pulso_rega` e depois desliga a bomba, aplicando assim um pulso único e de duração fixa a cada rega, sem nenhuma lógica de histerese entre dois limiares diferentes. Os dois estados de iluminação, `estado_iluminar_planta` e `estado_para_de_iluminar_planta`, seguem uma estrutura parecida, ligando ou desligando o atuador de luz e atualizando a variável `luz_ligada` de acordo, enquanto o estado `estado_receber_dados_nada` apenas informa que nenhuma ação foi necessária naquele ciclo. Todos esses quatro estados finais voltam ao início do laço de recebimento de dados assim que terminam, fechando o ciclo e reiniciando a contagem de cento e cinquenta amostras para a próxima janela de cinco minutos.

recebe_info.s

```
.global recebe_info
.global rx_frame
.global imprime_valor_fixo
.global obtem_timestamp
.global converte_num_para_string
.extern gpio_write
.extern gpio_read
.extern converte_temp
.extern converte_luz
.extern converte_umidade_ar
.extern converte_umidade_solo
.extern armazena_amostra
.extern tabela_temp
.extern indice_temp
.extern tabela_umidade_ar
.extern indice_umidade_ar
.extern tabela_luz
.extern indice_luz
.extern tabela_solo
.extern indice_solo
```

```

.equ SYS_WRITE, 64
.equ SYS_NANOSLEEP, 101

.equ SYS_CLOCK_GETTIME, 113
.equ CLOCK_REALTIME, 0

.equ SCLK_GPIO, 5
.equ CS_GPIO, 6
.equ MOSI_GPIO, 12

.equ TAMANHO_FRAME, 14
.equ STATUS_OK, 0x5A
.equ COMANDO_SOLICITA, 0xA5

.section .data

rx_frame: .byte 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

.align 8
buffer_timespec:
    .quad 0
    .quad 0

msg_cabecalho_prefixo: .asciz "\n--- Iteracao "
msg_cabecalho_prefixo_fim = . - msg_cabecalho_prefixo - 1

msg_timestamp_prefixo: .asciz " (t="
msg_timestamp_prefixo_fim = . - msg_timestamp_prefixo - 1

msg_timestamp_sufixo: .asciz ")"
msg_timestamp_sufixo_fim = . - msg_timestamp_sufixo - 1

msg_cabecalho_meio: .asciz " ---\n"

```

```
msg_cabecalho_meio_fim = . - msg_cabecalho_meio - 1
```

```
msg_erro_status: .asciz "ERRO: FPGA nao reconheceu o comando (status invalido) -  
> leitura descartada\n"
```

```
msg_erro_status_fim = . - msg_erro_status - 1
```

```
msg_erro_checksum: .asciz "ERRO: checksum nao confere -> leitura descartada\n"
```

```
msg_erro_checksum_fim = . - msg_erro_checksum - 1
```

```
msg_pressao: .asciz "Pressao bruta: 0x"
```

```
msg_pressao_fim = . - msg_pressao - 1
```

```
msg_temperatura: .asciz "Temperatura bruta: 0x"
```

```
msg_temperatura_fim = . - msg_temperatura - 1
```

```
msg_umidade: .asciz "Umidade bruta: 0x"
```

```
msg_umidade_fim = . - msg_umidade - 1
```

```
msg_luminosidade: .asciz "Luminosidade bruta: 0x"
```

```
msg_luminosidade_fim = . - msg_luminosidade - 1
```

```
msg_umidade_solo: .asciz "Umidade do solo bruta: 0x"
```

```
msg_umidade_solo_fim = . - msg_umidade_solo - 1
```

```
msg_temp_convertida: .asciz "Temperatura: "
```

```
msg_temp_convertida_fim = . - msg_temp_convertida - 1
```

```
msg_graus_c: .asciz " °C\n"
```

```
msg_graus_c_fim = . - msg_graus_c - 1
```

```
msg_umidade_ar_convertida: .asciz "Umidade do ar: "
```

```
msg_umidade_ar_convertida_fim = . - msg_umidade_ar_convertida - 1
```

```

msg_percent: .asciz "%\n"
msg_percent_fim = . - msg_percent - 1

msg_luz_convertida: .asciz "Luminosidade: "
msg_luz_convertida_fim = . - msg_luz_convertida - 1

msg_lux: .asciz " lux\n"
msg_lux_fim = . - msg_lux - 1

msg_umidade_solo_convertida: .asciz "Umidade do solo: "
msg_umidade_solo_convertida_fim = . - msg_umidade_solo_convertida - 1

msg_ponto: .asciz "."
msg_zero: .asciz "0"

.align 8
buffer_iteracao: .skip 20
buffer_hex:      .byte 0,0

.align 8
tempo_espera:
    .quad 0
    .quad 10000

.section .text

recebe_info:
    stp x29, x30, [sp, -80]!
    mov x29, sp
    stp x19, x20, [sp, 16]
    stp x21, x22, [sp, 32]
    stp x23, x24, [sp, 48]
    stp x25, x26, [sp, 64]

```

```
mov x19, x0
```

```
mov x20, x1
```

```
mov x0, #1
```

```
ldr x1, =msg_cabecalho_prefixo
```

```
mov x2, #msg_cabecalho_prefixo_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, x20
```

```
bl converte_num_para_string
```

```
mov x2, x1
```

```
mov x1, x0
```

```
mov x0, #1
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, #1
```

```
ldr x1, =msg_timestamp_prefixo
```

```
mov x2, #msg_timestamp_prefixo_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
bl obtem_timestamp
```

```
bl converte_num_para_string
```

```
mov x2, x1
```

```
mov x1, x0
```

```
mov x0, #1
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
mov x0, #1
```

```
ldr x1, =msg_timestamp_sufixo
mov x2, #msg_timestamp_sufixo_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, #1
ldr x1, =msg_cabecalho_meio
mov x2, #msg_cabecalho_meio_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x19
mov w1, #CS_GPIO
mov w2, #0
bl gpio_write
```

```
bl atraso_curto
```

```
mov w25, #COMANDO_SOLICITA
```

```
ldr x23, =rx_frame
mov x24, #0
```

```
loop_bytes:
    cmp x24, #TAMANHO_FRAME
    b.ge fim_bytes
```

```
mov x21, #0
mov x22, #0
```

```
loop_bits:
    cmp x22, #8
    b.ge fim_bits_do_byte
```

```
cmp x24, #0
b.ne pula_envio_mosi
```

```
lsr w26, w25, #7
and w26, w26, #1
mov x0, x19
mov w1, #MOSI_GPIO
mov w2, w26
bl gpio_write
lsl w25, w25, #1
```

pula_envio_mosi:

```
mov x0, x19
mov w1, #SCLK_GPIO
mov w2, #1
bl gpio_write
```

```
bl atraso_curto
```

```
mov x0, x19
mov w1, #13
bl gpio_read
```

```
lsl x21, x21, #1
orr x21, x21, x0
```

```
mov x0, x19
mov w1, #SCLK_GPIO
mov w2, #0
bl gpio_write
```

```
bl atraso_curto
```

```
add x22, x22, #1
```

```
b loop_bits
```

```
fim_bits_do_byte:
```

```
and w21, w21, #0xFF
```

```
strb w21, [x23, x24]
```

```
add x24, x24, #1
```

```
b loop_bytes
```

```
fim_bytes:
```

```
mov x0, x19
```

```
mov w1, #CS_GPIO
```

```
mov w2, #1
```

```
bl gpio_write
```

```
ldr x1, =rx_frame
```

```
ldrb w2, [x1]
```

```
cmp w2, #STATUS_OK
```

```
b.eq status_ok
```

```
mov x0, #1
```

```
ldr x1, =msg_erro_status
```

```
mov x2, #msg_erro_status_fim
```

```
mov x8, #SYS_WRITE
```

```
svc #0
```

```
b fim_recebe_info
```

```
status_ok:
```

```
mov w9, #0
```

```
mov w10, #1
```

```
verifica_checksum_loop:
```

```
cmp w10, #12
b.gt verifica_checksum_fim
ldrb w11, [x1, x10]
add w9, w9, w11
add w10, w10, #1
b verifica_checksum_loop
```

verifica_checksum_fim:

```
and w9, w9, #0xFF
ldrb w12, [x1, #13]
cmp w9, w12
b.eq checksum_ok
```

```
mov x0, #1
ldr x1, =msg_erro_checksum
mov x2, #msg_erro_checksum_fim
mov x8, #SYS_WRITE
svc #0
b fim_recebe_info
```

checksum_ok:

```
mov x0, #1
ldr x1, =msg_pressao
mov x2, #msg_pressao_fim
mov x8, #SYS_WRITE
svc #0
mov x0, #1
mov x1, #3
bl imprime_bytes_hex_n
bl imprime_quebra_linha
```

```
mov x0, #1
ldr x1, =msg_temperatura
```

```
mov x2, #msg_temperatura_fim
mov x8, #SYS_WRITE
svc #0

mov x0, #4
mov x1, #3
bl imprime_bytes_hex_n
bl imprime_quebra_linha
```

```
mov x0, #1
ldr x1, =msg_umidade
mov x2, #msg_umidade_fim
mov x8, #SYS_WRITE
svc #0

mov x0, #7
mov x1, #2
bl imprime_bytes_hex_n
bl imprime_quebra_linha
```

```
mov x0, #1
ldr x1, =msg_luminosidade
mov x2, #msg_luminosidade_fim
mov x8, #SYS_WRITE
svc #0

mov x0, #9
mov x1, #2
bl imprime_bytes_hex_n
bl imprime_quebra_linha
```

```
mov x0, #1
ldr x1, =msg_umidade_solo
mov x2, #msg_umidade_solo_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, #11
mov x1, #2
bl imprime_bytes_hex_n
bl imprime_quebra_linha

ldr x1, =rx_frame
ldrb w2, [x1, #4]
ldrb w3, [x1, #5]
ldrb w4, [x1, #6]
lsl w2, w2, #16
lsl w3, w3, #8
orr w2, w2, w3
orr w2, w2, w4
asr w0, w2, #4
bl converte_temp
mov x21, x0

mov x0, #1
ldr x1, =msg_temp_convertida
mov x2, #msg_temp_convertida_fim
mov x8, #SYS_WRITE
svc #0

mov x0, x21
mov x1, #100
mov x2, #2
bl imprime_valor_fixo

mov x0, #1
ldr x1, =msg_graus_c
mov x2, #msg_graus_c_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x21
ldr x1, =tabela_temp
ldr x2, =indice_temp
bl armazena_amostra
```

```
ldr x1, =rx_frame
ldrb w2, [x1, #7]
ldrb w3, [x1, #8]
lsl w2, w2, #8
orr w0, w2, w3
bl converte_umidade_ar
mov x22, x0
```

```
mov x0, #1
ldr x1, =msg_umidade_ar_convertida
mov x2, #msg_umidade_ar_convertida_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x22
mov x1, #100
mov x2, #2
bl imprime_valor_fixo
```

```
mov x0, #1
ldr x1, =msg_percent
mov x2, #msg_percent_fim
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x22
ldr x1, =tabela_umidade_ar
```

```
ldr x2, =indice_umidade_ar
bl armazena_amostra

ldr x1, =rx_frame
ldrb w2, [x1, #9]
ldrb w3, [x1, #10]
lsl w2, w2, #8
orr w0, w2, w3
bl converte_luz
mov x23, x0

mov x0, #1
ldr x1, =msg_luz_convertida
mov x2, #msg_luz_convertida_fim
mov x8, #SYS_WRITE
svc #0

mov x0, x23
mov x1, #10
mov x2, #1
bl imprime_valor_fixo

mov x0, #1
ldr x1, =msg_lux
mov x2, #msg_lux_fim
mov x8, #SYS_WRITE
svc #0

mov x0, x23
ldr x1, =tabela_luz
ldr x2, =indice_luz
bl armazena_amostra
```

```

ldr x1, =rx_frame
ldrb w2, [x1, #11]
ldrb w3, [x1, #12]
lsl w2, w2, #8
orr w0, w2, w3
bl converte_umidade_solo
mov x24, x0

mov x0, #1
ldr x1, =msg_umidade_solo_convertida
mov x2, #msg_umidade_solo_convertida_fim
mov x8, #SYS_WRITE
svc #0

mov x0, x24
mov x1, #100
mov x2, #2
bl imprime_valor_fixo

mov x0, #1
ldr x1, =msg_percent
mov x2, #msg_percent_fim
mov x8, #SYS_WRITE
svc #0

mov x0, x24
ldr x1, =tabela_solo
ldr x2, =indice_solo
bl armazena_amostra

fim_recebe_info:
ldp x25, x26, [sp, 64]
ldp x23, x24, [sp, 48]

```

```
ldp x21, x22, [sp, 32]
ldp x19, x20, [sp, 16]
ldp x29, x30, [sp], 80
ret
```

imprime_valor_fixo:

```
stp x29, x30, [sp, -48]!
mov x29, sp
stp x19, x20, [sp, 16]
stp x21, x22, [sp, 32]
```

```
mov x19, x1
mov x20, x2
```

```
udiv x9, x0, x19
msub x21, x9, x19, x0
```

```
mov x0, x9
bl converte_num_para_string
mov x2, x1
mov x1, x0
mov x0, #1
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, #1
ldr x1, =msg_ponto
mov x2, #1
mov x8, #SYS_WRITE
svc #0
```

```
mov x0, x21
bl converte_num_para_string
```

```

mov x21, x0
mov x22, x1

subs x3, x20, x22
ble imprime_fracao_fixo

loop_zeros_fixo:
    mov x0, #1
    ldr x1, =msg_zero
    mov x2, #1
    mov x8, #SYS_WRITE
    svc #0
    subs x3, x3, #1
    bgt loop_zeros_fixo

imprime_fracao_fixo:
    mov x0, #1
    mov x1, x21
    mov x2, x22
    mov x8, #SYS_WRITE
    svc #0

    ldp x21, x22, [sp, 32]
    ldp x19, x20, [sp, 16]
    ldp x29, x30, [sp], 48
    ret

converte_num_para_string:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    ldr x9, =(buffer_iteracao + 20)
    mov x10, #10

```

converte_loop:

```
    udiv x11, x0, x10
    msub x12, x11, x10, x0
    add w12, w12, #0x30
    sub x9, x9, #1
    strb w12, [x9]
    mov x0, x11
    cmp x0, #0
    b.ne converge_loop
```

```
    ldr x13, =(buffer_iteracao + 20)
    sub x1, x13, x9
    mov x0, x9
```

```
    ldp x29, x30, [sp], 16
    ret
```

imprime_bytes_hex_n:

```
    stp x29, x30, [sp, -16]!
    mov x29, sp
    mov x14, x0
    mov x15, x1
    ldr x12, =rx_frame
    ldr x13, =buffer_hex
```

loop_imprime_hex:

```
    cmp x15, #0
    b.eq fim_imprime_hex

    ldrb w16, [x12, x14]

    lsr w17, w16, #4
```

```

    and w17, w17, #0xF
    add w17, w17, #0x30
    cmp w17, #0x3A
    b.lt nibble_alto_ok2
    add w17, w17, #7
nibble_alto_ok2:
    strb w17, [x13]

    and w18, w16, #0xF
    add w18, w18, #0x30
    cmp w18, #0x3A
    b.lt nibble_baixo_ok2
    add w18, w18, #7
nibble_baixo_ok2:
    strb w18, [x13, #1]

    mov x0, #1
    mov x1, x13
    mov x2, #2
    mov x8, #SYS_WRITE
    svc #0

    add x14, x14, #1
    sub x15, x15, #1
    b loop_imprime_hex

fim_imprime_hex:
    ldp x29, x30, [sp], 16
    ret

imprime_quebra_linha:
    stp x29, x30, [sp, -16]!
    mov x29, sp

```

```
mov x0, #1
ldr x1, =msg_quebra
mov x2, #1
mov x8, #SYS_WRITE
svc #0
ldp x29, x30, [sp], 16
ret
```

atraso_curto:

```
stp x29, x30, [sp, -16]!
mov x29, sp
ldr x0, =tempo_espera
mov x1, #0
mov x8, #SYS_NANOSLEEP
svc #0
ldp x29, x30, [sp], 16
ret
```

obtem_timestamp:

```
stp x29, x30, [sp, -16]!
mov x29, sp

mov x0, #CLOCK_REALTIME
ldr x1, =buffer_timespec
mov x8, #SYS_CLOCK_GETTIME
svc #0

ldr x0, =buffer_timespec
ldr x0, [x0]

ldp x29, x30, [sp], 16
ret
```

```
.section .data
msg_quebra: .asciz "\\n"
```

Este arquivo concentra a comunicação SPI do lado do Raspberry Pi durante o funcionamento normal do sistema, além de diversas rotinas utilitárias de impressão e conversão usadas em outros pontos do programa. A rotina `recebe_info` começa imprimindo no terminal o número da iteração atual e o timestamp Unix correspondente, obtido através de `obtem_timestamp`, formando um cabeçalho que ajuda a acompanhar o funcionamento do sistema durante testes prolongados. Em seguida, a seleção de chip é abaixada e a rotina realiza a transação SPI completa com a FPGA, enviando o byte de comando `0xa5`, guardado na constante `COMANDO_SOLICITA`, pela linha `mosi`, um bit por vez através do registrador `w25`, enquanto lê simultaneamente os catorze bytes de resposta pela linha `miso`, já que o protocolo implementado é full duplex, com os dois lados transmitindo e recebendo ao mesmo tempo sobre os mesmos pulsos de clock. O laço externo `loop_bytes` percorre os catorze bytes do quadro, e o laço interno `loop_bits` percorre os oito bits de cada byte, sendo que o envio do bit de comando só acontece durante o primeiro byte da transação, condição verificada pelo desvio `pula_envio_mosi`, já que os treze bytes restantes não carregam nenhuma informação enviada pelo mestre.

Antes de processar qualquer valor recebido, a rotina confere se o primeiro byte do quadro, o status, corresponde ao valor esperado `STATUS_OK`, descartando a leitura inteira e imprimindo a mensagem de erro apropriada caso esse valor não bata. Superada essa primeira verificação, a rotina recalcula o checksum somando os doze bytes de dados, do índice um ao doze, através do laço `verifica_checksum_loop`, e compara o resultado, truncado para oito bits com uma operação `and`, contra o último byte do quadro, descartando novamente a leitura inteira em caso de divergência, através de uma segunda mensagem de erro específica para essa falha. Essa dupla verificação, de status e de checksum, é o que garante que uma falha de comunicação nunca seja silenciosamente tratada como se fosse uma leitura válida do ambiente.

Passadas as duas verificações, a rotina imprime os cinco valores brutos em hexadecimal através da rotina auxiliar `imprime_bytes_hex_n`, e então converte cada um deles chamando a rotina de conversão apropriada, `converte_temp`, `converte_umidade_ar`, `converte_luz` e `converte_umidade_solo`, sempre remontando o valor de dezesseis ou vinte e quatro bits a partir dos bytes individuais do quadro através de deslocamentos e operações `or`. Cada valor já convertido é impresso em formato de ponto fixo através de

`imprime_valor_fixo` e em seguida armazenado na tabela de amostras correspondente através da rotina `armazena_amostra`, alimentando assim o histórico que `fsm_principal.s` usa mais tarde para calcular as médias de cinco minutos.

As rotinas auxiliares definidas no restante do arquivo cumprem papéis bem específicos. `imprime_valor_fixo` formata um valor inteiro como se fosse um número de ponto fixo, separando a parte inteira da fracionária através de divisão inteira e completando com zeros à esquerda quando necessário. `converte_num_para_string` transforma um número inteiro em sua representação textual, construindo os dígitos de trás para frente dentro de `buffer_iteracao` através de divisões sucessivas por dez, já que o resto de cada divisão corresponde exatamente ao próximo dígito, do menos significativo para o mais significativo. `imprime_bytes_hex_n` imprime uma quantidade especificada de bytes do quadro recebido em formato hexadecimal, convertendo cada nibble de quatro bits em seu caractere correspondente, somando sete extra quando o valor cai na faixa das letras de A a F. Por fim, `atraso_curto` aplica uma pequena pausa entre transações SPI através de `nanosleep`, e `obtem_timestamp` lê o relógio do sistema através da chamada de sistema `clock_gettime`, usando o relógio `CLOCK_REALTIME`, que reflete a hora real do sistema e não apenas o tempo decorrido desde a inicialização.

converte_temp.s

```
.global converge_temp
.global t_fine_global
.extern dig_t1_valor
.extern dig_t2_valor
.extern dig_t3_valor

.section .data
.align 8
t_fine_global: .quad 0

.section .text

converte_temp:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    mov w9, w0

    ldr x10, =dig_t1_valor
    ldrh w10, [x10]
    ldr x11, =dig_t2_valor
    ldrsh w11, [x11]
    ldr x12, =dig_t3_valor
    ldrsh w12, [x12]

    asr w13, w9, #3
    lsl w14, w10, #1
    sub w13, w13, w14
    mul w13, w13, w11
    asr w13, w13, #11

    asr w14, w9, #4
```

```

sub w14, w14, w10
mul w15, w14, w14
asr w15, w15, #12
mul w15, w15, w12
asr w15, w15, #14

add w16, w13, w15

sxtw x16, w16
ldr x17, =t_fine_global
str x16, [x17]

mov w0, #5
mul w0, w16, w0
add w0, w0, #128
asr w0, w0, #8
sxtw x0, w0

ldp x29, x30, [sp], 16
ret

```

Esta rotina implementa a fórmula de compensação de temperatura do BME280 exatamente como especificada no datasheet do fabricante, usando aritmética inteira em vez de ponto flutuante, o que exige bastante cuidado com deslocamentos e arredondamentos em cada etapa. O valor bruto de vinte bits recebido em w0 é combinado com os três coeficientes de calibração de fábrica, dig_t1_valor, dig_t2_valor e dig_t3_valor, lidos da seção de dados de coeficientes_calibracao.s, sendo o primeiro tratado como um valor sem sinal, através de ldrh, e os outros dois como valores com sinal, através de ldrsh, já que é assim que o próprio datasheet define esses três coeficientes.

O cálculo é dividido em duas parcelas somadas ao final. A primeira parcela subtrai do valor bruto deslocado três bits à direita o dobro do primeiro coeficiente, multiplica o resultado pelo segundo coeficiente e desloca o produto onze bits à direita. A segunda parcela subtrai do valor bruto deslocado quatro bits à direita o primeiro coeficiente, eleva esse resultado ao quadrado através de uma multiplicação por si mesmo, desloca doze bits

à direita, multiplica pelo terceiro coeficiente e desloca mais catorze bits à direita. A soma dessas duas parcelas, guardada na variável global `t_fine_global`, é um valor intermediário de grande importância, já que ele não representa a temperatura final, mas sim um fator de ajuste fino que também é reutilizado pela fórmula de compensação de umidade em `converte_umidade_ar.s`, o que exige que essa rotina seja sempre chamada antes daquela, nunca depois.

Por fim, a temperatura em graus Celsius multiplicada por cem, para preservar duas casas decimais, é obtida multiplicando `t_fine_global` por cinco, somando cento e vinte e oito para arredondamento e deslocando o resultado oito bits à direita, replicando exatamente a fórmula de ponto fixo sugerida pelo datasheet para essa conversão final.

converte_umidade_ar.s

```
.global converge_umidade_ar
.extern t_fine_global
.extern dig_h1_valor
.extern bloco_h2_h6_valor

.section .text

converte_umidade_ar:
    stp x29, x30, [sp, -80]!
    mov x29, sp
    stp x19, x20, [sp, 16]
    stp x21, x22, [sp, 32]
    stp x23, x24, [sp, 48]
    stp x25, x26, [sp, 64]

    mov w19, w0

    ldr x1, =dig_h1_valor
    ldrb w20, [x1]

    ldr x1, =bloco_h2_h6_valor
    ldrb w2, [x1, #0]
    ldrb w3, [x1, #1]
    ldrb w4, [x1, #2]
    ldrb w5, [x1, #3]
    ldrb w6, [x1, #4]
    ldrb w7, [x1, #5]
    ldrb w8, [x1, #6]

    lsl w21, w3, #8
    orr w21, w21, w2
    sxth w21, w21
```

```
mov w22, w4
```

```
and w9, w6, #0xF
```

```
lsl w23, w5, #4
```

```
orr w23, w23, w9
```

```
lsl w23, w23, #20
```

```
asr w23, w23, #20
```

```
lsr w9, w6, #4
```

```
lsl w24, w7, #4
```

```
orr w24, w24, w9
```

```
lsl w24, w24, #20
```

```
asr w24, w24, #20
```

```
sxtb w25, w8
```

```
ldr x1, =t_fine_global
```

```
ldr w26, [x1]
```

```
ldr w9, =76800
```

```
sub w26, w26, w9
```

```
lsl w9, w19, #14
```

```
lsl w10, w23, #20
```

```
sub w9, w9, w10
```

```
mul w11, w24, w26
```

```
sub w9, w9, w11
```

```
add w9, w9, #16384
```

```
asr w9, w9, #15
```

```
mul w12, w26, w25
```

```
asr w12, w12, #10
```

```

mul w13, w26, w22
asr w13, w13, #11
add w13, w13, #32768
mul w12, w12, w13
asr w12, w12, #10
add w12, w12, #2097152
mul w12, w12, w21
add w12, w12, #8192
asr w12, w12, #14

mul w14, w9, w12

asr w15, w14, #15
mul w16, w15, w15
asr w16, w16, #7
mul w16, w16, w20
asr w16, w16, #4
sub w14, w14, w16

cmp w14, #0
bge nao_negativo_umid
mov w14, #0
nao_negativo_umid:
ldr w17, =419430400
cmp w14, w17
ble nao_estoura_umid
mov w14, w17
nao_estoura_umid:

lsr w0, w14, #12

mov w1, #100
mul w0, w0, w1

```

```
mov w1, #1024
udiv w0, w0, w1

ldp x25, x26, [sp, 64]
ldp x23, x24, [sp, 48]
ldp x21, x22, [sp, 32]
ldp x19, x20, [sp, 16]
ldp x29, x30, [sp], 80
ret
```

Esta rotina implementa a fórmula de compensação de umidade do ar do BME280, que é a mais elaborada das três conversões de sensor do projeto, dependendo tanto do valor bruto de umidade recebido em w0 quanto do valor de t_fine_global calculado previamente por converte_temp.s. Logo no início, os seis coeficientes de calibração relacionados à umidade são extraídos de bloco_h2_h6_valor, um bloco de sete bytes lido diretamente do sensor físico, através de uma sequência de deslocamentos e máscaras que remonta cada coeficiente em sua largura de bits e sinal corretos, já que o BME280 empacota esses seis valores de forma bem compacta e não alinhada a bytes inteiros, com alguns deles usando apenas doze bits e outros usando quatro bits compartilhados entre dois campos diferentes.

O núcleo do cálculo segue de perto a fórmula de compensação de ponto fixo de trinta e dois bits sugerida pelo datasheet, calculando primeiro um termo intermediário que combina o valor bruto de umidade, o segundo coeficiente extraído e o valor ajustado de temperatura, subtraído de 76800, e depois um segundo termo que combina os coeficientes terceiro, sexto, quinto e primeiro em uma sequência de multiplicações e deslocamentos que se encaixam exatamente na ordem definida pelo fabricante. O produto final entre esses dois termos intermediários sofre ainda uma correção adicional, subtraindo um termo proporcional ao quadrado do próprio resultado ajustado pelo primeiro coeficiente de umidade, o que compensa uma não linearidade conhecida do sensor.

Depois de calculado, o resultado passa por duas saturações, sendo forçado para zero caso fique negativo e para o valor máximo representável, 419430400, caso ultrapasse esse limite, já que a fórmula original pode produzir valores fora da faixa fisicamente possível em condições extremas de entrada. Por fim, o valor é deslocado doze bits à direita para trazê-lo de volta à escala de ponto fixo Q22.10 usada internamente pelo BME280, e

então multiplicado por cem e dividido por 1024 para converter essa escala para um percentual multiplicado por cem, pronto para ser impresso com duas casas decimais pelas rotinas de exibição.

converte_umidade_solo.s

```
.global converge_umidade_solo

.section .data
.align 8
LIMIAR_SECO_VALOR: .word 600
LIMIAR_MOLHADO_VALOR: .word 130

.section .text

converte_umidade_solo:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    mov w9, w0

    ldr x1, =LIMIAR_SECO_VALOR
    ldr w10, [x1]
    ldr x1, =LIMIAR_MOLHADO_VALOR
    ldr w11, [x1]

    cmp w9, w10
    bgt satura_seco_solo
    cmp w9, w11
    blt satura_molhado_solo

    sub w12, w10, w9
    sub w13, w10, w11
    mov w14, #10000
    mul w12, w12, w14
    udiv w0, w12, w13
    b fim_conversao_solo
```

```

satura_seco_solo:
    mov w0, #0
    b fim_conversao_solo

satura_molhado_solo:
    mov w0, #10000

fim_conversao_solo:
    sxtw x0, w0
    ldp x29, x30, [sp], 16
    ret

```

Esta rotina recebe em w0 o valor bruto de dez bits lido do conversor analógico digital MCP3008 para o sensor de umidade do solo e devolve em w0 o percentual de umidade correspondente, já multiplicado por cem para preservar duas casas decimais de precisão sem precisar recorrer a ponto flutuante, algo que se repete em várias outras rotinas de conversão do projeto. Logo no início, o valor recebido é copiado para w9, e as duas constantes de calibração, LIMIAR_SECO_VALOR e LIMIAR_MOLHADO_VALOR, são carregadas da seção de dados para w10 e w11 respectivamente, sendo seiscentos o valor bruto esperado quando o solo está completamente seco e cento e trinta o valor esperado quando está completamente molhado, valores esses obtidos empiricamente ao testar o sensor físico nas duas condições extremas.

Como o sensor entrega valores mais altos quanto mais seco está o solo, o que é o oposto do que normalmente se espera de um percentual de umidade, a lógica precisa inverter essa relação, e é isso que os dois primeiros desvios condicionais fazem. Se w9 for maior que o limiar de seco, guardado em w10, o fluxo salta para satura_seco_solo, retornando zero por cento diretamente, já que qualquer valor acima desse limiar já representa o extremo seco. De forma semelhante, se w9 for menor que o limiar de molhado, guardado em w11, o fluxo salta para satura_molhado_solo, retornando dez mil, que representa cem por cento depois de multiplicado pela escala de duas casas decimais.

Quando o valor lido está dentro da faixa esperada, entre os dois limiares, a rotina aplica uma interpolação linear simples para converter esse valor bruto em percentual. Primeiro é calculada a diferença entre o limiar seco e o valor lido, em w12, e a diferença

entre o limiar seco e o limiar molhado, em w13, representando respectivamente o quanto o valor atual já se afastou do extremo seco e a largura total da faixa de calibração. Em seguida, w12 é multiplicado por dez mil, através da constante carregada em w14, e o resultado é dividido por w13 usando a instrução udiv, que realiza divisão inteira sem sinal, já que todos os valores envolvidos nessa conta são sempre positivos. O resultado dessa divisão, ainda em w0, representa o percentual de umidade multiplicado por cem, sendo por fim estendido de trinta e dois para sessenta e quatro bits através da instrução sxtw antes de a função retornar, garantindo que o valor seja tratado corretamente nos registradores de sessenta e quatro bits usados pelo restante do programa.

converte_luz.s

```
.global converge_luz

.section .text

converte_luz:
    mov w1, #25
    mul w0, w0, w1
    mov w1, #3
    udiv w0, w0, w1
    ret
```

Esta rotina converte o valor bruto de dezesseis bits entregue pelo BH1750 em um valor de luminosidade em lux, multiplicado por dez para preservar uma casa decimal de precisão sem depender de ponto flutuante. O datasheet do BH1750 especifica que o valor bruto deve ser dividido por 1,2 para se obter o lux, e a rotina reproduz essa divisão através de uma aproximação inteira equivalente, multiplicando o valor recebido em w0 por vinte e cinco e em seguida dividindo o resultado por três através da instrução udiv, já que vinte e cinco dividido por três é aproximadamente igual a 1 dividido por 1,2 multiplicado por dez, e essa forma evita qualquer arredondamento intermediário que uma divisão direta por 1,2 exigiria caso fosse feita em ponto flutuante. Por ser tão direta, essa é a menor das rotinas de conversão do projeto, sem nenhuma dependência de coeficientes externos ou de outras rotinas.

coeficientes_calibracao.s

```
.global dig_t1_valor  
.global dig_t2_valor  
.global dig_t3_valor  
.global dig_p1_valor  
.global dig_p2_valor  
.global dig_p3_valor  
.global dig_p4_valor  
.global dig_p5_valor  
.global dig_p6_valor  
.global dig_p7_valor  
.global dig_p8_valor  
.global dig_p9_valor  
.global dig_h1_valor  
.global bloco_h2_h6_valor
```

```
.section .data
```

```
.align 8
```

```
dig_t1_valor: .hword 0x6FA3
```

```
dig_t2_valor: .hword 0x680E
```

```
dig_t3_valor: .hword 0x0032
```

```
dig_p1_valor: .hword 0x83E6
```

```
dig_p2_valor: .hword 0xD665
```

```
dig_p3_valor: .hword 0x0BD0
```

```
dig_p4_valor: .hword 0x160C
```

```
dig_p5_valor: .hword 0x00FE
```

```
dig_p6_valor: .hword 0xFFFF9
```

```
dig_p7_valor: .hword 0x2DB4
```

```
dig_p8_valor: .hword 0xD1E8
```

```
dig_p9_valor: .hword 0x1388
```

```
dig_h1_valor: .byte 0x4B
```

```
.align 8
```

```
bloco_h2_h6_valor: .byte 0x70, 0x01, 0x00, 0x13, 0x21, 0x03, 0x1E
```

Este arquivo não contém nenhuma rotina executável, apenas os valores fixos dos coeficientes de calibração de fábrica do BME280 utilizado neste projeto, obtidos uma única vez diretamente do sensor físico e fixados em código a partir de então. Os coeficientes de temperatura, dig_t1_valor a dig_t3_valor, e os de pressão, dig_p1_valor a dig_p9_valor, são todos declarados como meias palavras de dezesseis bits através da diretiva hword, já que é assim que o BME280 os armazena internamente em seus próprios registradores de calibração. Os coeficientes de umidade seguem um formato mais irregular, com dig_h1_valor ocupando um único byte e os cinco coeficientes seguintes, de h2 a h6, compactados juntos no bloco bloco_h2_h6_valor de sete bytes, com alguns campos compartilhando bytes entre si, exatamente como o datasheet do sensor especifica esse bloco de calibração de umidade.

Essa abordagem, de ler os coeficientes uma única vez e fixá-los em código, substitui uma versão anterior do projeto na qual esses mesmos coeficientes eram transmitidos pela FPGA a cada ciclo de leitura, prática abandonada por não haver necessidade real de retransmitir constantemente um valor que nunca muda durante a execução do sistema, economizando assim largura de banda na interface SPI a cada quadro de doze bytes.

tabelas_amostras.s

```
.global tabela_temp
.global indice_temp
.global tabela_umidade_ar
.global indice_umidade_ar
.global tabela_luz
.global indice_luz
.global tabela_solo
.global indice_solo
.global armazena_amostra

.equ TAMANHO_TABELA, 200

.section .data
.align 8
tabela_temp:    .skip (TAMANHO_TABELA * 4)
indice_temp:    .word 0

tabela_umidade_ar: .skip (TAMANHO_TABELA * 4)
indice_umidade_ar: .word 0

tabela_luz:      .skip (TAMANHO_TABELA * 4)
indice_luz:      .word 0

tabela_solo:     .skip (TAMANHO_TABELA * 4)
indice_solo:     .word 0

.section .text

armazena_amostra:
    stp x29, x30, [sp, -16]!
    mov x29, sp
```

```

ldr w9, [x2]
cmp w9, #TAMANHO_TABELA
bge fim_armazena_amostra

lsl x10, x9, #2
str w0, [x1, x10]

add w9, w9, #1
str w9, [x2]

fim_armazena_amostra:
ldp x29, x30, [sp], 16
ret

```

Este arquivo implementa o mecanismo de armazenamento das leituras convertidas ao longo da janela de cinco minutos usada pela lógica de decisão em `fsm_principal.s`. Quatro tabelas independentes são reservadas na seção de dados, uma para cada parâmetro ambiental, `tabela_temp`, `tabela_umidade_ar`, `tabela_luz` e `tabela_solo`, cada uma com espaço para duzentas posições de quatro bytes cada, definido pela constante `TAMANHO_TABELA`, e cada tabela é acompanhada de um índice próprio, `indice_temp`, `indice_umidade_ar`, `indice_luz` e `indice_solo`, que guarda quantas posições já foram preenchidas naquela tabela específica.

A rotina `armazena_amostra` recebe o valor a ser guardado em `x0`, o endereço base da tabela em `x1` e o endereço do índice correspondente em `x2`, e sua lógica é propositalmente simples: o índice atual é lido, comparado contra o limite de duzentas posições, e, caso ainda haja espaço, o valor é escrito na posição calculada através de um deslocamento de dois bits à esquerda sobre o índice, já que cada posição ocupa quatro bytes, e o índice é então incrementado. Caso o limite já tenha sido atingido, a rotina simplesmente não escreve nada e retorna sem erro, um comportamento seguro já que, na prática, `fsm_principal.s` sempre lê e zera esse índice antes de as cento e cinquenta amostras da janela se completarem, tornando essa condição de limite uma proteção que nunca chega a ser acionada durante a operação normal do sistema.

calcula_info.s

```
.global calcula_media
.global calcula_media_benchmark
.extern converte_num_para_string

.equ SYS_WRITE, 64
.equ SYS_CLOCK_GETTIME, 113
.equ CLOCK_MONOTONIC, 1
.equ BENCH_TAMANHO, 150
.equ BENCH_REPETICOES, 200000

.macro zera_acumulador_vetorial reg
    movi \reg\().4s, #0
.endm

.macro carrega_vetor_generico registro_dst, registro_ptr, offset=16
    ld1 {\registro_dst\().4s}, [\registro_ptr], #\offset
.endm

.section .data
.align 4
bench_buffer: .skip (BENCH_TAMANHO * 4)

.align 8
bench_ts_inicio:
    .quad 0
    .quad 0
bench_ts_fim:
    .quad 0
    .quad 0

msg_bench_titulo: .asciz "\n=== Benchmark calcula_media (150 amostras, 200000
repeticoes) ===\n"
```

```

msg_bench_titulo_fim = . - msg_bench_titulo - 1

msg_verificacao_ok: .asciz "Verificacao: os tres metodos produziram a mesma soma
(OK)\n"
msg_verificacao_ok_fim = . - msg_verificacao_ok - 1

msg_verificacao_falha: .asciz "Verificacao: DIVERGENCIA entre os metodos\n"
msg_verificacao_falha_fim = . - msg_verificacao_falha - 1


msg_bench_escalar: .asciz "Escalar:      "
msg_bench_escalar_fim = . - msg_bench_escalar - 1

msg_bench_vet_int: .asciz "Vetorial (inteiro): "
msg_bench_vet_int_fim = . - msg_bench_vet_int - 1

msg_bench_vet_float: .asciz "Vetorial (float):  "
msg_bench_vet_float_fim = . - msg_bench_vet_float - 1


msg_bench_ns: .asciz " ns totais\n"
msg_bench_ns_fim = . - msg_bench_ns - 1


.section .text

calcula_media:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    ldr w9, [x1]
    cmp w9, #0
    beq media_vazia

    bl soma_vetorial_inteira

```

```
sxtw x9, w9
sdiv x0, x10, x9
b zera_indice_media
```

media_vazia:

```
mov x0, #0
```

zera_indice_media:

```
mov w9, #0
str w9, [x1]
```

```
ldp x29, x30, [sp], 16
ret
```

soma_escalar:

```
mov x10, #0
mov w11, #0
```

soma_escalar_loop:

```
cmp w11, w9
bge fim_soma_escalar
ldr w12, [x0, x11, lsl #2]
sxtw x12, w12
add x10, x10, x12
add w11, w11, #1
b soma_escalar_loop
```

fim_soma_escalar:

```
ret
```

soma_vetorial_inteira:

```
zera_acumulador_vetorial v1
lsr w13, w9, #2
```

```
mov w11, #0
```

```
grupo_vetorial_inteira:
```

```
    cbz w13, fim_grupo_vetorial_inteira
```

```
    carrega_vetor_generico v0, x0
```

```
    add v1.4s, v1.4s, v0.4s
```

```
    sub w13, w13, #1
```

```
    add w11, w11, #4
```

```
    b grupo_vetorial_inteira
```

```
fim_grupo_vetorial_inteira:
```

```
    addv s1, v1.4s
```

```
    fmov w10, s1
```

```
resto_vetorial_inteira:
```

```
    cmp w11, w9
```

```
    bge fim_soma_vetorial_inteira
```

```
    ldr w12, [x0], #4
```

```
    add w10, w10, w12
```

```
    add w11, w11, #1
```

```
    b resto_vetorial_inteira
```

```
fim_soma_vetorial_inteira:
```

```
    sxtw x10, w10
```

```
    ret
```

```
soma_vetorial_float:
```

```
    zera_acumulador_vetorial v1
```

```
    lsr w13, w9, #2
```

```
    mov w11, #0
```

```
grupo_vetorial_float:
```

```
    cbz w13, fim_grupo_vetorial_float
```

```
carrega_vetor_generico v0, x0
scvtf v0.4s, v0.4s
fadd v1.4s, v1.4s, v0.4s
sub w13, w13, #1
add w11, w11, #4
b grupo_vetorial_float
```

fim_grupo_vetorial_float:

```
faddp v2.4s, v1.4s, v1.4s
faddp s0, v2.2s
```

resto_vetorial_float:

```
cmp w11, w9
bge fim_soma_vetorial_float
ldr w12, [x0], #4
scvtf s3, w12
fadd s0, s0, s3
add w11, w11, #1
b resto_vetorial_float
```

fim_soma_vetorial_float:

```
ret
```

calcula_delta_ns:

```
ldr x9, [x0]
ldr x10, [x0, 8]
ldr x11, [x1]
ldr x12, [x1, 8]
sub x13, x11, x9
sub x14, x12, x10
ldr x15, =1000000000
mul x13, x13, x15
add x0, x13, x14
```

ret

calcula_media_benchmark:

stp x29, x30, [sp, -64]!

mov x29, sp

stp x19, x20, [sp, 16]

stp x21, x22, [sp, 32]

str x23, [sp, 48]

ldr x19, =bench_buffer

mov w11, #0

bench_preenche_loop:

cmp w11, #BENCH_TAMANHO

bge bench_preenche_fim

mov w12, w11

mov w13, #7

mul w12, w12, w13

sub w12, w12, #350

str w12, [x19, x11, lsl #2]

add w11, w11, #1

b bench_preenche_loop

bench_preenche_fim:

mov x0, #1

ldr x1, =msg_bench_titulo

mov x2, #msg_bench_titulo_fim

mov x8, #SYS_WRITE

svc #0

mov x0, x19

mov w9, #BENCH_TAMANHO

bl soma_escalar

mov x21, x10

```
mov x0, x19
mov w9, #BENCH_TAMANHO
bl soma_vetorial_inteira
mov x22, x10
```

```
mov x0, x19
mov w9, #BENCH_TAMANHO
bl soma_vetorial_float
fcvtzs x23, s0
```

```
cmp x21, x22
bne verificacao_falhou
cmp x21, x23
bne verificacao_falhou
```

```
mov x0, #1
ldr x1, =msg_verificacao_ok
mov x2, #msg_verificacao_ok_fim
mov x8, #SYS_WRITE
svc #0
b verificacao_fim
```

verificacao_falhou:

```
mov x0, #1
ldr x1, =msg_verificacao_falha
mov x2, #msg_verificacao_falha_fim
mov x8, #SYS_WRITE
svc #0
```

verificacao_fim:

```
mov x0, #CLOCK_MONOTONIC
```

```

ldr x1, =bench_ts_inicio
mov x8, #SYS_CLOCK_GETTIME
svc #0
mov w20, #0
bench_loop_escalar:
ldr w14, =BENCH_REPETICOES
cmp w20, w14
bge bench_fim_loop_escalar
mov x0, x19
mov w9, #BENCH_TAMANHO
bl soma_escalar
add w20, w20, #1
b bench_loop_escalar
bench_fim_loop_escalar:
mov x0, #CLOCK_MONOTONIC
ldr x1, =bench_ts_fim
mov x8, #SYS_CLOCK_GETTIME
svc #0
ldr x0, =bench_ts_inicio
ldr x1, =bench_ts_fim
bl calcula_delta_ns
mov x21, x0
mov x0, #1
ldr x1, =msg_bench_escalar
mov x2, #msg_bench_escalar_fim
mov x8, #SYS_WRITE
svc #0
mov x0, x21
bl converte_num_para_string
mov x2, x1
mov x1, x0
mov x0, #1
mov x8, #SYS_WRITE

```

```

svc #0

mov x0, #1
ldr x1, =msg_bench_ns
mov x2, #msg_bench_ns_fim
mov x8, #SYS_WRITE
svc #0

mov x0, #CLOCK_MONOTONIC
ldr x1, =bench_ts_inicio
mov x8, #SYS_CLOCK_GETTIME
svc #0
mov w20, #0
bench_loop_vetorial:
    ldr w14, =BENCH_REPETICOES
    cmp w20, w14
    bge bench_fim_loop_vetorial
    mov x0, x19
    mov w9, #BENCH_TAMANHO
    bl soma_vetorial_inteira
    add w20, w20, #1
    b bench_loop_vetorial
bench_fim_loop_vetorial:
    mov x0, #CLOCK_MONOTONIC
    ldr x1, =bench_ts_fim
    mov x8, #SYS_CLOCK_GETTIME
    svc #0
    ldr x0, =bench_ts_inicio
    ldr x1, =bench_ts_fim
    bl calcula_delta_ns
    mov x21, x0
    mov x0, #1
    ldr x1, =msg_bench_vet_int
    mov x2, #msg_bench_vet_int_fim

```

```

mov x8, #SYS_WRITE
svc #0

mov x0, x21
bl converte_num_para_string
mov x2, x1
mov x1, x0
mov x0, #1
mov x8, #SYS_WRITE
svc #0

mov x0, #1
ldr x1, =msg_bench_ns
mov x2, #msg_bench_ns_fim
mov x8, #SYS_WRITE
svc #0

mov x0, #CLOCK_MONOTONIC
ldr x1, =bench_ts_inicio
mov x8, #SYS_CLOCK_GETTIME
svc #0

mov w20, #0
bench_loop_float:
ldr w14, =BENCH_REPETICOES
cmp w20, w14
bge bench_fim_loop_float
mov x0, x19
mov w9, #BENCH_TAMANHO
bl soma_vetorial_float
add w20, w20, #1
b bench_loop_float
bench_fim_loop_float:
mov x0, #CLOCK_MONOTONIC
ldr x1, =bench_ts_fim
mov x8, #SYS_CLOCK_GETTIME

```

```

svc #0
ldr x0, =bench_ts_inicio
ldr x1, =bench_ts_fim
bl calcula_delta_ns
mov x21, x0
mov x0, #1
ldr x1, =msg_bench_vet_float
mov x2, #msg_bench_vet_float_fim
mov x8, #SYS_WRITE
svc #0
mov x0, x21
bl converte_num_para_string
mov x2, x1
mov x1, x0
mov x0, #1
mov x8, #SYS_WRITE
svc #0
mov x0, #1
ldr x1, =msg_bench_ns
mov x2, #msg_bench_ns_fim
mov x8, #SYS_WRITE
svc #0

ldr x23, [sp, 48]
ldp x21, x22, [sp, 32]
ldp x19, x20, [sp, 16]
ldp x29, x30, [sp], 64
ret

```

Este arquivo reúne toda a lógica de cálculo de médias do projeto, incluindo as três variantes de implementação comparadas no benchmark de desempenho, escalar, vetorial inteira e vetorial em ponto flutuante, além da própria rotina de benchmark que as compara. Duas macros são definidas logo no início, `zera_acumulador_vetorial`, que zera um registrador vetorial de quatro elementos de trinta e dois bits através da instrução `movi`,

e `carrega_vetor_generico`, que carrega quatro elementos consecutivos de memória para um registrador vetorial através de `ldl`, avançando automaticamente o ponteiro de origem pelos dezesseis bytes lidos, já que cada macro encapsula um padrão repetido várias vezes ao longo do arquivo.

A rotina `calcula_media`, chamada diretamente por `fsm_principal.s`, recebe o endereço de uma tabela de amostras e o endereço do índice correspondente, soma todos os valores acumulados através de `soma_vetorial_inteira` e divide o resultado pela quantidade de amostras através de `sdiv`, tratando o caso especial de uma tabela vazia retornando zero em vez de dividir por zero. Ao final, o índice daquela tabela é zerado, preparando a tabela para acumular a próxima janela de amostras. As três rotinas de soma propriamente ditas, `soma_escalar`, `soma_vetorial_inteira` e `soma_vetorial_float`, calculam exatamente o mesmo resultado por caminhos diferentes. A versão `escalar` percorre as amostras uma a uma em um laço simples, somando cada valor, estendido de sinal, em um acumulador de sessenta e quatro bits. As duas versões vetoriais processam quatro amostras por vez usando as macros já descritas, somando grupos inteiros em paralelo dentro do registrador vetorial `v1`, e, ao final dos grupos completos, reduzem esse vetor a um único valor através de `addv`, no caso inteiro, ou de duas chamadas a `faddp`, no caso em ponto flutuante, já que somar quatro elementos de um vetor requer uma operação dedicada de redução horizontal, diferente de somar dois registradores escalares comuns. As amostras restantes, que não completam um grupo de quatro, são somadas uma a uma logo depois, garantindo que o resultado final esteja sempre correto independentemente de a quantidade de amostras ser ou não um múltiplo de quatro.

A rotina `calcula_delta_ns` calcula a diferença em nanossegundos entre dois instantes de tempo capturados no formato `timespec`, subtraindo separadamente os campos de segundos e de nanossegundos e depois combinando os dois em um único valor através de uma multiplicação por um bilhão, já que `timespec` representa o tempo como dois campos separados e não como um único número. Por fim, `calcula_media_benchmark` preenche um buffer de cento e cinquenta amostras com valores sintéticos gerados por uma fórmula simples, confere que os três métodos de soma produzem exatamente o mesmo resultado sobre esse buffer, imprimindo uma mensagem de sucesso ou de divergência conforme o caso, e então mede, através de `clock_gettime` com o relógio `CLOCK_MONOTONIC`, o tempo total gasto por cada um dos três métodos ao longo de duzentas mil repetições, imprimindo o resultado de cada um em nanossegundos totais no terminal.

eh_dia.s

```
.global eh_dia
.extern obtem_timestamp

.equ OFFSET.UTC_SEGUNDOS, 7200
.equ HORA_INICIO_DIA, 7
.equ HORA_FIM_DIA, 21

.section .text

eh_dia:
    stp x29, x30, [sp, -16]!
    mov x29, sp

    bl obtem_timestamp

    mov x1, #OFFSET.UTC_SEGUNDOS
    add x0, x0, x1
    ldr x1, =86400
    udiv x2, x0, x1
    msub x0, x2, x1, x0

    mov x1, #3600
    udiv x0, x0, x1

    mov x1, #HORA_INICIO_DIA
    cmp x0, x1
    blt fim_eh_dia_noite

    mov x1, #HORA_FIM_DIA
    cmp x0, x1
    bge fim_eh_dia_noite
```

```
mov x0, #1
b fim_eh_dia

fim_eh_dia_noite:
mov x0, #0

fim_eh_dia:
ldp x29, x30, [sp], 16
ret
```

Esta rotina determina se o sistema está atualmente no período diurno ou noturno, informação usada por `fsm_principal.s` para decidir se a luz de crescimento deve ser considerada. A rotina começa obtendo o timestamp Unix atual através de `obtem_timestamp`, que representa a quantidade de segundos passados desde primeiro de janeiro de mil novecentos e setenta, sempre em relação ao horário UTC, e soma a esse valor um deslocamento fixo de fuso horário de sete mil e duzentos segundos, guardado na constante `OFFSET.UTC.SEGUNDOS`, o equivalente a duas horas, ajustando o horário para o fuso da Europa Central usado no local onde o sistema opera fisicamente.

Em seguida, a rotina calcula o resto da divisão desse timestamp ajustado por 86400, o número de segundos em um dia completo, através da combinação de `udiv` e `msub`, isolando assim apenas a parte do timestamp que corresponde ao horário dentro do dia atual, descartando quantos dias completos já se passaram. Esse resto, ainda em segundos, é dividido por três mil e seiscentos para obter a hora do dia como um número inteiro entre zero e vinte e três. Por fim, essa hora é comparada contra as constantes `HORA_INICIO_DIA` e `HORA_FIM_DIA`, fixadas em sete e vinte e uma respectivamente, retornando um em `x0` quando a hora atual está dentro desse intervalo, considerado o período diurno, e zero caso contrário. Vale registrar que esse deslocamento de fuso horário é fixo e não se ajusta automaticamente à mudança para o horário de verão europeu, uma limitação conscientemente aceita e documentada nas conclusões técnicas do projeto.

apresenta_info_em_tela.s

```
.global apresenta_info_em_tela
.extern lcd_posiciona_cursor
.extern lcd_escreve_string
.extern lcd_buffer_escreve
.extern lcd_buffer_completa_espacos
.extern formata_valor_fixo_buffer

.section .data
.align 8
buffer_linha_lcd: .skip 17
pos_linha_lcd: .word 0

string_T_lcd: .ascii "T:"
string_C_esp_lcd: .ascii "C "
string_U_lcd: .ascii "U:"
string_percent_lcd: .ascii "% "
string_L_lcd: .ascii "L:"
string_lux_esp_lcd: .ascii "lux "
string_S_lcd: .ascii "S:"
string_percent2_lcd: .ascii "% "

.section .text

apresenta_info_em_tela:
    stp x29, x30, [sp, -64]!
    mov x29, sp
    stp x19, x20, [sp, 16]
    stp x21, x22, [sp, 32]
    str x23, [sp, 48]

    mov x19, x0
```

```
mov x20, x1
mov x21, x2
mov x22, x3

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
mov w2, #0
str w2, [x1]

ldr x2, =string_T_lcd
mov x3, #2
bl lcd_buffer_escreve

mov x0, x19
mov x1, #10
udiv x0, x0, x1
mov x1, #10
mov x2, #1
ldr x3, =buffer_linha_lcd
ldr x4, =pos_linha_lcd
bl formata_valor_fixo_buffer

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
ldr x2, =string_C_esp_lcd
mov x3, #2
bl lcd_buffer_escreve

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
ldr x2, =string_U_lcd
mov x3, #2
bl lcd_buffer_escreve
```

```
mov x0, x20
mov x1, #100
mov x2, #0
ldr x3, =buffer_linha_lcd
ldr x4, =pos_linha_lcd
bl formata_valor_fixo_buffer

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
ldr x2, =string_percent_lcd
mov x3, #1
bl lcd_buffer_escreve

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
bl lcd_buffer_completa_espacos

mov x0, #0
mov x1, #0
bl lcd_posiciona_cursor

ldr x0, =buffer_linha_lcd
mov x1, #16
bl lcd_escreve_string

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
mov w2, #0
str w2, [x1]

ldr x2, =string_L_lcd
mov x3, #2
```

```
bl lcd_buffer_escreve
```

```
mov x0, x21
```

```
mov x1, #10
```

```
udiv x0, x0, x1
```

```
mov x1, #1
```

```
mov x2, #0
```

```
ldr x3, =buffer_linha_lcd
```

```
ldr x4, =pos_linha_lcd
```

```
bl formata_valor_fixo_buffer
```

```
ldr x0, =buffer_linha_lcd
```

```
ldr x1, =pos_linha_lcd
```

```
ldr x2, =string_lux_esp_lcd
```

```
mov x3, #4
```

```
bl lcd_buffer_escreve
```

```
ldr x0, =buffer_linha_lcd
```

```
ldr x1, =pos_linha_lcd
```

```
ldr x2, =string_S_lcd
```

```
mov x3, #2
```

```
bl lcd_buffer_escreve
```

```
mov x0, x22
```

```
mov x1, #100
```

```
mov x2, #0
```

```
ldr x3, =buffer_linha_lcd
```

```
ldr x4, =pos_linha_lcd
```

```
bl formata_valor_fixo_buffer
```

```
ldr x0, =buffer_linha_lcd
```

```
ldr x1, =pos_linha_lcd
```

```
ldr x2, =string_percent2_lcd
```

```

mov x3, #1
bl lcd_buffer_escreve

ldr x0, =buffer_linha_lcd
ldr x1, =pos_linha_lcd
bl lcd_buffer_completa_espacos

mov x0, #1
mov x1, #0
bl lcd_posiciona_cursor

ldr x0, =buffer_linha_lcd
mov x1, #16
bl lcd_escreve_string

ldr x23, [sp, 48]
ldp x21, x22, [sp, 32]
ldp x19, x20, [sp, 16]
ldp x29, x30, [sp], 64
ret

```

Esta rotina monta e escreve as duas linhas de dezesseis caracteres exibidas no display de cristal líquido, recebendo os quatro valores já convertidos de temperatura, umidade do ar, luminosidade e umidade do solo através dos registradores x0 a x3, copiados logo no início para os registradores salvos x19 a x22, já que múltiplas chamadas às rotinas de buffer serão feitas ao longo da função. A primeira linha é montada inteiramente dentro de `buffer_linha_lcd` antes de qualquer escrita física acontecer, começando pelo rótulo T seguido de dois pontos, através de `lcd_buffer_escreve`, seguido da temperatura formatada com uma casa decimal através de `formata_valor_fixo_buffer`, do sufixo C com um espaço, do rótulo U de umidade, do valor de umidade do ar sem nenhuma casa decimal e do sinal de porcentagem. Depois de montada, essa linha é completada com espaços em branco até as dezesseis posições através de `lcd_buffer_completa_espacos`, o cursor é posicionado na primeira linha através de

lcd_posiciona_cursor e a linha inteira é finalmente escrita no display através de lcd_escreve_string.

A segunda linha segue exatamente o mesmo padrão de montagem, reiniciando a posição do buffer para zero e escrevendo o rótulo L de luminosidade seguido do valor com uma casa decimal, o rótulo S de solo seguido do valor de umidade do solo sem casas decimais e do sinal de porcentagem, sendo então completada com espaços e escrita na segunda linha do display. A escolha de montar cada linha inteira em memória antes de escrevê-la fisicamente, em vez de escrever caractere por caractere diretamente no display, evita que qualquer atraso entre uma escrita I2C e outra produza uma exibição parcial ou piscante na tela, já que o display só recebe cada linha depois que ela está completamente pronta.

Makefile

```
AS    = as
AR    = ar
LD    = ld
OBJDUMP = objdump
GDB    = gdb

TARGET = main

OBJS_TOP = main.o fsm_principal.o

OBJS_ARITMETICA = converte_temp.o converte_luz.o converte_umidade_ar.o
converte_umidade_solo.o \
    coeficientes_calibracao.o calcula_info.o

OBJS_IO    =    gpio.o    estabelece_conexao.o    atuadores.o    usa_lcd.o
apresenta_info_em_tela.o

OBJS_PARSING = recebe_info.o tabelas_amstras.o

OBJS_UTILIDADES = eh_dia.o

LIB_ARITMETICA = libaritmetica.a
LIB_IO = libio.a
LIB_PARSING = libparsing.a
LIB_UTILIDADES = libutilidades.a

LIBS = $(LIB_ARITMETICA) $(LIB_IO) $(LIB_PARSING) $(LIB_UTILIDADES)

all: $(TARGET)

$(TARGET): $(OBJS_TOP) $(LIBS)
    $(LD) -o $(TARGET) $(OBJS_TOP) --start-group $(LIBS) --end-group
```

\$(LIB_ARITMETICA): \$(OBJS_ARITMETICA)

\$(AR) rcs \$(LIB_ARITMETICA) \$(OBJS_ARITMETICA)

\$(LIB_IO): \$(OBJS_IO)

\$(AR) rcs \$(LIB_IO) \$(OBJS_IO)

\$(LIB_PARSING): \$(OBJS_PARSING)

\$(AR) rcs \$(LIB_PARSING) \$(OBJS_PARSING)

\$(LIB_UTILIDADES): \$(OBJS_UTILIDADES)

\$(AR) rcs \$(LIB_UTILIDADES) \$(OBJS_UTILIDADES)

main.o: main.s

\$(AS) -g -o main.o main.s

fsm_principal.o: fsm_principal.s

\$(AS) -g -o fsm_principal.o fsm_principal.s

converte_temp.o: converte_temp.s

\$(AS) -g -o converte_temp.o converte_temp.s

converte_luz.o: converte_luz.s

\$(AS) -g -o converte_luz.o converte_luz.s

converte_umidade_ar.o: converte_umidade_ar.s

\$(AS) -g -o converte_umidade_ar.o converte_umidade_ar.s

converte_umidade_solo.o: converte_umidade_solo.s

\$(AS) -g -o converte_umidade_solo.o converte_umidade_solo.s

coeficientes_calibracao.o: coeficientes_calibracao.s

\$(AS) -g -o coeficientes_calibracao.o coeficientes_calibracao.s

calcula_info.o: calcula_info.s

\$(AS) -g -o calcula_info.o calcula_info.s

gpio.o: gpio.s

\$(AS) -g -o gpio.o gpio.s

estabelece_conexao.o: estabelece_conexao.s

\$(AS) -g -o estabelece_conexao.o estabelece_conexao.s

atuadores.o: atuadores.s

\$(AS) -g -o atuadores.o atuadores.s

usa_lcd.o: usa_lcd.s

\$(AS) -g -o usa_lcd.o usa_lcd.s

apresenta_info_em_tela.o: apresenta_info_em_tela.s

\$(AS) -g -o apresenta_info_em_tela.o apresenta_info_em_tela.s

recebe_info.o: recebe_info.s

\$(AS) -g -o recebe_info.o recebe_info.s

tabelas_amstras.o: tabelas_amstras.s

\$(AS) -g -o tabelas_amstras.o tabelas_amstras.s

eh_dia.o: eh_dia.s

\$(AS) -g -o eh_dia.o eh_dia.s

run: \$(TARGET)

sudo ./\$(TARGET)

debug: \$(TARGET)

sudo \$(GDB) ./\$(TARGET)

```
disasm: $(TARGET)
        $(OBJDUMP) -d $(TARGET)

clean:
        rm -f *.o *.a $(TARGET)
```

Este arquivo organiza a compilação do projeto Assembly em quatro bibliotecas estáticas temáticas, em vez de compilar todos os arquivos soltos diretamente no executável final, reunidas nas variáveis LIB_ARITMETICA, LIB_IO, LIB_PARSING e LIB_UTILIDADES. A biblioteca de aritmética agrupa as rotinas de conversão de sensores e a rotina de cálculo de médias, listadas em OBJS_ARITMETICA, já que todas lidam essencialmente com contas sobre valores numéricos. A biblioteca de entrada e saída agrupa o acesso a GPIO, o estabelecimento da conexão SPI, o controle dos atuadores e o display, listados em OBJS_IO, por serem todas rotinas que conversam diretamente com o hardware externo. A biblioteca de parsing agrupa a recepção e o armazenamento das amostras, em OBJS_PARSING, e a biblioteca de utilidades agrupa, por enquanto, apenas o cálculo de dia e noite, em OBJS_UTILIDADES, deixada separada por não se encaixar claramente em nenhuma das outras três categorias.

Cada biblioteca é construída por uma regra que invoca o utilitário ar com as opções rcs, criando um novo arquivo, inserindo os objetos e ordenando o índice interno da biblioteca em uma única operação. O executável final, definido pela variável TARGET como main, depende apenas dos dois objetos de topo, main.o e fsm_principal.o, listados em OBJS_TOP, e das quatro bibliotecas, sendo ligado através do utilitário ld com as opções start-group e end-group envolvendo as quatro bibliotecas, uma construção necessária porque essas bibliotecas dependem umas das outras de forma cruzada, por exemplo a rotina de conversão de temperatura sendo chamada tanto pela rotina de recepção quanto indiretamente pela de umidade do ar, e o agrupamento entre start-group e end-group instrui o ligador a repetir a busca por símbolos não resolvidos entre as bibliotecas até que todas as dependências cruzadas sejam satisfeitas. Cada arquivo objeto individual é compilado com a flag -g, preservando informação de depuração para uso posterior com o GDB, e os alvos auxiliares run, debug, disasm e clean executam respectivamente o programa com privilégios elevados através de sudo, já que o acesso

direto ao GPIO exige isso, abrem uma sessão de depuração, mostram o código desmontado do executável e removem todos os arquivos gerados pela compilação.

Arquivos Técnicos - Tang Nano 9k e Verilog

top_fpga.v

```
module top_fpga (  
    input  wire clk_pino,  
    input  wire sclk_pino,  
    input  wire cs_pino,  
    input  wire mosi_pino,  
    output wire miso_pino,  
    output wire led_status,  
    output wire led_azul,  
    inout  wire sda_i2c,  
    output wire scl_i2c,  
    output wire sclk_adc_pino,  
    output wire cs_adc_pino,  
    output wire din_adc_pino,  
    input  wire dout_adc_pino  
);  
  
reg [15:0] contador_reset = 16'd0;  
reg      reset_interno  = 1'b1;  
localparam RESET_CICLOS = 16'd60000;  
  
always @(posedge clk_pino) begin  
    if (contador_reset != RESET_CICLOS)  
        contador_reset <= contador_reset + 16'd1;  
    else  
        reset_interno <= 1'b0;  
end  
  
wire [23:0] pressao_bruta;  
wire [23:0] temperatura_bruta;
```

```

wire [15:0] umidade_bruta;
wire [15:0] luminosidade_bruta;
wire      leitura_concluida;

wire sda_saida_i2c;
wire sda_direcao_i2c;

assign sda_i2c = sda_direcao_i2c ? sda_saida_i2c : 1'bz;

fsm_ambiente u_fsm (
    .clk          (clk_pino),
    .reset        (reset_interno),
    .scl          (scl_i2c),
    .sda_saida    (sda_saida_i2c),
    .sda_direcao  (sda_direcao_i2c),
    .sda_entrada  (sda_i2c),
    .pressao_bruta (pressao_bruta),
    .temperatura_bruta (temperatura_bruta),
    .umidade_bruta  (umidade_bruta),
    .luminosidade_bruta (luminosidade_bruta),
    .leitura_concluida (leitura_concluida)
);

wire [23:0] pressao_bruta_final;
wire [23:0] temperatura_bruta_final;
wire [15:0] umidade_bruta_final;
wire      leitura_concluida_ambiente;

i2c_recebe_dados_temp_umiAr_pres u_dados_ambiente (
    .clk          (clk_pino),
    .reset        (reset_interno),
    .pressao_bruta (pressao_bruta),
    .temperatura_bruta (temperatura_bruta),

```

```

        .umidade_bruta      (umidade_bruta),
        .leitura_concluida  (leitura_concluida),
        .pressao_bruta_saida (pressao_bruta_final),
        .temperatura_bruta_saida (temperatura_bruta_final),
        .umidade_bruta_saida  (umidade_bruta_final),
        .leitura_concluida_saida (leitura_concluida_ambiente)
    );

```

```

wire [15:0] luminosidade_bruta_final;
wire      leitura_concluida_luz;

```

```

i2c_recebe_dados_luz u_dados_luz (
    .luminosidade_bruta      (luminosidade_bruta),
    .leitura_concluida        (leitura_concluida),
    .luminosidade_bruta_saida (luminosidade_bruta_final),
    .leitura_concluida_saida  (leitura_concluida_luz)
);

```

```

wire [15:0] umidade_solo_bruta;
wire      leitura_concluida_solo_fsm;

```

```

fsm_umiSolo u_fsm_solo (
    .clk      (clk_pino),
    .reset     (reset_interno),
    .sclk_adc  (sclk_adc_pino),
    .cs_adc    (cs_adc_pino),
    .din_adc   (din_adc_pino),
    .dout_adc  (dout_adc_pino),
    .umidade_solo_bruta (umidade_solo_bruta),
    .leitura_concluida  (leitura_concluida_solo_fsm)
);

```

```

wire [15:0] umidade_solo_bruta_final;

```

```

wire    leitura_concluida_solo;

spi_recebe_dados_umiSolo u_dados_solo (
    .clk            (clk_pino),
    .reset          (reset_interno),
    .umidade_solo_bruta    (umidade_solo_bruta),
    .leitura_concluida    (leitura_concluida_solo_fsm),
    .umidade_solo_bruta_saida (umidade_solo_bruta_final),
    .leitura_concluida_saida (leitura_concluida_solo)
);

reg [95:0] dados_para_spi = 96'hA0A1A2A3A4A5A6A7A8A9AAAB;
always @(posedge clk_pino) begin
    dados_para_spi <= {pressao_bruta_final, temperatura_bruta_final,
                       umidade_bruta_final, luminosidade_bruta_final,
                       umidade_solo_bruta_final};
end

wire comando_valido;

spi_transmite_dados u_spi (
    .sclk      (sclk_pino),
    .cs_n      (cs_pino),
    .mosi      (mosi_pino),
    .miso      (miso_pino),
    .dados_atuais  (dados_para_spi),
    .comando_valido (comando_valido)
);

assign led_status = ~cs_pino;
assign led_azul  = comando_valido;

endmodule

```

Este módulo é o topo da hierarquia de todo o projeto na FPGA, instanciando e conectando entre si todos os demais módulos Verilog do sistema, sem conter, ele mesmo, nenhuma lógica de leitura de sensor específica. Logo no início, um contador de dezesseis bits, `contador_reset`, é incrementado a cada ciclo de clock até atingir sessenta mil, definido pela constante `RESET_CICLOS`, mantendo o sinal `reset_interno` em nível alto durante esse período, o equivalente a pouco mais de dois milissegundos na frequência de vinte e sete megahertz da placa, e esse reset interno, gerado localmente em vez de depender de um pino de reset externo, garante que todos os módulos internos comecem de um estado conhecido assim que a FPGA termina de configurar seu bitstream.

O módulo instancia `fsm_ambiente`, responsável pela leitura combinada do BME280 e do BH1750 pelo mesmo barramento I2C, conectando seus sinais `sda_saida`, `sda_direcao` e `sda_entrada` à linha física `sda_i2c` através de uma atribuição condicional que implementa o comportamento de dreno aberto exigido pelo protocolo, deixando a linha em alta impedância sempre que nenhum lado estiver ativamente puxando o sinal para baixo. Os quatro valores brutos entregues por essa leitura, `pressao_bruta`, `temperatura_bruta`, `umidade_bruta` e `luminosidade_bruta`, são então encaminhados a dois módulos de pré-processamento distintos, `i2c_recebe_dados_temp_umiAr_pres`, que aplica o buffer circular de suavização aos três primeiros, e `i2c_recebe_dados_luz`, que apenas repassa a luminosidade adiante sem nenhum tratamento adicional. Em paralelo, o módulo instancia `fsm_umiSolo` para a leitura do sensor de solo via MCP3008, encaminhando seu resultado bruto a `spi_recebe_dados_umiSolo`, que aplica o filtro de média móvel exponencial descrito nas conclusões técnicas do projeto.

Os cinco valores já tratados, três provenientes do caminho ambiental e um do caminho de solo, mais a luminosidade repassada sem alteração, são agregados em um único registrador de noventa e seis bits, `dados_para_spi`, atualizado a cada ciclo de clock por uma concatenação direta desses campos, e inicializado com um valor de teste reconhecível, `0xA0A1A2A3A4A5A6A7A8A9AAAB`, que só é sobrescrito depois que a primeira leitura completa termina. Por fim, o módulo `spi_transmite_dados` é instanciado recebendo esse registrador de noventa e seis bits como entrada de dados e expõe diretamente os sinais `sclk_pino`, `cs_pino`, `mosi_pino` e `miso_pino` usados pelo Raspberry Pi, junto com a saída `comando_valido`, que é atribuída diretamente ao pino físico `led_azul`, enquanto o pino `led_status` recebe a negação do próprio sinal `cs_pino`, acendendo fisicamente sempre que uma transação SPI está em andamento.

fsm_ambiente.v

```
module fsm_ambiente (  
    input wire    clk,  
    input wire    reset,  
    output reg     scl,  
    output reg     sda_saida,  
    output reg     sda_direcao,  
    input wire     sda_entrada,  
  
    output reg [23:0] pressao_bruta,  
    output reg [23:0] temperatura_bruta,  
    output reg [15:0] umidade_bruta,  
    output reg [15:0] luminosidade_bruta,  
    output reg      leitura_concluida  
);  
  
localparam [6:0] ENDERECO_BME280 = 7'h76;  
localparam [6:0] ENDERECO_LUZ   = 7'h23;  
  
localparam IDLE      = 4'd0;  
localparam START     = 4'd1;  
localparam WRITE_BYTE = 4'd2;  
localparam CHECK_ACK  = 4'd3;  
localparam RESTART    = 4'd4;  
localparam READ_BYTE  = 4'd5;  
localparam SEND_ACK   = 4'd6;  
localparam SEND_NACK  = 4'd7;  
localparam STOP       = 4'd8;  
localparam DONE       = 4'd9;  
localparam ESPERA     = 4'd10;  
  
reg [3:0] estado;  
reg [1:0] fase;
```

```

reg [2:0] indice_bit;
reg [7:0] byte_atual;

reg [3:0] passo;

reg    configurado;
reg    configurado_luz;
reg [1:0] indice_config;
reg [2:0] indice_leitura;
reg    indice_leitura_luz;

reg [7:0] reg_config;
reg [7:0] valor_config;
always @(*) begin
    case (indice_config)
        2'd0: begin reg_config = 8'hF2; valor_config = 8'h01; end
        2'd1: begin reg_config = 8'hF4; valor_config = 8'h27; end
        default: begin reg_config = 8'hF5; valor_config = 8'hA0; end
    endcase
end

reg [6:0] endereco_atual;
always @(*) begin
    case (passo)
        4'd6, 4'd7, 4'd8: endereco_atual = ENDERECO_LUZ;
        default:         endereco_atual = ENDERECO_BME280;
    endcase
end

reg [15:0] divisor_clock;
localparam DIVISOR_MAX = 16'd270;

localparam ESPERA_MAX = 16'd20000;

```

```

reg [15:0] contador_espera;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        estado          <= IDLE;
        fase             <= 2'd0;
        scl              <= 1'b1;
        sda_saida        <= 1'b1;
        sda_direcao      <= 1'b1;
        divisor_clock     <= 16'd0;
        passo            <= 4'd0;
        indice_config     <= 2'd0;
        indice_leitura    <= 3'd0;
        indice_leitura_luz <= 1'b0;
        configurado       <= 1'b0;
        configurado_luz   <= 1'b0;
        contador_espera   <= 16'd0;
        leitura_concluida <= 1'b0;
        pressao_bruta     <= 24'd0;
        temperatura_bruta <= 24'd0;
        umidade_bruta     <= 16'd0;
        luminosidade_bruta <= 16'd0;
    end else begin
        leitura_concluida <= 1'b0;

        if (divisor_clock != DIVISOR_MAX) begin
            divisor_clock <= divisor_clock + 16'd1;
        end else begin
            divisor_clock <= 16'd0;

            case (estado)

                IDLE: begin

```

```

scl      <= 1'b1;
sda_saida <= 1'b1;
sda_direcao <= 1'b1;
fase     <= 2'd0;
passo    <= configurado ? 4'd3 : 4'd0;
estado   <= START;
end

```

START: begin

```

sda_saida <= 1'b0;
sda_direcao <= 1'b1;
byte_atual <= {endereco_atual, (passo == 4'd8) ? 1'b1 : 1'b0};
indice_bit <= 3'd7;
fase      <= 2'd0;
estado    <= WRITE_BYTE;
end

```

WRITE_BYTE: begin

```

case (fase)
  2'd0: begin
    scl      <= 1'b0;
    sda_direcao <= 1'b1;
    sda_saida <= byte_atual[indice_bit];
    fase     <= 2'd1;
  end
  2'd1: begin
    scl <= 1'b1;
    fase <= 2'd2;
  end
  2'd2: begin
    fase <= 2'd3;
  end
  default: begin

```

```

        scl <= 1'b0;
        if (indice_bit == 3'd0) begin
            fase <= 2'd0;
            estado <= CHECK_ACK;
        end else begin
            indice_bit <= indice_bit - 3'd1;
            fase <= 2'd0;
        end
    end
end
endcase
end

```

```

CHECK_ACK: begin
    case (fase)
        2'd0: begin
            scl <= 1'b0;
            sda_direcao <= 1'b0;
            fase <= 2'd1;
        end
        2'd1: begin
            scl <= 1'b1;
            fase <= 2'd2;
        end
        2'd2: begin
            fase <= 2'd3;
        end
        default: begin
            scl <= 1'b0;
            sda_direcao <= 1'b1;
            fase <= 2'd0;
            case (passo)
                4'd0: begin
                    byte_atual <= reg_config;

```

```

        indice_bit <= 3'd7;
        passo    <= 4'd1;
        estado   <= WRITE_BYTE;
    end
    4'd1: begin
        byte_atual <= valor_config;
        indice_bit <= 3'd7;
        passo    <= 4'd2;
        estado   <= WRITE_BYTE;
    end
    4'd2: begin
        estado <= STOP;
    end
    4'd3: begin
        byte_atual <= 8'hF7;
        indice_bit <= 3'd7;
        passo    <= 4'd4;
        estado   <= WRITE_BYTE;
    end
    4'd4: begin
        passo <= 4'd5;
        estado <= RESTART;
    end
    4'd5: begin
        indice_leitura <= 3'd0;
        indice_bit    <= 3'd7;
        estado        <= READ_BYTE;
    end
    4'd6: begin
        byte_atual <= 8'h10;
        indice_bit <= 3'd7;
        passo    <= 4'd7;
        estado   <= WRITE_BYTE;
    end

```

```

        end
        4'd7: begin
            estado <= STOP;
        end
        4'd8: begin
            indice_leitura_luz <= 1'b0;
            indice_bit      <= 3'd7;
            estado          <= READ_BYTE;
        end
        default: estado <= IDLE;
    endcase
end
endcase
end

RESTART: begin
    case (fase)
        2'd0: begin
            scl <= 1'b0;
            fase <= 2'd1;
        end
        2'd1: begin
            sda_direcao <= 1'b1;
            sda_saida  <= 1'b1;
            fase        <= 2'd2;
        end
        2'd2: begin
            scl <= 1'b1;
            fase <= 2'd3;
        end
        default: begin
            sda_saida <= 1'b0;
            byte_atual <= {endereco_atual, 1'b1};
        end
    end
end

```

```

        indice_bit <= 3'd7;
        fase      <= 2'd0;
        estado    <= WRITE_BYTE;
    end
endcase
end

READ_BYTE: begin
    case (fase)
        2'd0: begin
            scl      <= 1'b0;
            sda_direcao <= 1'b0;
            fase      <= 2'd1;
        end
        2'd1: begin
            scl <= 1'b1;
            fase <= 2'd2;
        end
        2'd2: begin
            byte_atual <= {byte_atual[6:0], sda_entrada};
            fase      <= 2'd3;
        end
        default: begin
            scl <= 1'b0;
            if (indice_bit == 3'd0) begin
                if (passo == 4'd8) begin
                    case (indice_leitura_luz)
                        1'b0: luminosidade_bruta[15:8] <= byte_atual;
                        default: luminosidade_bruta[7:0] <= byte_atual;
                    endcase
                end
                fase <= 2'd0;
                estado <= (indice_leitura_luz == 1'b1) ? SEND_NACK :
SEND_ACK;
            end
        end
    end
end

```

```

end else begin
    case (indice_leitura)
        3'd0: pressao_bruta[23:16]  <= byte_atual;
        3'd1: pressao_bruta[15:8]   <= byte_atual;
        3'd2: pressao_bruta[7:0]    <= byte_atual;
        3'd3: temperatura_bruta[23:16] <= byte_atual;
        3'd4: temperatura_bruta[15:8] <= byte_atual;
        3'd5: temperatura_bruta[7:0]  <= byte_atual;
        3'd6: umidade_bruta[15:8]    <= byte_atual;
        default: umidade_bruta[7:0]  <= byte_atual;
    endcase
    fase <= 2'd0;
    estado <= (indice_leitura == 3'd7) ? SEND_NACK :
SEND_ACK;

    end
end else begin
    indice_bit <= indice_bit - 3'd1;
    fase <= 2'd0;
end
end
endcase
end

SEND_ACK: begin
    case (fase)
        2'd0: begin
            scl <= 1'b0;
            sda_direcao <= 1'b1;
            sda_saida <= 1'b0;
            fase <= 2'd1;
        end
        2'd1: begin scl <= 1'b1; fase <= 2'd2; end
        2'd2: begin fase <= 2'd3; end
    endcase
end

```

```

default: begin
    scl <= 1'b0;
    if (passo == 4'd8)
        indice_leitura_luz <= indice_leitura_luz + 1'b1;
    else
        indice_leitura <= indice_leitura + 3'd1;
    indice_bit <= 3'd7;
    fase    <= 2'd0;
    estado  <= READ_BYTE;
end
endcase
end

```

```

SEND_NACK: begin
    case (fase)
        2'd0: begin
            scl    <= 1'b0;
            sda_direcao <= 1'b1;
            sda_saida <= 1'b1;
            fase    <= 2'd1;
        end
        2'd1: begin scl <= 1'b1; fase <= 2'd2; end
        2'd2: begin fase <= 2'd3; end
        default: begin
            scl  <= 1'b0;
            fase <= 2'd0;
            estado <= STOP;
        end
    endcase
end

```

```

STOP: begin
    case (fase)

```

```

2'd0: begin
    scl <= 1'b0;
    fase <= 2'd1;
end
2'd1: begin
    sda_direcao <= 1'b1;
    sda_saida <= 1'b0;
    fase <= 2'd2;
end
2'd2: begin
    scl <= 1'b1;
    fase <= 2'd3;
end
default: begin
    sda_saida <= 1'b1;
    fase <= 2'd0;
    case (passo)
        4'd2: begin
            if (indice_config == 2'd2) begin
                configurado <= 1'b1;
                passo <= 4'd3;
            end else begin
                indice_config <= indice_config + 2'd1;
                passo <= 4'd0;
            end
            estado <= START;
        end
        4'd5: begin
            passo <= configurado_luz ? 4'd8 : 4'd6;
            estado <= START;
        end
        4'd7: begin
            estado <= ESPERA;
    endcase
end

```

```

        end
        default: begin
            estado <= DONE;
        end
    endcase
end
endcase
end

ESPERA: begin
    if (contador_espera == ESPERA_MAX) begin
        contador_espera <= 16'd0;
        configurado_luz <= 1'b1;
        passo          <= 4'd8;
        estado          <= START;
    end else begin
        contador_espera <= contador_espera + 16'd1;
    end
end

DONE: begin
    leitura_concluida <= 1'b1;
    estado             <= IDLE;
end

default: estado <= IDLE;
endcase
end
end
end

endmodule

```

Este módulo implementa a máquina de estados que conduz a leitura combinada do BME280 e do BH1750, dois dispositivos diferentes compartilhando o mesmo barramento I2C, o que exige uma máquina consideravelmente mais elaborada do que a usada para o sensor de solo. Um divisor de clock, controlado pela constante `DIVISOR_MAX` igual a duzentos e setenta, faz com que cada transição de estado ocorra a cada duzentos e setenta ciclos do clock de vinte e sete megahertz da placa, reduzindo a velocidade efetiva das transições da máquina de estados para uma faixa compatível com o protocolo I2C padrão, que opera tipicamente entre cem e quatrocentos quilohertz.

Logo depois do reset, se o sensor ainda não estiver configurado, sinalizado pela variável configurado em zero, a máquina entra em uma sequência de três transações independentes, uma para cada um dos registradores 0xF2, 0xF4 e 0xF5 do BME280, escrevendo respectivamente 0x01, 0x27 e 0xA0, valores esses selecionados através do combinacional que observa `indice_config` e monta `reg_config` e `valor_config` de acordo. Cada uma dessas três transações percorre os estados `START`, `WRITE_BYTE` e `CHECK_ACK` três vezes seguidas, uma para o byte de endereço do dispositivo, uma para o byte que aponta para o registrador de destino e uma para o byte de valor, encerrando com o estado `STOP` antes de a próxima transação começar, e somente depois que as três transações de configuração terminam a variável configurado é definida como um, fazendo com que os ciclos de leitura seguintes pulem essa etapa inteira e vão direto para a leitura de dados.

A leitura em rajada dos oito bytes ambientais, pressão em três bytes, temperatura em três bytes e umidade em dois bytes, começa escrevendo o byte 0xF7 no BME280, o endereço inicial desses registradores de dados, e então usa o estado `RESTART` para gerar uma condição de início repetido no barramento, sem soltar a linha entre a escrita do ponteiro e o início da leitura, uma exigência do próprio protocolo I2C para esse tipo de operação combinada. Os oito bytes são então lidos um a um através do estado `READ_BYTE`, cada um seguido de um bit de confirmação enviado pela FPGA através do estado `SEND_ACK`, exceto o último byte, que é seguido por `SEND_NACK`, sinalizando ao sensor que a leitura em rajada terminou. Cada byte lido é encaminhado ao campo correto, `pressao_bruta`, `temperatura_bruta` ou `umidade_bruta`, de acordo com o valor do contador `indice_leitura` naquele momento.

Depois de concluída essa leitura ambiental, a máquina decide, através da variável configurado_luz, se ainda precisa configurar o BH1750, e, caso ainda não tenha configurado, escreve o comando 0x10 nesse sensor, correspondente ao modo de medição

continua de alta resolução, e então entra no estado ESPERA, contando até vinte mil ciclos antes de prosseguir, reproduzindo o tempo de aquecimento exigido pelo datasheet do BH1750 antes de sua primeira leitura ser confiável, um intervalo que sozinho representa a maior parte do tempo total gasto no primeiro ciclo completo de leitura do sistema. Uma vez configurado, o BH1750 passa a ser lido diretamente a cada ciclo seguinte, sem precisar repetir esse comando nem essa espera, através de dois bytes lidos também pelos estados READ_BYTE, SEND_ACK e SEND_NACK, formando o campo luminosidade_bruta. Ao final de toda essa sequência, o estado DONE ativa o sinal leitura_concluida por um único ciclo de clock e retorna ao estado IDLE, reiniciando o processo para a próxima leitura.

i2c_recebe_dados_temp_umiAr_pres.v

```
module i2c_recebe_dados_temp_umiAr_pres (
    input wire    clk,
    input wire    reset,
    input wire [23:0] pressao_bruta,
    input wire [23:0] temperatura_bruta,
    input wire [15:0] umidade_bruta,
    input wire    leitura_concluida,

    output reg [23:0] pressao_bruta_saida,
    output reg [23:0] temperatura_bruta_saida,
    output reg [15:0] umidade_bruta_saida,
    output reg    leitura_concluida_saida
);

reg [63:0] buffer_amstras [0:15];
reg [3:0] ponteiro_escrita = 4'd0;

localparam OCIOSO    = 2'd0;
localparam VARRENDO  = 2'd1;
localparam FINALIZANDO = 2'd2;

reg [1:0] estado = OCIOSO;
reg [3:0] ponteiro_leitura = 4'd0;
reg [31:0] soma_pressao = 32'd0;
reg [31:0] soma_temp    = 32'd0;
reg [31:0] soma_umidade = 32'd0;

wire [63:0] amostra_lida = buffer_amstras[ponteiro_leitura];

always @(posedge clk) begin
    leitura_concluida_saida <= leitura_concluida;

    if (reset) begin
```

```

    ponteiro_escrita    <= 4'd0;
    estado              <= OCIOSO;
    pressao_bruta_saida <= 24'd0;
    temperatura_bruta_saida <= 24'd0;
    umidade_bruta_saida <= 16'd0;
end else begin
    case (estado)
        OCIOSO: begin
            if (leitura_concluida) begin
                buffer_amostras[ponteiro_escrita]    <=      {pressao_bruta,
temperatura_bruta, umidade_bruta};

                ponteiro_escrita <= ponteiro_escrita + 4'd1;
                ponteiro_leitura <= 4'd0;
                soma_pressao    <= 32'd0;
                soma_temp       <= 32'd0;
                soma_umidade    <= 32'd0;
                estado          <= VARRENDO;
            end
        end

        VARRENDO: begin
            soma_pressao    <= soma_pressao + amostra_lida[63:40];
            soma_temp       <= soma_temp    + amostra_lida[39:16];
            soma_umidade    <= soma_umidade + amostra_lida[15:0];
            ponteiro_leitura <= ponteiro_leitura + 4'd1;
            if (ponteiro_leitura == 4'd15)
                estado <= FINALIZANDO;
            end

        FINALIZANDO: begin
            pressao_bruta_saida    <= soma_pressao[27:4];
            temperatura_bruta_saida <= soma_temp[27:4];
            umidade_bruta_saida    <= soma_umidade[19:4];

```

```
        estado          <= OCIOSO;
    end

    default: estado <= OCIOSO;
endcase
end
end
endmodule
```

Este módulo aplica um filtro de suavização às três leituras vindas do BME280, pressão, temperatura e umidade do ar, usando um buffer circular de dezesseis posições implementado sobre o array `buffer_amostras`, cada posição armazenando os três valores concatenados em uma única palavra de sessenta e quatro bits. Uma pequena máquina de estados própria, com apenas três estados, OCIOSO, VARRENDO e FINALIZANDO, controla esse processo, permanecendo ociosa até que o sinal `leitura_concluida` vindo de `fsm_ambiente` seja ativado, momento em que a leitura mais recente é escrita na posição apontada por `ponteiro_escrita`, esse ponteiro é incrementado, e a máquina passa a varrer as dezesseis posições do buffer.

Durante a varredura, cada uma das dezesseis posições é lida através de `ponteiro_leitura`, e os três campos de cada posição são somados separadamente em três acumuladores de trinta e dois bits, `soma_pressao`, `soma_temp` e `soma_umidade`, um incremento por ciclo de clock, até que todas as dezesseis posições tenham sido percorridas. No estado FINALIZANDO, cada uma dessas três somas é dividida por dezesseis através de um deslocamento de quatro bits à direita, produzindo a média das últimas dezesseis leituras de cada parâmetro, que é então escrita nas saídas correspondentes e o sinal `leitura_concluida_saida` é propagado adiante com um ciclo de atraso em relação ao sinal de entrada. É importante registrar que, como o array `buffer_amostras` não é inicializado no reset, apenas o ponteiro de escrita é zerado, as primeiras dezesseis leituras depois de um reset ainda contêm posições com conteúdo indefinido, de modo que a saída deste módulo só reflete de fato a média de dezesseis amostras reais depois que o buffer circular tiver sido completamente preenchido pelo menos uma vez.

i2c_recebe_dados_luz.v

```
module i2c_recebe_dados_luz (  
    input wire [15:0] luminosidade_bruta,  
    input wire      leitura_concluida,  
  
    output wire [15:0] luminosidade_bruta_saida,  
    output wire      leitura_concluida_saida  
);  
  
    assign luminosidade_bruta_saida = luminosidade_bruta;  
    assign leitura_concluida_saida = leitura_concluida;  
  
endmodule
```

Este módulo é o mais simples de todo o projeto, funcionando puramente como um estágio de passagem para o valor de luminosidade vindo de fsm_ambiente. As duas atribuições contínuas que compõem todo o corpo do módulo apenas copiam luminosidade_bruta para luminosidade_bruta_saida e leitura_concluida para leitura_concluida_saida, sem nenhum registrador, sem nenhum clock e sem nenhuma lógica de suavização. Sua existência como um módulo separado, em vez de conectar o sinal diretamente dentro de top_fpga.v, mantém a simetria estrutural entre os três canais de pré-processamento do projeto, pressão e temperatura, umidade do ar e luminosidade, mesmo quando um deles, como é o caso aqui, não precisa de nenhum tratamento real, o que facilita eventualmente adicionar um filtro a esse canal no futuro sem alterar a interface do restante do sistema.

fsm_umiSolo.v

```
module fsm_umiSolo (  
    input wire    clk,  
    input wire    reset,  
    output reg    sclk_adc,  
    output reg    cs_adc,  
    output reg    din_adc,  
    input wire    dout_adc,  
  
    output reg [15:0] umidade_solo_bruta,  
    output reg      leitura_concluida  
);  
  
localparam [2:0] CANAL_SOLO = 3'b000;  
  
localparam IDLE      = 3'd0;  
localparam ESPERA_CS = 3'd1;  
localparam TRANSFERE = 3'd2;  
localparam ESPERA_CSH = 3'd3;  
localparam DONE      = 3'd4;  
  
reg [2:0] estado;  
reg      fase;  
reg [4:0] indice_bit;  
reg [4:0] comando;  
reg [9:0] dado_recebido;  
  
reg [15:0] divisor_clock;  
localparam DIVISOR_MAX = 16'd270;  
  
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        estado      <= IDLE;
```

```

fase          <= 1'b0;
sclk_adc      <= 1'b0;
cs_adc        <= 1'b1;
din_adc       <= 1'b0;
indice_bit    <= 5'd0;
comando       <= 5'd0;
dado_recebido <= 10'd0;
divisor_clock <= 16'd0;
umidade_solo_bruta <= 16'd0;
leitura_concluida <= 1'b0;
end else begin

    leitura_concluida <= 1'b0;

    if (divisor_clock != DIVISOR_MAX) begin
        divisor_clock <= divisor_clock + 16'd1;
    end else begin
        divisor_clock <= 16'd0;

        case (estado)

            IDLE: begin
                cs_adc    <= 1'b1;
                sclk_adc  <= 1'b0;
                comando   <= {2'b11, CANAL_SOLO};
                indice_bit <= 5'd0;
                fase       <= 1'b0;
                estado     <= ESPERA_CS;
            end

            ESPERA_CS: begin
                cs_adc <= 1'b0;
                estado <= TRANSFERE;
            end

```

```

TRANSFERE: begin
    case (fase)
        1'b0: begin
            sclk_adc <= 1'b0;
            din_adc <= (indice_bit < 5'd5) ? comando[4] : 1'b0;
            fase    <= 1'b1;
        end
        default: begin
            sclk_adc <= 1'b1;
            if (indice_bit >= 5'd7)
                dado_recebido <= {dado_recebido[8:0], dout_adc};
            if (indice_bit < 5'd5)
                comando <= {comando[3:0], 1'b0};
            if (indice_bit == 5'd16) begin
                fase <= 1'b0;
                estado <= ESPERA_CSH;
            end else begin
                indice_bit <= indice_bit + 5'd1;
                fase    <= 1'b0;
            end
        end
    endcase
end

```

```

ESPERA_CSH: begin
    sclk_adc    <= 1'b0;
    cs_adc      <= 1'b1;
    umidade_solo_bruta <= {6'd0, dado_recebido};
    estado      <= DONE;
end

```

```

DONE: begin

```

```

        leitura_concluida <= 1'b1;
        estado          <= IDLE;
    end

    default: estado <= IDLE;
endcase
end
end
end

endmodule

```

Este módulo conduz a leitura periódica do sensor de umidade do solo através do conversor analógico digital MCP3008, que se comunica por um protocolo semelhante ao SPI usando os sinais `sclk_adc`, `cs_adc`, `din_adc` e `dout_adc`. A máquina de estados definida aqui é mais enxuta do que a usada para o ambiente, com apenas cinco estados, IDLE, ESPERA_CS, TRANSFERE, ESPERA_CSH e DONE, já que o MCP3008 não exige nenhuma etapa de configuração inicial como a que o BME280 e o BH1750 exigem. No estado IDLE, o comando de leitura para o canal zero do conversor é montado na variável comando, combinando os dois bits de início fixos exigidos pelo protocolo do MCP3008 com os três bits que identificam o canal desejado, guardado na constante CANAL_SOLO.

O estado TRANSFERE conduz os dezessete pulsos de clock que uma leitura completa do MCP3008 exige, alternando a cada ciclo entre preparar o próximo bit a ser enviado, na fase zero, e efetivamente gerar a borda de subida do clock, na fase um. Durante os primeiros cinco pulsos, a linha `din_adc` envia os bits do comando montado anteriormente, e a partir do sétimo pulso até o décimo sexto, o contador `indice_bit` controla a captura de cada bit recebido pela linha `dout_adc`, deslocando-o para dentro do registrador `dado_recebido`, reproduzindo exatamente a janela de captura descrita no datasheet do conversor, na qual os dois primeiros pulsos depois do comando não carregam dado válido e os dez bits do resultado aparecem a partir do sétimo pulso. Ao atingir o décimo sétimo pulso, contado pelo valor dezesseis em `indice_bit` já que a contagem começa em zero, a máquina avança para ESPERA_CSH, que levanta a linha de seleção de chip e copia o valor capturado em `dado_recebido`, estendido com seis zeros à esquerda,

para a saída umidade_solo_bruta. O estado DONE ativa o sinal leitura_concluida por um único ciclo antes de retornar ao estado IDLE, reiniciando imediatamente o processo para a próxima leitura, sem nenhuma pausa proposital entre um ciclo e o seguinte.

spi_recebe_dados_umiSolo.v

```
module spi_recebe_dados_umiSolo (
    input wire    clk,
    input wire    reset,
    input wire [15:0] umidade_solo_bruta,
    input wire    leitura_concluida,

    output reg [15:0] umidade_solo_bruta_saida,
    output reg    leitura_concluida_saida
);
    localparam [15:0] ALPHA = 16'd6554;

    reg [15:0] media_atual = 16'd0;

    wire    umidade_maior = (umidade_solo_bruta > media_atual);
    wire [15:0] diferenca_abs = umidade_maior ? (umidade_solo_bruta - media_atual)
                                              : (media_atual - umidade_solo_bruta);
    wire [31:0] produto_abs  = diferenca_abs * ALPHA;
    wire [15:0] incremento_abs = produto_abs[31:16];

    always @(posedge clk) begin
        leitura_concluida_saida <= leitura_concluida;

        if (reset) begin
            media_atual <= 16'd0;
        end else if (leitura_concluida) begin
            if (umidade_maior)
                media_atual <= media_atual + incremento_abs;
            else
                media_atual <= media_atual - incremento_abs;
        end

        umidade_solo_bruta_saida <= media_atual;
    end
endmodule
```

```
end  
endmodule
```

Este módulo aplica um filtro de suavização à leitura de umidade do solo, mas usa uma abordagem diferente da empregada para os dados ambientais, optando por uma média móvel exponencial em vez de um buffer circular de amostras, já que o volume de dados de um único canal não justificaria o custo de memória de um buffer dedicado. A constante ALPHA, fixada em 6554, representa o peso dado a cada nova leitura na atualização da média, correspondendo a aproximadamente dez por cento, já que 6554 dividido por 65536 é próximo de 0,1.

A cada novo pulso de leitura concluída, o módulo primeiro determina se o novo valor lido, `umidade_solo_bruta`, é maior ou menor que a média atual, `media_atual`, calculando a diferença absoluta entre os dois em `diferenca_abs`, e então multiplica essa diferença pela constante ALPHA, extraindo os dezesseis bits mais significativos do produto de trinta e dois bits como o incremento a ser aplicado. Esse incremento é somado à média atual quando o novo valor é maior, e subtraído quando é menor, fazendo com que a média se mova sempre um décimo do caminho em direção ao valor mais recente, e nunca o valor bruto completo, o que suaviza oscilações repentinas na leitura do sensor às custas de introduzir um pequeno atraso e um resíduo de arredondamento permanente entre o valor filtrado e o valor de entrada, já discutido na análise da testbench correspondente. O valor de saída, `umidade_solo_bruta_saida`, é atualizado a cada ciclo de clock com o valor de `media_atual` anterior à atualização daquele mesmo ciclo, e o sinal de conclusão é repassado adiante com um ciclo de atraso em relação à entrada.

spi_transmite_dados.v

```
module spi_transmite_dados (
    input wire    sclk,
    input wire    cs_n,
    input wire    mosi,
    output wire    miso,
    input wire [95:0] dados_atuais,
    output reg     comando_valido = 1'b0
);

localparam [7:0] COMANDO_ESPERADO = 8'hA5;
localparam [7:0] STATUS_OK        = 8'h5A;
localparam [7:0] STATUS_ERRO      = 8'h00;

reg [7:0] comando_recebido = 8'd0;
reg [2:0] contagem_comando = 3'd0;
reg      comando_capturado = 1'b0;
reg [111:0] registrador_saida = 112'd0;

wire [7:0] checksum = dados_atuais[95:88] + dados_atuais[87:80] +
dados_atuais[79:72] +
                        dados_atuais[71:64] + dados_atuais[63:56] + dados_atuais[55:48] +
                        dados_atuais[47:40] + dados_atuais[39:32] + dados_atuais[31:24] +
                        dados_atuais[23:16] + dados_atuais[15:8] + dados_atuais[7:0];

always @(posedge sclk or posedge cs_n) begin
    if (cs_n) begin
        comando_capturado <= 1'b0;
        contagem_comando <= 3'd0;
        comando_valido <= 1'b0;
    end else if (!comando_capturado) begin
        comando_recebido <= {comando_recebido[6:0], mosi};
        if (contagem_comando == 3'd7) begin
            comando_capturado <= 1'b1;
        end
    end
end
```

```

        comando_valido      <=  ({comando_recebido[6:0],  mosi}  ==
COMANDO_ESPERADO);

        end else begin

            contagem_comando <= contagem_comando + 3'd1;

        end

    end

end

always @(negedge sclk or posedge cs_n) begin
    if (cs_n) begin
        registrador_saida <= { (comando_recebido == COMANDO_ESPERADO) ?
STATUS_OK : STATUS_ERRO,
                                dados_atuais,
                                checksum };

    end else begin

        registrador_saida <= registrador_saida << 1;

    end

end

assign miso = registrador_saida[111];

endmodule

```

Este módulo implementa o lado escravo da interface SPI voltada ao Raspberry Pi, incluindo um protocolo de handshaking que exige a validação de um comando antes de qualquer troca de dados ser considerada confiável. Um primeiro bloco sequencial, sensível à borda de subida do clock, captura os oito bits do byte de comando enviados pelo mestre através da linha mosi, acumulando-os em `comando_recebido`, e confere, assim que o oitavo bit chega, contado por `contagem_comando` atingindo sete, se o valor recebido corresponde ao comando esperado, hexadecimal A cinco, atualizando a saída `comando_valido` conforme esse resultado e mantendo esse valor congelado, através da variável `comando_capturado`, pelo restante daquela transação.

Um segundo bloco sequencial, sensível à borda de descida do clock, é responsável por montar e deslocar o quadro de resposta. Sempre que a linha de seleção de chip sobe, marcando o fim de uma transação e o início do intervalo até a próxima, esse bloco monta um novo quadro de cento e doze bits no registrador `registrador_saida`, concatenando um byte de status, que assume o valor hexadecimal cinco A quando o comando foi reconhecido ou zero zero caso contrário, os noventa e seis bits de dados vindos de `dados_atuais`, e um byte de checksum, calculado por uma atribuição contínua separada que soma os doze bytes de dados. Enquanto a transação está em andamento, esse mesmo registrador é deslocado um bit para a esquerda a cada borda de descida, expondo constantemente o bit mais significativo na linha `miso`, de modo que o Raspberry Pi consegue puxar o quadro completo de catorze bytes através do protocolo de bit banging implementado em `recebe_info.s`, byte a byte, exatamente na velocidade que o próprio mestre define ao gerar os pulsos de clock.

Testbenches

spi_transmite_dados_tb.v

```
`timescale 1ns/1ps

module spi_transmite_dados_tb;

    localparam [7:0] COMANDO_VALIDO = 8'hA5;
    localparam [7:0] COMANDO_INVALIDO = 8'h00;
    localparam [7:0] STATUS_OK = 8'h5A;
    localparam [7:0] STATUS_ERRO = 8'h00;

    reg    sclk = 1'b0;
    reg    cs_n = 1'b1;
    reg    mosi = 1'b0;
    wire    miso;
    reg [95:0] dados_atuais;
    wire    comando_valido;

    reg [7:0] rx_frame [0:13];
    integer  erros;
    integer  i, b;
    reg [7:0] soma_recalculada;

    spi_transmite_dados dut (
        .sclk      (sclk),
        .cs_n      (cs_n),
        .mosi      (mosi),
        .miso      (miso),
        .dados_atuais (dados_atuais),
        .comando_valido (comando_valido)
    );
```

```

localparam MEIO_PERIODO = 50;

task le_frame_como_rpi;
    input [7:0] comando;
    integer i, b;
    begin
        cs_n = 1'b0;
        #(MEIO_PERIODO);
        for (i = 0; i < 14; i = i + 1) begin
            rx_frame[i] = 8'd0;
            for (b = 0; b < 8; b = b + 1) begin
                if (i == 0)
                    mosi = comando[7 - b];
                else
                    mosi = 1'b0;
                sclk = 1'b1;
                #(MEIO_PERIODO);
                rx_frame[i] = {rx_frame[i][6:0], miso};
                sclk = 1'b0;
                #(MEIO_PERIODO);
            end
        end
        cs_n = 1'b1;
        #(MEIO_PERIODO);
    end
endtask

task confere_byte;
    input [3:0] indice;
    input [7:0] esperado;
    begin
        if (rx_frame[indice] !== esperado) begin

```

```

        $display("FALHA: byte %0d esperado=0x%02h obtido=0x%02h",
                indice, esperado, rx_frame[indice]);
        erros = erros + 1;
    end else begin
        $display("OK:  byte %0d = 0x%02h", indice, rx_frame[indice]);
    end
end
endtask

task confere_status;
    input [7:0] esperado;
    begin
        if (rx_frame[0] !== esperado) begin
            $display("FALHA: status esperado=0x%02h obtido=0x%02h", esperado,
rx_frame[0]);
            erros = erros + 1;
        end else begin
            $display("OK:  status = 0x%02h", rx_frame[0]);
        end
    end
endtask

task confere_checksum;
    integer k;
    reg [7:0] soma;
    begin
        soma = 8'd0;
        for (k = 1; k <= 12; k = k + 1)
            soma = soma + rx_frame[k];
        if (soma !== rx_frame[13]) begin
            $display("FALHA: checksum esperado=0x%02h obtido=0x%02h", soma,
rx_frame[13]);
            erros = erros + 1;
        end
    end
endtask

```

```

        end else begin
            $display("OK:  checksum = 0x%02h", rx_frame[13]);
        end
    end
endtask

task confere_led;
    input [8*32-1:0] nome;
    input esperado;
    begin
        if (comando_valido !== esperado) begin
            $display("FALHA [%s]: led_azul esperado=%b obtido=%b", nome,
esperado, comando_valido);
            erros = erros + 1;
        end else begin
            $display("OK [%s]: led_azul=%b", nome, comando_valido);
        end
    end
endtask

reg [23:0] pressao_teste;
reg [23:0] temperatura_teste;
reg [15:0] umidade_teste;
reg [15:0] luz_teste;
reg [15:0] solo_teste;

initial begin
    $dumpfile("spi_transmite_dados_tb.vcd");
    $dumpvars(0, spi_transmite_dados_tb);

    erros = 0;
    dados_atuais = 96'd0;

```

```

confere_led("antes de qualquer transacao", 1'b0);

le_frame_como_rpi(COMANDO_VALIDO);

pressao_teste    = 24'h654321;
temperatura_teste = 24'hABCDEF;
umidade_teste    = 16'h1234;
luz_teste        = 16'h05DC;
solo_teste       = 16'h0288;

dados_atuais = {pressao_teste, temperatura_teste,
                umidade_teste, luz_teste, solo_teste};

le_frame_como_rpi(COMANDO_VALIDO);
le_frame_como_rpi(COMANDO_VALIDO);

$display("--- Caso 1: comando valido, frame conhecido ---");
confere_status(STATUS_OK);
confere_byte(1, pressao_teste[23:16]);
confere_byte(2, pressao_teste[15:8]);
confere_byte(3, pressao_teste[7:0]);
confere_byte(4, temperatura_teste[23:16]);
confere_byte(5, temperatura_teste[15:8]);
confere_byte(6, temperatura_teste[7:0]);
confere_byte(7, umidade_teste[15:8]);
confere_byte(8, umidade_teste[7:0]);
confere_byte(9, luz_teste[15:8]);
confere_byte(10, luz_teste[7:0]);
confere_byte(11, solo_teste[15:8]);
confere_byte(12, solo_teste[7:0]);
confere_checksum;

le_frame_como_rpi(COMANDO_INVALIDO);

```

```

le_frame_como_rpi(COMANDO_INVALIDO);
$display("--- Caso 2: comando invalido (0x00 em vez de 0xA5) ---");
confere_status(STATUS_ERRO);

dados_atuais = 96'd0;
le_frame_como_rpi(COMANDO_VALIDO);
le_frame_como_rpi(COMANDO_VALIDO);
$display("--- Caso 3a: frame todo zero ---");
confere_status(STATUS_OK);
confere_byte(1, 8'h00);
confere_byte(6, 8'h00);
confere_byte(12, 8'h00);
confere_checksum;

dados_atuais = {96{1'b1}};
le_frame_como_rpi(COMANDO_VALIDO);
le_frame_como_rpi(COMANDO_VALIDO);
$display("--- Caso 3b: frame todo um ---");
confere_status(STATUS_OK);
confere_byte(1, 8'hFF);
confere_byte(6, 8'hFF);
confere_byte(12, 8'hFF);
confere_checksum;

$display("--- Caso 4: LED azul durante e depois de uma transacao valida ---");
cs_n = 1'b0;
#(MEIO_PERIODO);
for (i = 7; i >= 0; i = i - 1) begin
    mosi = COMANDO_VALIDO[i];
    sclk = 1'b1;
    #(MEIO_PERIODO);
    sclk = 1'b0;
    #(MEIO_PERIODO);
end

```

```

end
confere_led("logo apos os 8 bits do comando", 1'b1);

for (i = 0; i < 104; i = i + 1) begin
    mosi = 1'b0;
    sclk = 1'b1;
    #(MEIO_PERIODO);
    sclk = 1'b0;
    #(MEIO_PERIODO);
end
cs_n = 1'b1;
#(MEIO_PERIODO);
confere_led("logo depois que o CS subiu", 1'b0);

#500;
confere_led("no intervalo entre uma leitura e outra", 1'b0);

dados_atuais = {pressao_teste, temperatura_teste,
                 umidade_teste, luz_teste, solo_teste};
le_frame_como_rpi(8'hA4);
le_frame_como_rpi(8'hA4);
$display("--- Caso 5: comando quase certo (0xA4 em vez de 0xA5) ---");
confere_status(STATUS_ERRO);

le_frame_como_rpi(COMANDO_VALIDO);
le_frame_como_rpi(COMANDO_VALIDO);
$display("--- Caso 6: bit corrompido em transito -> checksum nao deve bater ---");
rx_frame[6] = rx_frame[6] ^ 8'h01;
soma_recalculada = 8'd0;
for (i = 1; i <= 12; i = i + 1)
    soma_recalculada = soma_recalculada + rx_frame[i];
if (soma_recalculada == rx_frame[13]) begin

```

```

        $display("FALHA: checksum deveria acusar corrupcao e nao acusou");
        erros = erros + 1;
    end else begin
        $display("OK:      checksum recalculado=0x%02h != recebido=0x%02h ->
corrupcao detectada",
                soma_recalculada, rx_frame[13]);
    end

    if (erros == 0)
        $display("\n=== TODOS OS TESTES PASSARAM (spi_transmite_dados)
===");
    else
        $display("\n=== %0d FALHA(S) (spi_transmite_dados) ===", erros);

    $finish;
end

endmodule

```

Resultado

```

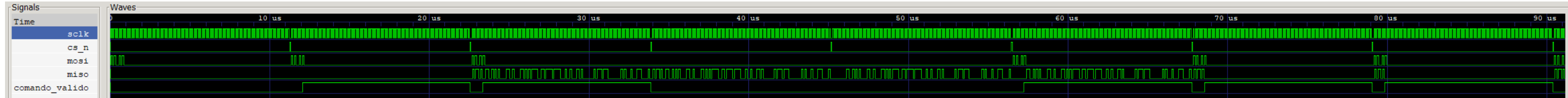
(.venv) C:\Users\gribe\OneDrive\Documentos\Codes\INFNET\2026.1\Projeto de
Bloco\Projeto-de-Bloco-Sistemas-Digitais-Embarcados-26E2_5-AT\FPGA>vvp
sim_spi
VCD info: dumpfile spi_transmite_dados_tb.vcd opened for output.
OK [ antes de qualquer transacao]: led_azul=0
--- Caso 1: comando valido, frame conhecido ---
OK: status = 0x5a
OK: byte 1 = 0x65
OK: byte 2 = 0x43
OK: byte 3 = 0x21
OK: byte 4 = 0xab
OK: byte 5 = 0xcd

```

```
OK:  byte 6 = 0xef
OK:  byte 7 = 0x12
OK:  byte 8 = 0x34
OK:  byte 9 = 0x05
OK:  byte 10 = 0xdc
OK:  byte 11 = 0x02
OK:  byte 12 = 0x88
OK:  checksum = 0xe1
--- Caso 2: comando invalido (0x00 em vez de 0xA5) ---
OK:  status = 0x00
--- Caso 3a: frame todo zero ---
OK:  status = 0x5a
OK:  byte 1 = 0x00
OK:  byte 6 = 0x00
OK:  byte 12 = 0x00
OK:  checksum = 0x00
--- Caso 3b: frame todo um ---
OK:  status = 0x5a
OK:  byte 1 = 0xff
OK:  byte 6 = 0xff
OK:  byte 12 = 0xff
OK:  checksum = 0xf4
--- Caso 4: LED azul durante e depois de uma transacao valida ---
OK  [ logo apos os 8 bits do comando]: led_azul=1
OK  [ logo depois que o CS subiu]: led_azul=0
OK  [ ervalo entre uma leitura e outra]: led_azul=0
--- Caso 5: comando quase certo (0xA4 em vez de 0xA5) ---
OK:  status = 0x00
--- Caso 6: bit corrompido em transito -> checksum nao deve bater ---
OK:  checksum recalculado=0xe0 != recebido=0xe1 -> corrupcao detectada

=== TODOS OS TESTES PASSARAM (spi_transmite_dados) ===
spi_transmite_dados_tb.v:236: $finish called at 158700000 (1ps)
```

GTK Wave



Análise

O terminal confirma que os seis casos de teste passaram sem nenhuma falha, o que já pode ser conferido logo na primeira linha, já que antes de qualquer transação o sinal `comando_valido` aparece em zero, exatamente como esperado de um módulo que ainda não recebeu comando algum. No primeiro caso, os doze bytes de dados chegam na ordem correta, sendo eles 0x65, 0x43 e 0x21 para a pressão, 0xab, 0xcd e 0xef para a temperatura, 0x12 e 0x34 para a umidade do ar, 0x05 e 0xdc para a luminosidade, e 0x02 e 0x88 para a umidade do solo, e o byte de status vem como 0x5a, confirmando que o comando 0xa5 foi reconhecido. O checksum recebido, 0xe1, também está correto, já que somando esses doze bytes byte a byte e descartando o estouro além de oito bits chega se exatamente a esse mesmo valor, o que mostra que o módulo está calculando essa soma corretamente sobre o barramento de noventa e seis bits antes de anexá-la ao quadro de resposta.

O segundo caso confirma que, ao enviar o comando 0x00 no lugar de 0xa5, o status retornado passa a ser 0x00, mostrando que o módulo realmente distingue um comando reconhecido de um não reconhecido e reage imediatamente a essa diferença. Os casos três a e três b repetem a mesma verificação de status e checksum, dessa vez com o quadro de dados inteiro em zero e inteiro em um, e os dois checksums batem novamente, sendo 0x00 para o quadro zerado, já que a soma de doze bytes nulos continua nula, e 0xf4 para o quadro com todos os bits em um, valor que corresponde exatamente ao resto da divisão de mil e duzentos e sessenta bits acumulados, considerando doze bytes de valor 0xff cada, por duzentos e cinquenta e seis. O quarto caso acompanha o sinal `comando_valido` em três momentos diferentes de uma transação, mostrando que ele sobe para um logo depois que os oito bits do comando terminam de ser recebidos e volta para zero assim que a linha de seleção de chip sobe outra vez, permanecendo em zero também no intervalo entre uma leitura e a seguinte, o que evidencia que esse sinal reflete exatamente o estado da última transação e não fica preso em nenhum valor incorreto entre uma transação e outra.

Os dois últimos casos validam a capacidade do protocolo de detectar falhas de comunicação mais sutis. No quinto, o comando 0xa4, que difere do valor correto por um único bit, ainda assim resulta em status 0x00, confirmando que o módulo não aceita nenhuma aproximação do comando esperado e exige uma correspondência exata. No sexto, depois de uma transação válida com o mesmo quadro do primeiro caso, um único

bit do sexto byte já recebido é invertido artificialmente dentro da própria testbench, e o checksum recalculado sobre esse quadro corrompido resulta em 0xe0, diferente do checksum de 0xe1 que havia sido de fato recebido, o que confirma que a verificação de checksum é sensível o suficiente para detectar a corrupção de um único bit em um único byte entre os doze transmitidos.

A forma de onda capturada mostra os cinco sinais selecionados ao longo dos primeiros noventa e cinco microssegundos da simulação, cobrindo a maior parte, mas não a totalidade, das seis transações exigidas pelos seis casos de teste, já que a simulação inteira só termina em cento e cinquenta e oito vírgula sete microssegundos. O sinal `selk` aparece como uma sucessão praticamente contínua de pulsos ao longo de quase toda a janela, o que é esperado, já que ele alterna de nível a cada cinquenta nanossegundos sempre que uma transação está em andamento, tornando cada período individual pequeno demais para ser distinguido visualmente nessa escala de tempo. O sinal `cs_n`, por sua vez, aparece como uma linha majoritariamente alta com uma sequência de quedas breves e regularmente espaçadas, cada uma delimitando o início e o fim de uma transação completa de catorze bytes, e o espaçamento entre essas quedas cresce um pouco ao longo do tempo, refletindo o fato de que alguns casos de teste, como o quarto, envolvem controle manual dos sinais fora da tarefa padrão de leitura, alongando ligeiramente o intervalo entre uma transação e a seguinte.

O contraste entre mosi e miso é o aspecto mais evidente dessa captura, já que mosi mostra apenas um pequeno grupo de oito pulsos concentrados logo no início de cada janela em que `cs_n` está baixo, correspondendo exatamente aos oito bits do byte de comando, enquanto miso mostra um trem bem mais longo e denso de pulsos preenchendo praticamente todo o restante de cada transação, já que ele carrega os cento e quatro bits restantes do quadro de resposta, formados pelo status, pelos doze bytes de dados e pelo checksum. Essa diferença de densidade entre as duas linhas ilustra de forma direta a natureza assimétrica do protocolo, no qual o mestre envia pouquíssima informação para receber uma resposta bem mais longa a cada ciclo. Por fim, o sinal `comando_valido` aparece como uma linha que permanece baixa durante um bom trecho inicial da simulação e depois passa a alternar entre os níveis alto e baixo em sincronia com as quedas de `cs_n`, subindo nas transações em que o comando enviado corresponde a 0xa5 e permanecendo baixo nas transações em que um comando diferente, como 0x00 ou 0xa4, foi utilizado, o que é exatamente o comportamento que os casos dois, quatro e cinco se propõem a verificar.

fsm_umiSolo_tb.v

```
`timescale 1ns/1ps

module fsm_umiSolo_tb;

    real CLK_PERIODO = 37.037;

    reg clk;
    reg reset;
    wire sclk_adc;
    wire cs_adc;
    wire din_adc;
    reg dout_adc;

    wire [15:0] umidade_solo_bruta;
    wire      leitura_concluida;

    fsm_umiSolo dut (
        .clk          (clk),
        .reset        (reset),
        .sclk_adc      (sclk_adc),
        .cs_adc        (cs_adc),
        .din_adc       (din_adc),
        .dout_adc      (dout_adc),
        .umidade_solo_bruta (umidade_solo_bruta),
        .leitura_concluida (leitura_concluida)
    );

    always #(CLK_PERIODO/2) clk = ~clk;

    reg [4:0] neg_count;
    reg [9:0] valor_adc_atual;
```

```

always @(negedge cs_adc) begin
    neg_count <= 5'd0;
    dout_adc <= 1'b0;
end

always @(negedge sclk_adc) begin
    if (!cs_adc) begin
        neg_count <= neg_count + 5'd1;
        if ((neg_count + 5'd1) >= 5'd7 && (neg_count + 5'd1) <= 5'd16)
            dout_adc <= valor_adc_atual[16 - (neg_count + 5'd1)];
        else
            dout_adc <= 1'b0;
        end
    end
end

integer erros;

task programa_e_confere;
    input [9:0]    valor;
    input [8*24-1:0] nome;
    begin
        @(posedge leitura_concluida);
        valor_adc_atual = valor;
        @(posedge leitura_concluida);
        #1;
        if (umidade_solo_bruta !== {6'd0, valor}) begin
            $display("FALHA [%s]: esperado=0x%03h obtido=0x%03h",
                    nome, valor, umidade_solo_bruta);
            erros = erros + 1;
        end else begin
            $display("OK          [%s]:  umidade_solo_bruta  =  0x%03h", nome,
umidade_solo_bruta);
        end
    end
end

```

```

end
endtask

initial begin
    $dumpfile("fsm_umiSolo_tb.vcd");
    $dumpvars(0, fsm_umiSolo_tb);

    erros        = 0;
    clk          = 1'b0;
    reset        = 1'b1;
    dout_adc     = 1'b0;
    neg_count    = 5'd0;
    valor_adc_atual = 10'd0;

    #(CLK_PERIODO * 5);
    reset = 1'b0;

    @(posedge leitura_concluida);

    programa_e_confere(10'b0000000000, "zero");
    programa_e_confere(10'b1111111111, "todos_um");
    programa_e_confere(10'b1000000001, "extremos_msb_lsb");
    programa_e_confere(10'b0101010101, "alternado_1");
    programa_e_confere(10'b1010101010, "alternado_2");
    programa_e_confere(10'd595,      "solo_seco_tipico");
    programa_e_confere(10'd130,      "solo_molhado_tipico");

    if (erros == 0)
        $display("\n=== TODOS OS TESTES PASSARAM (fsm_umiSolo) ===");
    else
        $display("\n=== %0d FALHA(S) (fsm_umiSolo) ===", erros);

    $finish;

```

```
end

initial begin
    #(CLK_PERIODO * 200000);
    $display("FALHA: watchdog estourou - simulacao travada");
    $finish;
end

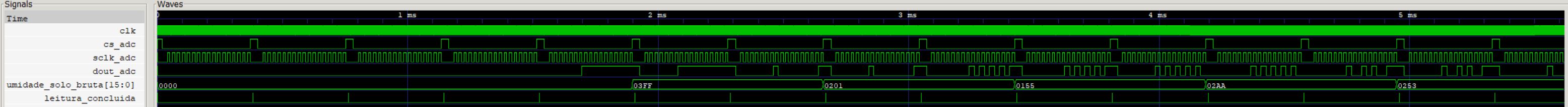
endmodule
```

Resultado

```
(.venv) C:\Users\gribe\OneDrive\Documentos\Codes\INFNET\2026.1\Projeto de
Bloco\Projeto-de-Bloco-Sistemas-Digitais-Embarcados-26E2_5-\AT\FPGA>vvp
sim_umiSolo
VCD info: dumpfile fsm_umiSolo_tb.vcd opened for output.
OK [          zero]: umidade_solo_bruta = 0x0000
OK [        todos_um]: umidade_solo_bruta = 0x03ff
OK [   extremos_msb_lsb]: umidade_solo_bruta = 0x0201
OK [        alternado_1]: umidade_solo_bruta = 0x0155
OK [        alternado_2]: umidade_solo_bruta = 0x02aa
OK [    solo_seco_tipico]: umidade_solo_bruta = 0x0253
OK [  solo_molhado_tipico]: umidade_solo_bruta = 0x0082

=== TODOS OS TESTES PASSARAM (fsm_umiSolo) ===
fsm_umiSolo_tb.v:97: $finish called at 5721427531 (1ps)
```

GTK Wave



Análise

O terminal confirma que os sete casos de teste passaram sem nenhuma falha, com o valor lido na saída `umidade_solo_bruta` correspondendo exatamente, em todos os casos, ao valor que havia sido programado no modelo comportamental do MCP3008 dentro da própria testbench. Os dois primeiros casos cobrem os extremos absolutos de dez bits, sendo `0x0000` o valor mínimo possível e `0x03ff`, que equivale a mil e vinte e três em decimal, o valor máximo possível, confirmando que o módulo não perde nem satura nenhum bit nas duas pontas da faixa de leitura. O terceiro caso, `extremos_msb_lsb`, combina o bit mais significativo e o bit menos significativo ligados com todos os demais desligados, resultando em `0x0201`, que equivale a quinhentos e treze em decimal, e esse padrão em particular é o mais sensível a um eventual erro de deslocamento de um único bit dentro da janela de captura, já que qualquer desalinhamento nessa janela faria esses dois bits isolados aparecerem em posições erradas do resultado final.

Os dois casos seguintes, `alternado_1` e `alternado_2`, testam sequências de bits que se alternam entre zero e um, resultando respectivamente em `0x0155`, trezentos e quarenta e um em decimal, e `0x02aa`, seiscentos e oitenta e dois em decimal, e esses dois padrões, por alternarem a cada posição, expõem de forma direta qualquer inversão na ordem em que os bits são recebidos pelo módulo, já que uma inversão de ordem transformaria um padrão alternado em outro padrão alternado diferente, o que seria fácil de identificar como falha. Por fim, os dois últimos casos usam valores representativos da operação real do sensor de umidade do solo, sendo `0x0253`, ou quinhentos e noventa e cinco em decimal, o valor típico de solo seco, e `0x0082`, ou cento e trinta em decimal, o valor típico de solo molhado, e esses dois valores correspondem exatamente aos dois pontos de calibração usados posteriormente na rotina `converte_umidade_solo.s`, que transforma esse valor bruto em um percentual de umidade.

A forma de onda mostra os seis sinais selecionados ao longo de pouco mais de cinco milissegundos de simulação, o que corresponde a apenas uma pequena fração do tempo total de execução, já que a simulação inteira só termina em cinco vírgula sete milissegundos. O sinal `clk` aparece como uma faixa verde sólida e contínua ao longo de toda a janela, e isso acontece porque o período de clock é de apenas trinta e sete vírgula zero três sete nanossegundos, o que corresponde a uma frequência de aproximadamente vinte e sete megahertz, tempo curto demais para que cada transição individual seja

distinguida visualmente numa janela de milissegundos, de modo que o sinal acaba se apresentando como um bloco compacto de pulsos.

O sinal `cs_adc` mostra uma sucessão bastante regular de quedas breves e regularmente espaçadas ao longo de toda a captura, cada uma delimitando o início e o fim de um ciclo completo de leitura do sensor simulado, e dentro de cada uma dessas janelas em que `cs_adc` está baixo, o sinal `sclk_adc` exhibe um pequeno grupo denso de pulsos, correspondendo aos dezessete pulsos de clock que cada leitura completa do MCP3008 exige. O sinal `dout_adc`, por sua vez, mostra um padrão de pulsos que varia visivelmente de um trecho para outro da captura, já que ele carrega, bit a bit, justamente o valor que está programado em `valor_adc_atual` naquele momento, tornando visualmente perceptível a diferença entre, por exemplo, um trecho em que quase nenhum pulso aparece, compatível com o valor `0x0000`, e um trecho totalmente preenchido de pulsos, compatível com o valor `0x03ff`.

O barramento `umidade_solo_bruta[15:0]` é o sinal que resume de forma mais direta o comportamento do módulo, aparecendo dividido em seis segmentos ao longo da janela capturada, mostrando na ordem os valores `0000`, `03FF`, `0201`, `0155`, `02AA` e `0253`, cada um deles permanecendo estável durante vários ciclos consecutivos de leitura até que a testbench programe um novo valor em `valor_adc_atual`, o que acontece porque o módulo sob teste opera de forma autônoma e contínua, iniciando um novo ciclo de leitura imediatamente após o anterior terminar, independentemente de o valor de entrada ter mudado ou não. O sétimo e último valor da sequência, `0082`, correspondente ao caso `solo_molhado_tipico`, não chega a aparecer dentro dessa janela de cinco milissegundos, já que sua transição acontece um pouco depois do recorte de tempo mostrado na imagem. Por fim, o sinal `leitura_concluida` aparece como uma sequência de pulsos estreitos e regulares, cada um marcando exatamente o instante em que um ciclo de leitura termina e um novo valor passa a estar disponível na saída, e a cadência desses pulsos acompanha diretamente a cadência das quedas de `cs_adc`, já que cada ciclo completo de aquisição gera exatamente um pulso desse sinal.

fsm_ambiente_tb.v

```
`timescale 1ns/1ps

module bme280_slave_bfm (
    inout wire sda,
    input wire scl
);
    localparam [6:0] ENDERECO = 7'h76;

    reg [7:0] regfile [0:255];
    reg    sda_oe;
    reg    sda_out;
    assign sda = sda_oe ? sda_out : 1'bz;

    event ev_start, ev_stop;
    reg  flag_start, flag_stop;

    always @(negedge sda) if (scl === 1'b1) begin
        flag_start = 1'b1;
        -> ev_start;
    end
    always @(posedge sda) if (scl === 1'b1) begin
        flag_stop = 1'b1;
        -> ev_stop;
    end

    task proximo_bit;
        output bit_amostrado;
        begin
            @(posedge scl);
            bit_amostrado = sda;
            @(negedge scl);
        end
    endtask
endmodule
```

```
endtask
```

```
task recebe_byte;
```

```
    output [7:0] byte_lido;
```

```
    integer i;
```

```
    reg    b;
```

```
    begin
```

```
        byte_lido = 8'd0;
```

```
        for (i = 0; i < 8; i = i + 1) begin
```

```
            sda_oe = 1'b0;
```

```
            proximo_bit(b);
```

```
            byte_lido = {byte_lido[6:0], b};
```

```
        end
```

```
    end
```

```
endtask
```

```
task envia_ack;
```

```
    reg descarte;
```

```
    begin
```

```
        sda_oe = 1'b1;
```

```
        sda_out = 1'b0;
```

```
        proximo_bit(descarte);
```

```
        sda_oe = 1'b0;
```

```
    end
```

```
endtask
```

```
task envia_byte;
```

```
    input [7:0] valor;
```

```
    integer i;
```

```
    reg    descarte;
```

```
    begin
```

```
        for (i = 7; i >= 0; i = i - 1) begin
```

```
            sda_oe = 1'b1;
```

```

        sda_out = valor[i];
        proximo_bit(descarte);
    end
end
endtask

task le_ack_do_mestre;
    output ack_recebido;
    begin
        sda_oe = 1'b0;
        proximo_bit(ack_recebido);
    end
endtask

task decide_pos_ponteiro;
    output tipo;
    output bit7_amostrado;
    reg  venceu_negedge;
    begin
        @(posedge scl);
        bit7_amostrado = sda;
        fork
            begin : fecha_bit_dp
                @(negedge scl);
                venceu_negedge = 1'b1;
                disable detecta_restart_dp;
            end
            begin : detecta_restart_dp
                @(ev_start);
                venceu_negedge = 1'b0;
                disable fecha_bit_dp;
            end
        join
    end
endtask

```

```

        tipo = venceu_negedge ? 1'b0 : 1'b1;
    end

endtask

reg [7:0] endereco_rw;
reg [7:0] byte_recebido_1;
reg [7:0] byte_dado;
reg [7:0] ponteiro;
reg    ack_mestre;
reg    tipo_pos;
reg    bit7_pos;
integer k;
reg    bk;
integer i;

initial begin
    sda_oe = 1'b0;
    sda_out = 1'b0;
    ponteiro = 8'd0;
    for (i = 0; i < 256; i = i + 1) regfile[i] = 8'h00;

    forever begin
        @(ev_start);
        recebe_byte(endereco_rw);

        if (endereco_rw[7:1] == ENDERECO) begin
            envia_ack;

            if (endereco_rw[0] == 1'b0) begin
                recebe_byte(byte_recebido_1);
                envia_ack;

                decide_pos_ponteiro(tipo_pos, bit7_pos);
            end
        end
    end
end

```

```

if (tipo_pos == 1'b0) begin
    byte_dado = {7'd0, bit7_pos};
    for (k = 6; k >= 0; k = k - 1) begin
        sda_oe = 1'b0;
        proximo_bit(bk);
        byte_dado = {byte_dado[6:0], bk};
    end
    regfile[byte_recebido_1] = byte_dado;
    envia_ack;
end else begin
    ponteiro = byte_recebido_1;
    recebe_byte(endereco_rw);
    if (endereco_rw[7:1] == ENDERECO) begin
        envia_ack;
        ack_mestre = 1'b0;
        while (ack_mestre == 1'b0) begin
            envia_byte(regfile[ponteiro]);
            ponteiro = ponteiro + 8'd1;
            le_ack_do_mestre(ack_mestre);
        end
    end
end
end
end else begin
    ack_mestre = 1'b0;
    while (ack_mestre == 1'b0) begin
        envia_byte(regfile[ponteiro]);
        ponteiro = ponteiro + 8'd1;
        le_ack_do_mestre(ack_mestre);
    end
end
end
end
end

```

```

end
endmodule

module bh1750_slave_bfm (
    inout wire sda,
    input wire scl
);
    localparam [6:0] ENDERECO = 7'h23;

    reg sda_oe;
    reg sda_out;
    assign sda = sda_oe ? sda_out : 1'bz;

    event ev_start;
    reg flag_start;

    always @(negedge sda) if (scl === 1'b1) begin
        flag_start = 1'b1;
        -> ev_start;
    end

    task proximo_bit;
        output bit_amostrado;
        begin
            @(posedge scl);
            bit_amostrado = sda;
            @(negedge scl);
        end
    endtask

    task recebe_byte;
        output [7:0] byte_lido;
        integer i;

```

```

    reg  b;
    begin
        byte_lido = 8'd0;
        for (i = 0; i < 8; i = i + 1) begin
            sda_oe = 1'b0;
            proximo_bit(b);
            byte_lido = {byte_lido[6:0], b};
        end
    end
endtask

```

```

task envia_ack;
    reg descarte;
    begin
        sda_oe = 1'b1;
        sda_out = 1'b0;
        proximo_bit(descarte);
        sda_oe = 1'b0;
    end
endtask

```

```

task envia_byte;
    input [7:0] valor;
    integer i;
    reg  descarte;
    begin
        for (i = 7; i >= 0; i = i - 1) begin
            sda_oe = 1'b1;
            sda_out = valor[i];
            proximo_bit(descarte);
        end
    end
endtask

```

```

task le_ack_do_mestre;
    output ack_recebido;
    begin
        sda_oe = 1'b0;
        proximo_bit(ack_recebido);
    end
endtask

reg [7:0] endereco_rw;
reg [7:0] comando_recebido;
reg [15:0] luz_bruta_teste;
reg      ack_mestre;

initial begin
    sda_oe      = 1'b0;
    sda_out     = 1'b0;
    luz_bruta_teste = 16'h05DC;

    forever begin
        @(ev_start);
        recebe_byte(endereco_rw);

        if (endereco_rw[7:1] == ENDERECO) begin
            envia_ack;

            if (endereco_rw[0] == 1'b0) begin
                recebe_byte(comando_recebido);
                envia_ack;
                if (comando_recebido != 8'h10)
                    $display("AVISO [BH1750 BFM]: comando inesperado 0x%02h
(esperado 0x10)",
                        comando_recebido);
            end
        end
    end
end

```

```

        end else begin
            envia_byte(luz_bruta_teste[15:8]);
            le_ack_do_mestre(ack_mestre);
            envia_byte(luz_bruta_teste[7:0]);
            le_ack_do_mestre(ack_mestre);
        end
    end
end
end
end
endmodule

module fsm_ambiente_tb;

    real CLK_PERIODO = 37.037;

    reg clk;
    reg reset;
    wire scl;
    wire sda_saida;
    wire sda_direcao;
    wire sda_bus;

    wire [23:0] pressao_bruta;
    wire [23:0] temperatura_bruta;
    wire [15:0] umidade_bruta;
    wire [15:0] luminosidade_bruta;
    wire      leitura_concluida;

    pullup(sda_bus);
    assign sda_bus = sda_direcao ? sda_saida : 1'bz;

    fsm_ambiente dut (
        .clk          (clk),

```

```

.reset      (reset),
.scl        (scl),
.sda_saida  (sda_saida),
.sda_direcao (sda_direcao),
.sda_entrada (sda_bus),
.pressao_bruta (pressao_bruta),
.temperatura_bruta (temperatura_bruta),
.umidade_bruta (umidade_bruta),
.luminosidade_bruta (luminosidade_bruta),
.leitura_concluida (leitura_concluida)
);

bme280_slave_bfm u_bme280 (.sda(sda_bus), .scl(scl));
bh1750_slave_bfm u_bh1750 (.sda(sda_bus), .scl(scl));

always #(CLK_PERIODO/2) clk = ~clk;

integer erros;

localparam [23:0] PRESSAO_TESTE    = 24'h548A10;
localparam [23:0] TEMPERATURA_TESTE = 24'h7F3C20;
localparam [15:0] UMIDADE_TESTE    = 16'h6B90;
localparam [15:0] LUZ_TESTE        = 16'h1A2B;

task confere;
    input [8*32-1:0] nome;
    input [31:0]    esperado;
    input [31:0]    obtido;
    begin
        if (esperado !== obtido) begin
            $display("FALHA [%s]: esperado=0x%08h obtido=0x%08h", nome,
esperado, obtido);
            erros = erros + 1;
        end
    end
endtask

```

```

        end else begin
            $display("OK   [%s]: 0x%08h", nome, obtido);
        end
    end
endtask

initial begin
    $dumpfile("fsm_ambiente_tb.vcd");
    $dumpvars(0, fsm_ambiente_tb);

    erros = 0;
    clk  = 1'b0;
    reset = 1'b1;

    u_bme280.regfile[8'hF7] = PRESSAO_TESTE[23:16];
    u_bme280.regfile[8'hF8] = PRESSAO_TESTE[15:8];
    u_bme280.regfile[8'hF9] = PRESSAO_TESTE[7:0];
    u_bme280.regfile[8'hFA] = TEMPERATURA_TESTE[23:16];
    u_bme280.regfile[8'hFB] = TEMPERATURA_TESTE[15:8];
    u_bme280.regfile[8'hFC] = TEMPERATURA_TESTE[7:0];
    u_bme280.regfile[8'hFD] = UMIDADE_TESTE[15:8];
    u_bme280.regfile[8'hFE] = UMIDADE_TESTE[7:0];
    u_bh1750.luz_bruta_teste = LUZ_TESTE;

    #(CLK_PERIODO * 5);
    reset = 1'b0;

    @(posedge leitura_concluida);
    #1;

    $display("\n--- Configuracao do BME280 ---");
    confere("reg 0xF2 (ctrl_hum)", 8'h01, u_bme280.regfile[8'hF2]);
    confere("reg 0xF4 (ctrl_meas)", 8'h27, u_bme280.regfile[8'hF4]);

```

```

confere("reg 0xF5 (config)", 8'hA0, u_bme280.regfile[8'hF5]);

$display("\n--- Primeira leitura ambiental ---");
confere("pressao_bruta", PRESSAO_TESTE, pressao_bruta);
confere("temperatura_bruta", TEMPERATURA_TESTE, temperatura_bruta);
confere("umidade_bruta", UMIDADE_TESTE, umidade_bruta);
confere("luminosidade_bruta", LUZ_TESTE, luminosidade_bruta);

u_bme280.regfile[8'hF7] = 8'h11;
u_bme280.regfile[8'hF8] = 8'h22;
u_bme280.regfile[8'hF9] = 8'h33;
u_bme280.regfile[8'hFA] = 8'h44;
u_bme280.regfile[8'hFB] = 8'h55;
u_bme280.regfile[8'hFC] = 8'h66;
u_bme280.regfile[8'hFD] = 8'h77;
u_bme280.regfile[8'hFE] = 8'h88;
u_bh1750.luz_bruta_teste = 16'h4321;

@(posedge leitura_concluida);
#1;

$display("\n--- Segunda leitura ambiental (regime) ---");
confere("pressao_bruta", 24'h112233, pressao_bruta);
confere("temperatura_bruta", 24'h445566, temperatura_bruta);
confere("umidade_bruta", 16'h7788, umidade_bruta);
confere("luminosidade_bruta", 16'h4321, luminosidade_bruta);

if (erros == 0)
    $display("\n=== TODOS OS TESTES PASSARAM (fsm_ambiente) ===");
else
    $display("\n=== %0d FALHA(S) (fsm_ambiente) ===", erros);

$finish;

```

```
end

initial begin
    #(CLK_PERIODO * 15000000);
    $display("FALHA: watchdog estourou - simulacao travada");
    $finish;
end

endmodule
```

Resultado

```
(.venv) C:\Users\gribe\OneDrive\Documentos\Codes\INFNET\2026.1\Projeto de
Bloco\Projeto-de-Bloco-Sistemas-Digitais-Embarcados-26E2_5-\AT\FPGA>vvp
sim_ambiente
VCD info: dumpfile fsm_ambiente_tb.vcd opened for output.

--- Configuracao do BME280 ---
OK [      reg 0xF2 (ctrl_hum)]: 0x00000001
OK [      reg 0xF4 (ctrl_meas)]: 0x00000027
OK [      reg 0xF5 (config)]: 0x000000a0

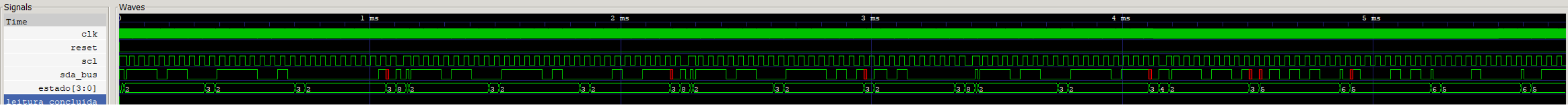
--- Primeira leitura ambiental ---
OK [      pressao_bruta]: 0x00548a10
OK [      temperatura_bruta]: 0x007f3c20
OK [      umidade_bruta]: 0x00006b90
OK [      luminosidade_bruta]: 0x00001a2b

--- Segunda leitura ambiental (regime) ---
OK [      pressao_bruta]: 0x00112233
OK [      temperatura_bruta]: 0x00445566
OK [      umidade_bruta]: 0x00007788
OK [      luminosidade_bruta]: 0x00004321
```

=== TODOS OS TESTES PASSARAM (fsm_ambiente) ===

fsm_ambiente_tb.v:393: \$finish called at 215370470857 (1ps)

GTK Wave



Análise

O terminal confirma, na primeira parte, que a rotina de configuração do BME280 escreveu corretamente os três registradores esperados no modelo comportamental do sensor, sendo 0x01 gravado no registrador 0xF2, responsável por habilitar a medição de umidade, 0x27 gravado no registrador 0xF4, responsável por configurar o modo de medição de temperatura e pressão, e 0xA0 gravado no registrador 0xF5, responsável pelos parâmetros de filtro e tempo de espera entre amostras. Esses três valores são lidos diretamente de dentro do modelo comportamental do sensor, através do caminho hierárquico `u_bme280.regfile`, o que confirma que a escrita realmente chegou ao destino certo dentro do barramento simulado, e não apenas que a FPGA acreditava ter enviado algo correto.

Na segunda parte, a primeira leitura ambiental completa confirma que os quatro valores brutos entregues pelo módulo, `pressao_bruta`, `temperatura_bruta`, `umidade_bruta` e `luminosidade_bruta`, correspondem exatamente aos valores programados no início da simulação nos registradores 0xF7 a 0xFE do BME280 simulado e na variável `luz_bruta_teste` do BH1750 simulado, sendo eles 0x548A10 para pressão, 0x7F3C20 para temperatura, 0x6B90 para umidade do ar e 0x1A2B para luminosidade. A terceira parte, chamada de segunda leitura ambiental em regime, repete essa mesma verificação depois que a testbench reprograma os dois sensores simulados com um segundo conjunto de valores, 0x112233, 0x445566, 0x7788 e 0x4321 respectivamente, e o fato de esses novos valores também aparecerem corretamente na saída confirma que o módulo é capaz de realizar leituras repetidas de forma contínua, sem travar depois da primeira leitura e sem precisar reconfigurar os sensores a cada novo ciclo, já que a configuração só é feita uma única vez, logo depois do reset.

A forma de onda mostra os seis sinais selecionados ao longo de pouco mais de cinco milissegundos de simulação, e é importante destacar logo de início que essa janela cobre uma fração muito pequena do tempo total, já que a simulação inteira só termina em duzentos e quinze vírgula trinta e sete milissegundos. Isso acontece porque, depois que o BH1750 recebe o comando de início de medição, o módulo entra no estado ESPERA e permanece ali contando até vinte mil ciclos antes de prosseguir, reproduzindo o tempo de aquecimento exigido pelo datasheet desse sensor, o que sozinho já responde por cerca de duzentos dos duzentos e quinze milissegundos totais. Na prática, isso significa que a janela capturada nessa imagem mostra apenas a configuração do BME280 e a leitura em rajada dos seus oito bytes de dados, e termina antes mesmo de a FPGA começar a

conversar com o BH1750, de modo que nenhum pulso do sinal leitura_concluida chega a aparecer dentro desse recorte, já que esse sinal só sobe uma vez que o ciclo inteiro, incluindo a espera do BH1750, estiver terminado.

O sinal clk aparece como uma faixa verde compacta e praticamente sólida ao longo de toda a janela, pelo mesmo motivo já observado nas outras testbenches, sendo o período de trinta e sete nanossegundos curto demais para que cada transição individual seja distinguida numa escala de milissegundos. O sinal reset permanece baixo durante toda a captura, já que ele só fica em nível alto durante os primeiros cinco períodos de clock da simulação, tempo tão curto que não chega a aparecer de forma visível nessa escala. O sinal scl mostra uma sucessão regular de pulsos ao longo de quase toda a janela, refletindo o fato de que o barramento I2C está ativo na maior parte do tempo mostrado, primeiro transportando as três escritas de configuração e depois a leitura em rajada.

O sinal estado[3:0] é o que melhor conta a história dessa captura, já que ele alterna entre os valores dois e três repetidamente logo no início, o que corresponde exatamente ao estado WRITE_BYTE seguido do estado CHECK_ACK, sendo esse par repetido três vezes seguidas para cada um dos três bytes de uma mesma escrita de registrador, sendo eles o byte de endereço do sensor, o byte que aponta para o registrador de destino e o byte com o valor a ser gravado. Depois desses três pares, aparece brevemente o valor oito, correspondente ao estado STOP, que fecha essa transação antes de a máquina voltar ao início da sequência para configurar o próximo registrador, e esse padrão completo, de seis alternâncias entre dois e três seguidas de um oito, se repete três vezes ao longo da captura, uma para cada um dos registradores 0xF2, 0xF4 e 0xF5. Mais à direita na captura, o padrão muda de figura, e passam a aparecer os valores quatro, cinco e seis intercalados, correspondendo respectivamente aos estados RESTART, READ_BYTE e SEND_ACK, o que já corresponde ao início da leitura em rajada dos oito bytes de dados ambientais, na qual a FPGA reconhece cada byte recebido com um bit de confirmação antes de solicitar o próximo, sendo esse o trecho da simulação em que a captura termina, um pouco antes de o oitavo e último byte ser confirmado com um NACK em vez de um ACK.

O sinal sda_bus é o que exige leitura mais cuidadosa, já que ele representa fisicamente um barramento de dreno aberto compartilhado entre a FPGA e os dois modelos de sensor, e por isso alterna entre níveis alto e baixo conforme quem estiver com a direção do barramento a cada momento, sendo os pulsos mais curtos e agrupados os bits de cada byte transmitido e as quedas mais isoladas os bits de confirmação enviados pelo sensor de volta para a FPGA. Alguns trechos muito curtos aparecem destacados em

vermelho ao longo desse sinal, o que no GTKWave indica um valor momentaneamente indefinido, e isso é esperado nesse tipo de barramento compartilhado, já que existe um pequeno intervalo de tempo, da ordem de frações de um período de clock, em que a direção do sinal está sendo trocada entre um driver e outro, e nesse intervalo o simulador não consegue determinar com certeza qual valor prevalece, sem que isso represente nenhum problema real de funcionamento, já que esses instantes acontecem sempre fora do momento em que o valor do bit é de fato amostrado, na borda de subida de scl.

O sinal `leitura_concluida` realmente está entre os sinais selecionados para essa captura, mas não chega a mostrar nenhuma transição visível dentro da janela de cinco milissegundos mostrada, e o motivo é simplesmente a diferença de escala de tempo entre esse sinal e todos os outros. Como esse sinal só sobe uma única vez ao final de cada ciclo completo de leitura, e o primeiro desses ciclos só termina depois de aproximadamente duzentos milissegundos por causa da espera de aquecimento do BH1750, os dois únicos pulsos que ele produz ao longo de toda a simulação ficam concentrados nos extremos mais distantes da linha do tempo, um perto dos duzentos milissegundos e outro perto do final da simulação, em duzentos e quinze milissegundos. Para capturar esses dois pulsos numa única imagem seria necessário abrir a escala de tempo do GTKWave para englobar a simulação inteira, e isso comprimiria justamente os sinais mais rápidos, como `clk`, `scl` e `sda_bus`, a ponto de suas transições individuais deixarem de ser distinguíveis, transformando essas linhas em blocos sólidos sem nenhuma informação legível, o que anularia o propósito da captura, que é mostrar em detalhe a sequência de configuração e a leitura em rajada. Por isso optou-se por manter o recorte inicial, mais estreito, e o comportamento de `leitura_concluida` nos dois instantes em que ele de fato pulsa pode ser conferido diretamente abrindo o arquivo `fsm_ambiente_tb.vcd` no GTKWave e navegando manualmente até esses dois pontos específicos se necessário.

top_fpga_tb.v

```
`timescale 1ns/1ps

module bme280_slave_bfm (
    inout wire sda,
    input wire scl
);
    localparam [6:0] ENDERECO = 7'h76;

    reg [7:0] regfile [0:255];
    reg    sda_oe;
    reg    sda_out;
    assign sda = sda_oe ? sda_out : 1'bz;

    event ev_start, ev_stop;
    reg  flag_start, flag_stop;

    always @(negedge sda) if (scl === 1'b1) begin
        flag_start = 1'b1;
        -> ev_start;
    end
    always @(posedge sda) if (scl === 1'b1) begin
        flag_stop = 1'b1;
        -> ev_stop;
    end

    task proximo_bit;
        output bit_amostrado;
        begin
            @(posedge scl);
            bit_amostrado = sda;
            @(negedge scl);
        end
    endtask
endmodule
```

```
endtask
```

```
task recebe_byte;
```

```
    output [7:0] byte_lido;
```

```
    integer i;
```

```
    reg    b;
```

```
    begin
```

```
        byte_lido = 8'd0;
```

```
        for (i = 0; i < 8; i = i + 1) begin
```

```
            sda_oe = 1'b0;
```

```
            proximo_bit(b);
```

```
            byte_lido = {byte_lido[6:0], b};
```

```
        end
```

```
    end
```

```
endtask
```

```
task envia_ack;
```

```
    reg descarte;
```

```
    begin
```

```
        sda_oe = 1'b1;
```

```
        sda_out = 1'b0;
```

```
        proximo_bit(descarte);
```

```
        sda_oe = 1'b0;
```

```
    end
```

```
endtask
```

```
task envia_byte;
```

```
    input [7:0] valor;
```

```
    integer i;
```

```
    reg    descarte;
```

```
    begin
```

```
        for (i = 7; i >= 0; i = i - 1) begin
```

```
            sda_oe = 1'b1;
```

```

        sda_out = valor[i];
        proximo_bit(descarte);
    end
end
endtask

task le_ack_do_mestre;
    output ack_recebido;
    begin
        sda_oe = 1'b0;
        proximo_bit(ack_recebido);
    end
endtask

task decide_pos_ponteiro;
    output tipo;
    output bit7_amostrado;
    reg  venceu_negedge;
    begin
        @(posedge scl);
        bit7_amostrado = sda;
        fork
            begin : fecha_bit_dp
                @(negedge scl);
                venceu_negedge = 1'b1;
                disable detecta_restart_dp;
            end
            begin : detecta_restart_dp
                @(ev_start);
                venceu_negedge = 1'b0;
                disable fecha_bit_dp;
            end
        join
    end
endtask

```

```

        tipo = venceu_negedge ? 1'b0 : 1'b1;
    end
endtask

reg [7:0] endereco_rw;
reg [7:0] byte_recebido_1;
reg [7:0] byte_dado;
reg [7:0] ponteiro;
reg    ack_mestre;
reg    tipo_pos;
reg    bit7_pos;
integer k;
reg    bk;
integer i;

initial begin
    sda_oe = 1'b0;
    sda_out = 1'b0;
    ponteiro = 8'd0;
    for (i = 0; i < 256; i = i + 1) regfile[i] = 8'h00;

    forever begin
        @(ev_start);
        recebe_byte(endereco_rw);

        if (endereco_rw[7:1] == ENDERECO) begin
            envia_ack;

            if (endereco_rw[0] == 1'b0) begin
                recebe_byte(byte_recebido_1);
                envia_ack;

                decide_pos_ponteiro(tipo_pos, bit7_pos);
            end
        end
    end
end

```

```

if (tipo_pos == 1'b0) begin
    byte_dado = {7'd0, bit7_pos};
    for (k = 6; k >= 0; k = k - 1) begin
        sda_oe = 1'b0;
        proximo_bit(bk);
        byte_dado = {byte_dado[6:0], bk};
    end
    regfile[byte_recebido_1] = byte_dado;
    envia_ack;
end else begin
    ponteiro = byte_recebido_1;
    recebe_byte(endereco_rw);
    if (endereco_rw[7:1] == ENDERECO) begin
        envia_ack;
        ack_mestre = 1'b0;
        while (ack_mestre == 1'b0) begin
            envia_byte(regfile[ponteiro]);
            ponteiro = ponteiro + 8'd1;
            le_ack_do_mestre(ack_mestre);
        end
    end
end
end
end else begin
    ack_mestre = 1'b0;
    while (ack_mestre == 1'b0) begin
        envia_byte(regfile[ponteiro]);
        ponteiro = ponteiro + 8'd1;
        le_ack_do_mestre(ack_mestre);
    end
end
end
end
end

```

```

end
endmodule

module bh1750_slave_bfm (
    inout wire sda,
    input wire scl
);
    localparam [6:0] ENDERECO = 7'h23;

    reg sda_oe;
    reg sda_out;
    assign sda = sda_oe ? sda_out : 1'bz;

    event ev_start;
    reg flag_start;

    always @(negedge sda) if (scl === 1'b1) begin
        flag_start = 1'b1;
        -> ev_start;
    end

    task proximo_bit;
        output bit_amostrado;
        begin
            @(posedge scl);
            bit_amostrado = sda;
            @(negedge scl);
        end
    endtask

    task recebe_byte;
        output [7:0] byte_lido;
        integer i;

```

```

    reg  b;
begin
    byte_lido = 8'd0;
    for (i = 0; i < 8; i = i + 1) begin
        sda_oe = 1'b0;
        proximo_bit(b);
        byte_lido = {byte_lido[6:0], b};
    end
end
endtask

```

```

task envia_ack;
    reg descarte;
begin
    sda_oe = 1'b1;
    sda_out = 1'b0;
    proximo_bit(descarte);
    sda_oe = 1'b0;
end
endtask

```

```

task envia_byte;
    input [7:0] valor;
    integer i;
    reg  descarte;
begin
    for (i = 7; i >= 0; i = i - 1) begin
        sda_oe = 1'b1;
        sda_out = valor[i];
        proximo_bit(descarte);
    end
end
endtask

```

```

task le_ack_do_mestre;
    output ack_recebido;
    begin
        sda_oe = 1'b0;
        proximo_bit(ack_recebido);
    end
endtask

reg [7:0] endereco_rw;
reg [7:0] comando_recebido;
reg [15:0] luz_bruta_teste;
reg      ack_mestre;

initial begin
    sda_oe      = 1'b0;
    sda_out     = 1'b0;
    luz_bruta_teste = 16'h05DC;

    forever begin
        @(ev_start);
        recebe_byte(endereco_rw);

        if (endereco_rw[7:1] == ENDERECO) begin
            envia_ack;

            if (endereco_rw[0] == 1'b0) begin
                recebe_byte(comando_recebido);
                envia_ack;
                if (comando_recebido != 8'h10)
                    $display("AVISO [BH1750 BFM]: comando inesperado 0x%02h
(esperado 0x10)",
                        comando_recebido);
            end
        end
    end
end

```

```

        end else begin
            envia_byte(luz_bruta_teste[15:8]);
            le_ack_do_mestre(ack_mestre);
            envia_byte(luz_bruta_teste[7:0]);
            le_ack_do_mestre(ack_mestre);
        end
    end
end
end
end
endmodule

```

```

module top_fpga_tb;

```

```

    localparam [7:0] COMANDO_VALIDO = 8'hA5;
    localparam [7:0] COMANDO_INVALIDO = 8'h00;
    localparam [7:0] STATUS_OK = 8'h5A;
    localparam [7:0] STATUS_ERRO = 8'h00;

```

```

    real CLK_PERIODO = 37.037;

```

```

    reg clk_pino = 1'b0;
    reg sclk_pino = 1'b0;
    reg cs_pino = 1'b1;
    reg mosi_pino = 1'b0;
    wire miso_pino;
    wire led_status;
    wire led_azul;
    wire sda_i2c;
    wire scl_i2c;
    wire sclk_adc_pino;
    wire cs_adc_pino;
    wire din_adc_pino;
    reg dout_adc_pino = 1'b0;

```

```
pullup(sda_i2c);
```

```
top_fpga dut (  
    .clk_pino    (clk_pino),  
    .sclk_pino   (sclk_pino),  
    .cs_pino     (cs_pino),  
    .mosi_pino   (mosi_pino),  
    .miso_pino   (miso_pino),  
    .led_status  (led_status),  
    .led_azul    (led_azul),  
    .sda_i2c     (sda_i2c),  
    .scl_i2c     (scl_i2c),  
    .sclk_adc_pino (sclk_adc_pino),  
    .cs_adc_pino (cs_adc_pino),  
    .din_adc_pino (din_adc_pino),  
    .dout_adc_pino (dout_adc_pino)  
);
```

```
bme280_slave_bfm u_bme280 (.sda(sda_i2c), .scl(scl_i2c));
```

```
bh1750_slave_bfm u_bh1750 (.sda(sda_i2c), .scl(scl_i2c));
```

```
reg [4:0] neg_count;
```

```
reg [9:0] valor_adc_teste;
```

```
always @(negedge cs_adc_pino) begin
```

```
    neg_count    <= 5'd0;
```

```
    dout_adc_pino <= 1'b0;
```

```
end
```

```
always @(negedge sclk_adc_pino) begin
```

```
    if (!cs_adc_pino) begin
```

```
        neg_count <= neg_count + 5'd1;
```

```

        if ((neg_count + 5'd1) >= 5'd7 && (neg_count + 5'd1) <= 5'd16)
            dout_adc_pino <= valor_adc_teste[16 - (neg_count + 5'd1)];
        else
            dout_adc_pino <= 1'b0;
        end
    end

always #(CLK_PERIODO/2) clk_pino = ~clk_pino;

reg [7:0] rx_frame [0:13];
localparam MEIO_PERIODO_SPI = 50;

task le_frame_como_rpi;
    input [7:0] comando;
    integer i, b;
    begin
        cs_pino = 1'b0;
        #(MEIO_PERIODO_SPI);
        for (i = 0; i < 14; i = i + 1) begin
            rx_frame[i] = 8'd0;
            for (b = 0; b < 8; b = b + 1) begin
                if (i == 0)
                    mosi_pino = comando[7 - b];
                else
                    mosi_pino = 1'b0;
                sclk_pino = 1'b1;
                #(MEIO_PERIODO_SPI);
                rx_frame[i] = {rx_frame[i][6:0], miso_pino};
                sclk_pino = 1'b0;
                #(MEIO_PERIODO_SPI);
            end
        end
    end

    cs_pino = 1'b1;

```

```

        #(MEIO_PERIODO_SPI);

    end

endtask

integer erros;

task confere_byte;
    input [8*16-1:0] nome;
    input [3:0]      indice;
    input [7:0]      esperado;
    begin
        if (rx_frame[indice] !== esperado) begin
            $display("FALHA [%s] byte %0d: esperado=0x%02h obtido=0x%02h",
                    nome, indice, esperado, rx_frame[indice]);
            erros = erros + 1;
        end else begin
            $display("OK   [%s] byte %0d = 0x%02h", nome, indice, rx_frame[indice]);
        end
    end
endtask

task confere_status;
    input [7:0] esperado;
    begin
        if (rx_frame[0] !== esperado) begin
            $display("FALHA [status]: esperado=0x%02h obtido=0x%02h", esperado,
rx_frame[0]);
            erros = erros + 1;
        end else begin
            $display("OK   [status] = 0x%02h", rx_frame[0]);
        end
    end
endtask

```

```

task confere_checksum;
    integer k;
    reg [7:0] soma;
    begin
        soma = 8'd0;
        for (k = 1; k <= 12; k = k + 1)
            soma = soma + rx_frame[k];
        if (soma !== rx_frame[13]) begin
            $display("FALHA [checksum]: esperado=0x%02h obtido=0x%02h", soma,
rx_frame[13]);
            erros = erros + 1;
        end else begin
            $display("OK [checksum] = 0x%02h", rx_frame[13]);
        end
    end
endtask

localparam [23:0] PRESSAO_TESTE    = 24'h548A10;
localparam [23:0] TEMPERATURA_TESTE = 24'h7F3C20;
localparam [15:0] UMIDADE_TESTE    = 16'h6B90;
localparam [15:0] LUZ_TESTE        = 16'h1A2B;
localparam [15:0] SOLO_TESTE       = 16'h0284;

initial begin
    $dumpfile("top_fpga_tb.vcd");
    $dumpvars(0, top_fpga_tb);

    erros          = 0;
    neg_count      = 5'd0;
    valor_adc_teste = SOLO_TESTE[9:0];

    u_bme280.regfile[8'hF7] = PRESSAO_TESTE[23:16];

```

```

u_bme280.regfile[8'hF8] = PRESSAO_TESTE[15:8];
u_bme280.regfile[8'hF9] = PRESSAO_TESTE[7:0];
u_bme280.regfile[8'hFA] = TEMPERATURA_TESTE[23:16];
u_bme280.regfile[8'hFB] = TEMPERATURA_TESTE[15:8];
u_bme280.regfile[8'hFC] = TEMPERATURA_TESTE[7:0];
u_bme280.regfile[8'hFD] = UMIDADE_TESTE[15:8];
u_bme280.regfile[8'hFE] = UMIDADE_TESTE[7:0];
u_bh1750.luz_bruta_teste = LUZ_TESTE;

```

```

repeat (17) @(posedge dut.leitura_concluida_ambiente);

```

```

#(CLK_PERIODO * 5);

```

```

le_frame_como_rpi(COMANDO_VALIDO);

```

```

#(CLK_PERIODO * 5);

```

```

le_frame_como_rpi(COMANDO_VALIDO);

```

```

$display("\n--- Caso 1: comando valido, pipeline completo (sensores -> FPGA ->
SPI) ---");

```

```

confere_status(STATUS_OK);
confere_byte("pressao", 1, PRESSAO_TESTE[23:16]);
confere_byte("pressao", 2, PRESSAO_TESTE[15:8]);
confere_byte("pressao", 3, PRESSAO_TESTE[7:0]);
confere_byte("temperatura", 4, TEMPERATURA_TESTE[23:16]);
confere_byte("temperatura", 5, TEMPERATURA_TESTE[15:8]);
confere_byte("temperatura", 6, TEMPERATURA_TESTE[7:0]);
confere_byte("umidade", 7, UMIDADE_TESTE[15:8]);
confere_byte("umidade", 8, UMIDADE_TESTE[7:0]);
confere_byte("luz", 9, LUZ_TESTE[15:8]);
confere_byte("luz", 10, LUZ_TESTE[7:0]);
confere_byte("solo", 11, SOLO_TESTE[15:8]);

```

```

confere_byte("solo",    12, 8'h7B);
confere_checksum;

le_frame_como_rpi(COMANDO_INVALIDO);
le_frame_como_rpi(COMANDO_INVALIDO);
$display("\n--- Caso 2: comando invalido (0x00 em vez de 0xA5) ---");
confere_status(STATUS_ERRO);

$display("\n--- Caso 3: led_azul durante e depois de uma transacao valida ---");
cs_pino = 1'b0;
#(MEIO_PERIODO_SPI);
for (integer i = 7; i >= 0; i = i - 1) begin
    mosi_pino = COMANDO_VALIDO[i];
    sclk_pino = 1'b1;
    #(MEIO_PERIODO_SPI);
    sclk_pino = 1'b0;
    #(MEIO_PERIODO_SPI);
end
if (led_azul !== 1'b1) begin
    $display("FALHA [led_azul]: esperado=1 obtido=%b logo apos o comando",
led_azul);
    erros = erros + 1;
end else begin
    $display("OK  [led_azul] = 1 logo apos o comando");
end

for (integer i = 0; i < 104; i = i + 1) begin
    mosi_pino = 1'b0;
    sclk_pino = 1'b1;
    #(MEIO_PERIODO_SPI);
    sclk_pino = 1'b0;
    #(MEIO_PERIODO_SPI);
end

```

```

cs_pino = 1'b1;
#(MEIO_PERIODO_SPI);
if (led_azul !== 1'b0) begin
    $display("FALHA [led_azul]: esperado=0 obtido=%b apos o fim da transacao",
led_azul);
    erros = erros + 1;
end else begin
    $display("OK [led_azul] = 0 apos o fim da transacao");
end

if (erros == 0)
    $display("\n=== TODOS OS TESTES PASSARAM (top_fpga - pipeline
completo) ===");
else
    $display("\n=== %0d FALHA(S) (top_fpga) ===", erros);

$finish;
end

initial begin
    #(CLK_PERIODO * 15000000);
    $display("FALHA: watchdog estourou - simulacao travada");
    $finish;
end

endmodule

```

Resultado

```

(.venv) C:\Users\gribe\OneDrive\Documentos\Codes\INFNET2026.1\Projeto de
Bloco\Projeto-de-Bloco-Sistemas-Digitais-Embarcados-26E2_5-\AT\FPGA>vvp
sim_top
VCD info: dumpfile top_fpga_tb.vcd opened for output.

```

--- Caso 1: comando valido, pipeline completo (sensores -> FPGA -> SPI) ---

OK [status] = 0x5a

OK [pressao] byte 1 = 0x54

OK [pressao] byte 2 = 0x8a

OK [pressao] byte 3 = 0x10

OK [temperatura] byte 4 = 0x7f

OK [temperatura] byte 5 = 0x3c

OK [temperatura] byte 6 = 0x20

OK [umidade] byte 7 = 0x6b

OK [umidade] byte 8 = 0x90

OK [luz] byte 9 = 0x1a

OK [luz] byte 10 = 0x2b

OK [solo] byte 11 = 0x02

OK [solo] byte 12 = 0x7b

OK [checksum] = 0x86

--- Caso 2: comando invalido (0x00 em vez de 0xA5) ---

OK [status] = 0x00

--- Caso 3: led_azul durante e depois de uma transacao valida ---

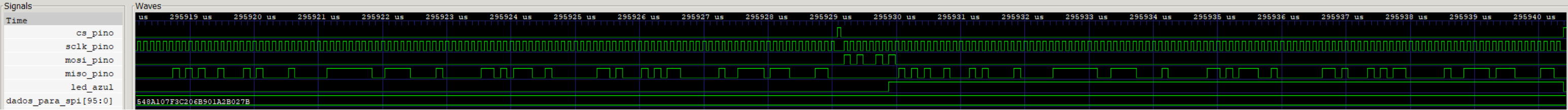
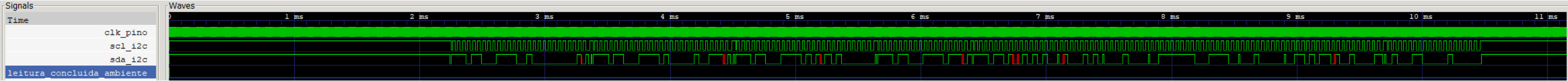
OK [led_azul] = 1 logo apos o comando

OK [led_azul] = 0 apos o fim da transacao

=== TODOS OS TESTES PASSARAM (top_fpga - pipeline completo) ===

top_fpga_tb.v:504: \$finish called at 295940433513 (1ps)

GTK Wave



Análise

O terminal confirma que o pipeline completo, desde os dois modelos de sensor simulados até a extração do quadro pela interface SPI, entrega exatamente os valores esperados no primeiro caso de teste. O byte de status vem como 0x5a, confirmando que o comando enviado foi reconhecido, e os doze bytes de dados seguem na ordem já conhecida do protocolo, sendo 0x54, 0x8a e 0x10 para a pressão, 0x7f, 0x3c e 0x20 para a temperatura, 0x6b e 0x90 para a umidade do ar, 0x1a e 0x2b para a luminosidade, e 0x02 e 0x7b para a umidade do solo. O checksum recebido, 0x86, também bate com o que é recalculado a partir desses doze bytes, o que mostra que o byte de verificação está sendo formado corretamente em cima dos valores que de fato saíram do restante do pipeline, e não apenas em cima de um valor fixo qualquer.

O segundo caso confirma que, ao enviar o comando 0x00 em vez de 0xa5, o status retornado passa a ser 0x00, mostrando que o reconhecimento do comando continua funcionando normalmente mesmo depois de todo o caminho de aquisição já ter sido percorrido uma vez. O terceiro caso acompanha o sinal `led_azul` em dois momentos de uma transação controlada manualmente, confirmando que ele sobe para um logo depois que os oito bits do comando terminam de ser recebidos e volta para zero assim que a linha de seleção de chip sobe novamente ao final da transação, exatamente o mesmo comportamento já validado isoladamente em `spi_transmite_dados_tb.v`, agora reproduzido dentro do sistema completo.

Essa testbench tem uma particularidade importante em relação à distribuição do tempo simulado, que fica bem evidente ao comparar as duas capturas. A simulação inteira dura pouco mais de duzentos e noventa e cinco vírgula nove milissegundos, mas a fase de aquisição pelos sensores, mostrada na primeira imagem, ocupa a esmagadora maioria desse tempo, enquanto a fase de extração pela interface SPI, mostrada na segunda imagem, acontece inteiramente dentro de uma janela de pouco mais de vinte microssegundos, posicionada nos últimos instantes da simulação. Por isso as duas capturas precisam ser vistas como complementares e em escalas completamente diferentes, já que tentar mostrar as duas fases numa única imagem faria a fase SPI, sendo tão mais curta, desaparecer por completo dentro da escala de milissegundos necessária para mostrar a fase de aquisição.

Na primeira imagem, cobrindo os primeiros onze milissegundos da simulação, o sinal `clk_pino` aparece como o já esperado bloco verde compacto e contínuo, dado o

período de trinta e sete nanossegundos. Os sinais `scl_i2c` e `sda_i2c`, por outro lado, permanecem completamente parados durante um pouco mais dos dois primeiros milissegundos, o que corresponde exatamente ao contador de reset interno do módulo `top_fpga`, que mantém toda a lógica em reset até contar sessenta mil ciclos de clock, o equivalente a cerca de dois vírgula dois milissegundos nessa frequência, antes de liberar a máquina de estados para começar a operar. A partir desse ponto, os dois sinais passam a mostrar uma atividade densa e praticamente contínua até o final da janela capturada, com `scl_i2c` pulsando de forma regular e `sda_i2c` alternando entre níveis conforme os bits de endereço, de registrador e de dado vão sendo transmitidos, entremeados pelos mesmos pequenos trechos em vermelho já observados em `fsm_ambiente_tb.v`, indicando os instantes de transição de direção do barramento compartilhado. Essa atividade contínua corresponde à configuração dos três registradores do BME280 seguida da primeira leitura em rajada dos oito bytes ambientais, e o sinal `leitura_concluida_ambiente`, apesar de estar entre os selecionados, não mostra nenhum pulso visível em nenhum ponto dessa janela, já que o primeiro ciclo completo de leitura só termina bem mais adiante, depois de a FPGA ainda precisar configurar o BH1750 e aguardar seu tempo de aquecimento, um intervalo sozinho responsável por boa parte dos duzentos e noventa e cinco milissegundos totais.

A segunda imagem, cobrindo uma janela de aproximadamente vinte e dois microssegundos entre duzentos e noventa e cinco mil novecentos e dezoito e duzentos e noventa e cinco mil novecentos e quarenta microssegundos, mostra exatamente essa desproporção de escalas: é apenas nesse trecho final, correspondente a uma fração ínfima do tempo total simulado, que qualquer atividade aparece nos sinais `cs_pino`, `sclk_pino`, `mosi_pino`, `miso_pino` e `led_azul`, já que são justamente os sinais da interface SPI, que só entra em ação depois que todo o processo de aquisição pelos sensores está encerrado. Dentro dessa janela é possível identificar pelo menos duas transações completas, cada uma delimitada por uma queda breve de `cs_pino`, com `mosi_pino` mostrando o pequeno grupo de pulsos do byte de comando concentrado logo no início de cada transação e `miso_pino` mostrando o trem bem mais longo de pulsos correspondente aos cento e quatro bits restantes do quadro de resposta, o mesmo padrão assimétrico já observado em `spi_transmite_dados_tb.v`. O sinal `led_azul` aparece subindo durante uma dessas transações, indicando que o comando enviado naquele momento foi reconhecido como válido. O barramento `dados_para_spi[95:0]` aparece com o valor fixo `548A107F3C206B901A2B027B` estampado ao longo de toda a janela, e decompondo

esse valor em pares de dígitos hexadecimais chega se exatamente aos doze bytes conferidos no terminal, o que confirma visualmente que o registrador de doze bytes já está estabilizado com o valor final antes mesmo de a primeira transação SPI capturada nessa imagem começar, incluindo o valor 0x027B da unidade do solo, que é o valor já tratado pelo filtro daquele canal, e não o valor bruto originalmente injetado no sensor simulado.

Evidências

Código

https://github.com/Gabriel5595/Projeto-de-Bloco-Sistemas-Digitais-Embarcados-26E2_5-

Vídeo

[Vídeo de Demonstração.mp4](#)

Imagens do Hardware

[Imagens do Hardware](#)

Imagens das Waves

[Waves](#)